

Table of Contents

1 🚀 Start

- 1.1 Introduction
- 1.2 Deployment and Launch

2 ✂️ How-To

- 2.1 Review
- 2.2 Creating a system module
- 2.3 Writing a custom script

3 🧠 Core

- 3.1 Life cycle and initialization (Boot)
- 3.2 Isolation and Safety (KMS, Zero Trust)
- 3.3 Security Center (аудит и изоляция)
- 3.4 State Manager (LifecycleState)
- 3.5 Fault tolerance (Emergency Gate)
- 3.6 Event bus (Redis Event Bus)
- 3.7 Access control (RBAC)
- 3.8 Concept Machine Spirit

4 📖 API

- 4.1 Interfaces and Protocols (Schemas)
 - 4.1.1 Base classes (BaseClass, API)
 - 4.1.2 Script and module interfaces
 - 4.1.3 Crypto Helper (HMAC/nonce)
- 4.2 Core Components (Core)
 - 4.2.1 Configuration and session managers
 - 4.2.2 Notification Manager
 - 4.2.3 Task Watcher (restart background cycles)
 - 4.2.4 Auth Manager
 - 4.2.5 Network Controller
 - 4.2.6 Postgres Manager
 - 4.2.7 Log Distributor
 - 4.2.8 Scheduler & Backup Scheduler
 - 4.2.9 Health Checker & Metrics Collector
 - 4.2.10 System Log Processor
 - 4.2.11 Number and text processing (number_processor)
 - 4.2.12 Authentication subsystem and RBAC
- 4.3 Services

- 4.3.1 Integration services (Bridges)
- 4.3.2 Isolated launch (Isolated Runner)
- 4.3.3 Energy consumption (Power Daemon)
- 4.3.4 Registration of external services
- 4.3.5 Registration Service data schema
- 4.3.6 Debug Intent Service
- 4.3.7 Remote Config Manager
- 4.3.8 Archive Service

4.4 Inputs/Senders

- 4.4.1 Telegram (Input/output interface)
- 4.4.2 Web UI & API Gateway (обзор)
- 4.4.3 Web API Reference
- 4.4.4 Deployment and Launch
- 4.4.5 CLI (Input/output interface)

4.5 Models and NLP

- 4.5.1 Intent Processor

4.6 Emotional Engine (Identity)

- 4.6.1 Emotional Engine

4.7 Embedded scripts

- 4.7.1 System Scripts Overview

5 Diagrams

- 5.1 Task life cycle
- 5.2 Power Management
- 5.3 User input flow

6 Status

- 6.1 Architectural design
- 6.2 Data sheet
- 6.3 Development plans

1 🚀 Start

1.1 S.U.B.O.T.A Project Documentation

S.U.B.O.T.A. - System Unit of Binary Operations & Technical Assistance

"Omnissiah save us from external APIs, for the local container is pure."

Welcome to the central documentation hub for **S.U.B.O.T.A** - a private, autonomous digital assistant. This documentation is structured according to the Diátaxis framework to ensure clarity and maintainability.

1.1.1 🧠 Architectural Pillars

- Absolute Autonomy (Local-First):** The system operates entirely on local hardware without dependencies on third-party cloud APIs to guarantee data privacy.
- Microservices & Event-Driven Topology:** Built using Python 3.13+, FastAPI, Redis, and PostgreSQL, leveraging isolated runtimes and clear service boundaries.
- Security & Policy Enforcement:** Strict separation of core system components from user-submitted scripts via secure proxies and dynamic access control (Zero Trust).
- Deterministic State & Emotional Engine:** System reactions and transitions are driven by an objective 3D state space and strict personality scales.

1.1.2 🌐 Navigation Matrix

Category	Description	Key Links
Tutorials	Step-by-step guides to set up your local development environment from scratch.	• <i>Environment Installation (In Progress)</i>
How-To Guides	Practical, goal-oriented recipes to extend system functionality.	• Add Custom Script • Register Core Module
Reference	Low-level technical specifications, API contracts, and database schemas.	• Base Classes & API • Interfaces
Explanation	Deep dives into high-level architecture, design decisions, and concepts.	• Design Document • Technical Specification

1.1.3 📊 Release & Progress Tracking

- Current Iteration Plan:** Documented in (russian only) [WORKPLAN_3_RELEASE.md](#).
- Implementation Status:** Live tracking available in (russian only) [PROGRESS.md](#).

4.4.4 Component Under Development

Omnissiah System Note

This section of the technical specification describes a component that is currently fixed in the architectural design plans, but does not yet have a physical implementation in the S.U.B.O.T.A. v3.0 core codebase.

1.2.1 Component Status

- **Current state:** Interface design / Writing stub modules.
- **Development branch:** `dev-v3-architecture`
- **Related release plan:** [WORKPLAN_3_RELEASE](#)

1.2.2 Architectural Boundaries

Once development of the module is complete, this page will expand into full technical documentation, including:

1. **API Contract:** A description of method input/output signatures, data types, and validation schemas (Pydantic models).
 2. **Component Lifecycle:** The initialization logic of the component within the core's overall boot cycle (Boot Manifest).
 3. **Security Context:** Access rights within the RBAC model and the `RestrictedProxy` restriction matrix.
 4. **Event Topology:** A list of events generated and handled on the `Redis Event Bus`.
-

1.2.3 What Should a Developer Do?

If you critically need the functionality of this component to write user scripts or related system modules:

1. **Use Mock objects:** During development, call stub interfaces that return deterministic test data.
2. **Check the task plan:** Current timelines and dependencies for the implementation of this module can be found in the document  Status -> Development Plans ([WORKPLAN_3_RELEASE](#)).
3. **Do not create hidden dependencies:** It is forbidden to build core logic that bypasses isolation by relying on this component's future structure. All interactions must strictly follow the Object Capabilities (OCaps) pattern.

2 How-To

2.1 S.U.B.O.T.A. Tutorials & Guides

Welcome to the practical guide section. This space contains step-by-step tutorials designed to help you install, configure, and extend the system locally.

2.1.1 Learning Path

1. Getting Started (Tutorials)

Focused on initial setup and onboarding. Learning by doing.

- [Environment Installation](#) (*In Progress*) — How to clone, configure local environment variables, and spin up the core docker containers.

2. Development & Extension (How-To Guides)

Goal-oriented recipes for extending system functionality.

- [Create Custom Script](#) — Step-by-step guide on writing isolated execution scripts managed by the system proxy.
 - [Register Core Module](#) — How to implement a new native microservice and connect it to the event-driven system core.
-

2.1.2 General Requirements

Before starting any tutorial, ensure your local station meets the minimum stack compliance:

- **OS:** Ubuntu Linux (22.04+ recommended)
- **Runtime:** Python 3.13+
- **Containerization:** Docker & Docker Compose
- **Storage:** Local instances of Redis & PostgreSQL

2.2 Creating a SUBOTA System Component

This document describes the step-by-step process for creating and integrating a new functional module into the SUBOTA core architecture, based on three base classes.

2.2.1 Step 1: Determining the Abstraction Level (Classification)

Before writing code, choose the correct base class. A mistake at this stage will break the isolation and the Zero Trust security model:

1. **BaseClass** — reserved exclusively for critical core components without which the system cannot start (Example: `SecurityCenter`). `tier=0` or `tier=1`. This class is not intended for interaction with the event bus or other APIs. It is not used directly in application development.
2. **SubotaInternalService** — for modules operating within the isolated perimeter. They have no direct access to external networks, sensors, or third-party APIs.
3. **SubotaExternalService** — for gateways, sensor drivers, and integrations with the outside world. Contains strict checks (signature validation, nonce control, protection against Replay attacks).

2.2.2 Step 2: Creating the File Structure

Each module must be isolated in its own directory inside `modules/`.

```
modules/
├── my_awesome_service/
│   ├── __init__.py
│   └── my_service.py      # Component class code
```

2.2.3 Step 3: Implementing the Component Class

Let's look at creating an internal component by inheriting from `SubotaInternalService`. The class is required to implement the asynchronous lifecycle methods `on_load` and `on_unload`.

```
import asyncio

from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from core.inner_core.schemas.payload_schema import SubotaMessage

from core.inner_core.base_class import ModuleStatus
from core.inner_core.internal_service import SubotaInternalService
from core.logger_manager import get_logger
logger = get_logger(__name__)

BASE_NAME = 'my_internal_service'
USER_ID = 'system'
SOURCE = 'core'
GROUP_NAME = f'group_{BASE_NAME}'
CONSUMER_NAME = f'consumer_{BASE_NAME}_1'

class MyInternalService(SubotaInternalService):
    def __init__(self):
        # Base class initialization. tier and configuration are read by the core.
```



```

        super().__init__(source=SOURCE, service_name=BASE_NAME, user_id=USER_ID,
config_name="my_service_conf", tier=2)
        self.loop_name = 'My_Internal_Service Loop'
        self.status = ModuleStatus.STARTING

    async def on_load(self):
        """Called by the core when the module is activated.
        This is where queue initialization, bus connection, and worker startup happen.
        """
        # MANDATORY CALL, without it dependencies will not be set and the module will crash with an error,
e.g.: 'NoneType' object has no attribute 'publish'
        await super().on_load()
        # Example of obtaining the core's infrastructure queue from the interfaces
        self.task_watcher = self.interfaces.task_watcher

        # It is recommended to move all the logic that should run in the module into a separate method,
e.g.:
        await self._initialize()

        self.status = ModuleStatus.ACTIVE
        logger.info("MyInternalService started successfully.")

    async def on_unload(self):
        """Called by the core when the component is deactivated or emergency-stopped.
        Must fully release resources and stop background tasks to save energy.
        """
        await self.task_watcher.stop_watching(name=self.loop_name) # stop the running loop in the system
TaskWatcher
        await self.task_watcher.stop_watching(name=self.loop_name + ' Subscription')
        self.status = ModuleStatus.OFFLINE
        await super().on_unload()
        logger.info("MyInternalService stopped, resources cleaned up.")

    async def _initialize(self):
        # Start a background async worker using the system TaskWatcher
        self.task_watcher.start_watching(self._worker_loop, name=self.loop_name)

        # Example of subscribing to events on the event bus
        try:
            await self.event_bus.ensure_group_exists(group_name=GROUP_NAME)
        except Exception as e:
            logger.error(f"Error while creating group: {e}")
            return

    async def _event_handler(event_type: str, payload: dict, ts: float)->None:
        if event_type == 'my_event': # ts validation can be done manually or simply ignored
            await self._do_things(payload)

    self.task_watcher.start_watching(
        lambda: self.event_bus.subscribe_to_group(
            group_name=GROUP_NAME,
            consumer_name=CONSUMER_NAME,
            handler_func=_event_handler,
        ),
        name=self.loop_name + ' Subscription'
    )

@SubotaInternalService.check_payload
async def _do_things(self, payload:'SubotaMessage'):
    pass

    async def _worker_loop(self):
        """Internal business logic loop of the component."""
        while self.status == ModuleStatus.ACTIVE:
            try:
                # Simulating a payload/workload
                await asyncio.sleep(10)
            except asyncio.CancelledError:
                break

```

```
except Exception as e:
    logger.error(f"Error in MyInternalService loop: {e}")
```

Important for security (Zero Trust): Any anomalies, unauthorized access attempts, or corrupted data (Malformed Payload) inside methods protected by the `check_payload` or `secure_event` decorators are immediately sent to `SecurityCenter` via the synchronous `report_incident` method.

```
```python self.interfaces.security_center.report_incident( actor="my_internal_service",
violation_type="MALFORMED_PAYLOAD", target="incoming_message_handler")
```
```

2.2.4 Step 4: Declaration in `system_map.yaml`

The core will not initialize a module unless it is described in the system map. Add the following to the `config/system_map.yaml` file in the module's directory:

```
modules:
  my_internal_service:
    class_path: "modules.my_awesome_service.my_service.MyInternalService"
    tier: 2
    capabilities: ["transit"]
    deps: # module names
      - "event_bus"
    methods: # model methods
      event_bus:
        - "publish"
        - "ensure_group_exists"
        - "subscribe_to_group"
      task_watcher:
        - "start_watching"
        - "stop_watching"
```

2.2.5 Step 5: Validation and Launch

1. Place the module directory in `modules/`.
2. Start the system core.
3. Check the system log:
4. The core must read `system_map.yaml` and build the dependency tree according to `tier`.
5. Calling `on_load()` must transition the module's status to `ModuleStatus.ACTIVE`.
6. If memory consumption limits are exceeded or errors are generated, `SecurityCenter` initiates isolation via the `SECURITY_ISOLATION` event.

Architectural Postscript (Critique of the Current Implementation)

When developing application modules, take into account the following system limitations of the current base class code:

1. **Shared state in methods:** When inheriting, remember that `BaseClass.__init__` initializes the `methods` and `deps` dictionaries as mutable default Python arguments. Do not modify `self.methods` directly bypassing the core, to avoid side

effects across the entire pool of modules.

2. **Locks in External services:** The current implementation of the protection decorator in

`SubotaExternalService.secure_event` uses a global lock on the entire method. Avoid long synchronous or blocking operations inside `ExternalService` methods, so as not to paralyze the processing of incoming queues from other sensors.

2.3 Script development for ScriptRunner v3

2.3.1 1. General Provisions and Placement

Each system script is assigned an isolated directory in the root of the scripts folder. The entry point for the current version of the loader is the strictly fixed filename `v3.py`.

```
../scripts/  
└─ **script_name**/  
    └─ v3.py # Fixed entry point
```

Registering in the System

For a script to be processed by `ScriptRunner`, it must be explicitly registered in the `system_config.yaml` configuration file. If this entry is missing, `ScriptRunner` completely ignores the script's directory.

2.3.2 2. Required environment constants

Every `v3.py` file must contain the following module-level global variables:

- `GROUP_NAME` (`str`): The name of the consumer group for coordinating events in `event_bus`.
- `CONSUMER_NAME` (`str`): A unique consumer identifier within the group.
- `ANSWER_TYPE` (`str`): The response handling type. The value `"assistant_response"` is reserved and guarantees automatic output of the execution result to the source from which the command originated.
- `SCRIPT_NAME` (`str`): The technical (human-readable) script name used for logging.
- `SCRIPT_SHORT_DESCRIPTION` (`str`): A brief description of the script's purpose.
- `SCRIPT_LONG_DESCRIPTION` (`str`): A detailed description of the script's logic.

2.3.3 3. Script Class Architecture and Interfaces

The script must inherit from the base class `SubotaInternalService`.

Initialization and Dependency Contract (`__init__`)

In the constructor, the `methods` dictionary describes the interfaces requested from `ScriptRunner` for dependency injection into the `self.interfaces` object. `event_bus` and `task_watcher` are mandatory for working with the message stream and tasks.

```
def __init__(self):  
    super().__init__(  
        service_name='script_runner',  
        source='script_runner',  
        user_id='script',  
        config_name=None,  
        deps=[],  
        methods={  
            'event_bus': ['publish', 'ensure_group_exists', 'subscribe_to_group'],  
            'task_watcher': ['start_watching', 'stop_watching'],  
            'infra': ['dry_run', 'redis_client']
```

```
}  
)
```

⚠ Critical security note: If `ScriptRunner` does not have a method requested in the `methods` dictionary, the initialization system raises an `AttributeError` exception. This event is classified as a security incident, aborts initialization, and sends a structured log entry to the audit subsystem.

2.3.4 4. Script Lifecycle

```
[Load] —> on_load() —> [super().on_load()] —> run() —> [Registration in task_watcher]  
                                     |  
[Send response] <— [publish_to_event_bus] <— [@check_payload] <—
```

`on_load()` Method

Used for the initial setup of infrastructure. The call `await super().on_load()` must be executed at the very beginning of the method for correct dependency injection. `await self.run()` is called at the end of the method.

`run()` Method (Func name not fixed)

Responsible for registering the consumer group and passing the dispatcher function to `task_watcher` for non-blocking listening of the event bus.

`on_unload()` Method

Called when the script is deactivated. Must explicitly stop loop tracking via `task_watcher.stop_watching()` and release resources.

2.3.5 5. Contract for Processing and Sending Responses

Any target script method that processes an incoming message and returns a response must strictly follow a three-stage pattern:

Stage 1: Incoming Payload Validation

The processing method must be protected by the `@SubotaInternalService.check_payload` decorator. This decorator automatically intercepts the incoming dictionary (`dict`) from the event bus, validates it, and converts it into a strictly typed `SubotaMessage` object. The method receives this object as the `payload` argument.

Stage 2: Business Logic

Inside the method, data is read from `payload` (for example, determining the source via `payload.source`), target actions are performed (Redis queries, report generation), and the `payload` object itself is modified (writing the response text to `payload.text` , distinguishing output destinations in `payload.data['content']['destinations']`).

Stage 3: Publishing the Response via Base Class Methods

Sending the result to the system must be done **strictly** through the parent class method `self.publish_to_event_bus`. Direct publication via third-party interfaces is prohibited if standard response output is required.

```
await self.publish_to_event_bus(answer_type=ANSWER_TYPE, payload=payload.model_dump())
```

The `payload` is passed as a serialized Pydantic model of the message via `.model_dump()`.

2.3.6 6. Full Reference Template (`v3.py`)

```
import json
import logging
from typing import TYPE_CHECKING
from core.inner_core.security.subota_internal_service import SubotaInternalService
from core.inner_core.base_class import ModuleStatus
from core.logger_manager import get_logger

if TYPE_CHECKING:
    import redis.asyncio as redis
    from schemas.protocols import TaskWatcherProtocol
    from core.inner_core.schemas.payload_schema import SubotaMessage

logger = get_logger(__name__)

# Global metadata and environment constants
SCRIPT_NAME = "System Status"
SCRIPT_SHORT_DESCRIPTION = "Displays the current system status and all loads"
SCRIPT_LONG_DESCRIPTION = "Displays complete system information from the Redis cache."

BASE_NAME = 'system_status'
GROUP_NAME = f'group_{BASE_NAME}'
CONSUMER_NAME = f'consumer_{BASE_NAME}_1'
ANSWER_TYPE = "assistant_response" # Reserved response output type

class SystemStatusScript(SubotaInternalService):
    def __init__(self):
        super().__init__(
            service_name='script_runner',
            source='script_runner',
            user_id='script',
            config_name=None,
            deps=[],
            methods={
                'event_bus': ['publish', 'ensure_group_exists', 'subscribe_to_group'],
                'task_watcher': ['start_watching', 'stop_watching'],
                'infra': ['dry_run', 'redis_client']
            }
        )
        self.loop_name = 'SystemStatusScript Loop'
        self.status = ModuleStatus.STARTING

    async def on_load(self):
        await super().on_load() # Mandatory interface injection
        self.task_watcher: 'TaskWatcherProtocol' = self.interfaces.task_watcher
        self.redis_client: 'redis.Redis' = self.interfaces.infra.redis_client
        self.dry_run: bool = self.interfaces.infra.dry_run
        await self.run()

    async def run(self):
        try:
            await self.event_bus.ensure_group_exists(GROUP_NAME)
        except Exception as e:
            logger.error(f"Error while creating consumer group: {e}")
```

```

        return

    async def stream_event_dispatcher(event_type: str, payload: dict, *args) -> None:
        if event_type == 'SYSTEM_STATUS':
            if self.dry_run:
                logger.info(f"DRY RUN: {payload}")
            else:
                await self.process_and_respond(payload=payload)

    # Non-blocking listener startup via task_watcher
    self.task_watcher.start_watching(
        lambda: self.event_bus.subscribe_to_group(
            group_name=GROUP_NAME,
            consumer_name=CONSUMER_NAME,
            handler_func=stream_event_dispatcher
        ),
        name=self.loop_name
    )
    self.status = ModuleStatus.ACTIVE

@SubotaInternalService.check_payload
async def process_and_respond(self, payload: 'SubotaMessage'):
    """
    Pattern for processing a message and sending a response.
    payload is automatically converted from dict to SubotaMessage by the decorator.
    """
    source = payload.source or None
    if not self.redis_client:
        return

    # --- Script business logic (Start) ---
    raw_cache = await self.redis_client.hgetall("subota:system:status:cache")
    status = {}
    for key, value in raw_cache.items():
        try:
            status[key] = json.loads(value)
        except Exception:
            status[key] = value

    # Modify payload for the system response
    payload.text = f"Status generated. Objects in cache: {len(status)}"

    # Route the response back to the sender
    if source == 'voice_input':
        payload.data['content'] = {'destinations': ['cli', 'telegram']}
    else:
        payload.data['content'] = {'destinations': [source]}
    # --- Script business logic (End) ---

    # Mandatory system call to send the response via base class methods
    await self.publish_to_event_bus(answer_type=ANSWER_TYPE, payload=payload.model_dump())

async def on_unload(self):
    await self.task_watcher.stop_watching(self.loop_name)
    self.status = ModuleStatus.OFFLINE
    logger.info(f'[SystemStatusScript]: disconnected')
    await super().on_unload()

```

3 🧠 Core

3.1 SYSTEM LIFECYCLE MANAGEMENT: BOOT & SHUTDOWN MANIFEST

3.1.1 1. ARCHITECTURAL LIFECYCLE PHASES (LIFECYCLE MAP)

The core existence process is deterministic and divided into 4 isolated stages. Transition to the next stage is impossible without 100% fulfillment of the conditions of the previous one.



Phase 0: BIOS (Pre-component startup)

- **Area of responsibility:** Synchronous context of `main.py` prior to `asyncio.run()` initialization.
- **Processes:**
 - Initialization of the atomic logger to track the startup.
 - Parsing of the primary YAML via `AppConfig` (`system_config.yaml`).
 - Initialization of `KeyManagerSystem` (KMS). Reading the physical master key file (`master.key`).
- **Release Gate (Crypto-control):** Verification of the digital signatures of the configuration maps `system_map.sig` and `isolated_runner_map.sig` via the public key `INFRA_PUBLIC_KEY` . If the signatures are compromised or the token is removed (`SecurityTokenRemovedError`), the process terminates with exit code `1` before the asynchronous runtime unfolds.

Phase 1: Infrastructure Setup (Core Preparation)

- **Area of responsibility:** Initialization of `Core` in the asynchronous runtime `main_runner()` .
- **Processes:**
 - Creation of asyncio synchronization primitives: the system bus `__inner_core_queue` , critical notifications queues `critical_notifications_queue` , and the shutdown event flag `core_stop_event` .
 - Registration of basic infrastructure wait flags for external bridges (`bridge_{name}_initialized`).

Phase 2: Tiered Boot (Step-by-step Cascaded Startup)

- **Area of responsibility:** `Core.boot_system()` method.
- **Processes:**
- **Digraph Validation:** The `_validate_dependency_graph()` method checks the `modules` map for cyclic dependencies and the correctness of the layer connection declarations.
- **Sequential Launch of Tiers (0 -> 5):** Modules are initialized strictly layer by layer. Level $N+1$ modules do not begin loading until all level N modules have transitioned to `ModuleStatus.ACTIVE` status.
- **ModuleAPI Injection:** Controlled import and passing of a stripped-down core access interface to the module (injection of only the methods allowed in `system_map.yaml`).

Layer Topology (Tiers) based on `system_map.yaml`

| Level (Tier) | Functional Group | Key Components | Layer Success Criterion |
|--------------|--|---|--|
| Tier 0 | Security components and low-level DBs | <code>security_center</code> , <code>secret_manager</code> , <code>system_state_manager</code> , <code>emergency_buffer</code> , <code>redis_connection_manager</code> , <code>task_watcher</code> , <code>redis_auth_manager</code> | Availability of local Redis, readiness of the emergency log buffer. |
| Tier 1 | System buses and transport | <code>event_bus</code> , <code>postgres_manager</code> , <code>auth_manager</code> , <code>telegram_bot</code> , <code>web_socket_connection_manager</code> , <code>network_controller</code> | Stable connection to PostgreSQL, active local network stack. |
| Tier 2 | Internal coordinators and authorization services | <code>log_distributor</code> , <code>context_manager</code> , <code>scheduler_manager</code> , <code>backup_scheduler</code> , <code>emergency_service</code> , <code>rbac_manager</code> , <code>debug_intent_service</code> , <code>bridge_auth_service</code> | Readiness of the permission distribution subsystem and bridge authorization services. |
| Tier 3 | Validators, session managers, and input drivers | <code>health_checker</code> , <code>metrics_collector</code> , <code>cli_input</code> , <code>web_ui_input</code> , <code>session_manager</code> , <code>system_log_processor</code> , <code>telegram_input</code> , <code>scheduler</code> , <code>external_registration_service</code> , <code>gate_keeper_service</code> , <code>power_daemon_service</code> | Launch of the task scheduler, activation of power sensors and primary data input interfaces. |
| Tier 4 | Isolated computing units and background managers | <code>intent_processor</code> (Isolated), <code>notification_manager</code> , <code>emotional_engine</code> , <code>archive_service</code> , <code>remote_config_manager</code> , <code>external_bridge_heartbeat</code> | Launch of the NLP processor in an isolated context, startup of external bridge watchdogs. |
| Tier 5 | High-level business logic and API controllers | <code>script_runner</code> , <code>web_controller</code> | Readiness of the user automation script execution environment. |

- **Stabilization:** After the successful loading of Tier 5, `SystemStateManager` transitions the system to `LifecycleState.READY` status. From this moment on, processing of external incoming events is opened.

Phase 3: Graceful / Emergency Shutdown (System Stop)

- **Area of responsibility:** `Core.stop()` method and `main_runner()` finalizer.

- **Shutdown Triggers:** `core_stop_func.execute()` invocation, setting of `core_stop_event`, or a hardware KMS `SecurityTokenRemovedError` event (key extraction).
 - **Processes:**
 - Changing the global status to `LifecycleState.SHUTDOWN`.
 - **Reversible Unloading of Layers (Tier 5 -> Tier 0):** Modules are stopped strictly in the reverse order of their startup, to prevent a situation where a low-level data bus (Tier 1) is destroyed before a script depending on it (Tier 5).
 - Calling the asynchronous `on_unload()` method for each module with a strict `_safe_unload` timeout of 10 seconds per module. If the timeout is exceeded, the module is forcefully terminated.
 - Cleaning up database connection pools and logging the core shutdown event.
-

3.1.2 2. CRITICAL VULNERABILITY ANALYSIS AND DESIGN LIMITATIONS (TECH DEBT)

The retry mechanism during module startup (`_load_module_with_retry`, Phase 2) is an intentional architectural decision, not technical debt: it protects against one-off dependency failures and allows a module and its consumer to be placed within the same `Tier`. This same mechanism handles the Redis ACL configuration for a module: if the background `credits` task (creating an ACL user via `redis_auth_manager`) fails to complete before the module attempts to use its Redis access in `on_load()`, the operation will fail with an authorization error—this exception will be caught in `_load_module_with_retry`, the module will receive an `ERROR` status, and it will go into a restart on general grounds. No separate race condition arises here.

Below is a real limitation of the current implementation that requires attention:

1. **KMS initialization occurs without reading `system_config.yaml`:** In `main_runner()` (`main.py`), the configuration for `KeyManagerSystem` is created as follows:

```
python
config = AppConfig()
kms = KeyManagerSystem(config=config)
```

`AppConfig()` is a standard pydantic model without automatic file loading; calling it without arguments means that each field takes the default value declared in the schema code (`core/config_manager.py`). The actual reading of `system_config.yaml` (via the `load_config()` function) occurs only once—inside `Core.__init__` (`Core.load_config()`), already **after** the KMS has been initialized.

- **Consequence:** `KeyManagerSystem` practically always uses the defaults of `KeyMenegerSystemConfig` (`label`, `key_filename`, `docker_mode`, `check_interval`), even if different values are specified in `system_config.yaml`. This currently does not manifest because the defaults in the code match the current values in the YAML—but in the event of any discrepancy in the future, configuration changes for KMS simply will not be applied, without any explicit error or warning.
 - **Limitation:** If KMS parameters need to be configured via `system_config.yaml`, `main_runner()` must pass the result of the actual `load_config()` to `KeyManagerSystem`, rather than an empty `AppConfig()`.
-

3.1.3 3. CORE CONFIGURATION PARAMETERS AND TIMINGS

Parameters governing core behavior, fixed in `system_config.yaml`:

- **Core version:** `3.0.0-alpha`.
- **Docker mode:** `False` (The core runs directly on the host system; `key_meneger_system.docker_mode: False`).

- **KMS Watchdog polling interval:** 2 seconds (`key_meneger_system.check_interval: 2`). The presence of a valid physical token is checked every 2 seconds.
 - **Module unload timeout (`_safe_unload`):** 10 seconds . The maximum time allocated for a module to complete transactions and save its state.
 - **TaskWatcher interval:** 10 seconds (`task_watcher.check_interval: 10`). Maximum number of restarts for a crashed module: 5 (`task_watcher.max_restarts: 5`).
-

The document records the current actual state of the codebase. Configurations are synchronized with the architectural description.

3.2 Isolation and Safety (KMS, Zero Trust)

3.2.1 1. KMS Specification and Integrity Control

The system core functions in an immutable configuration mode after startup. Integrity control of infrastructure maps is implemented at the KMS (Key Management Service) initialization stage before other core services are launched.

1.1. Cryptographic Verification (Ed25519)

All manifest configuration files (`system_map.yaml` , `isolated_runner_map.yaml`) must have an accompanying signature file (`.sig`) generated using the infrastructure private key.

- **Mechanism:** KMS reads the raw public key `INFRA_PUBLIC_KEY`.
- **Canonization:** YAML data is deserialized via `yaml.safe_load` , after which it is brought into a strict canonical form via `json.dumps(..., sort_keys=True).encode('utf-8')` to eliminate markup issues.
- **Verification:** Signature validation is carried out by an asymmetric algorithm via `nacl.signing.VerifyKey` . If the signature does not match, a `BadSignatureError` exception is generated, completely blocking core initialization.
- **Schema:** After cryptographic verification, the data is passed to Pydantic models (`SystemMap` , `ModuleManifest`) to validate structure and types.

```
[File .yaml] -> yaml.safe_load -> json.dumps(sort_keys=True) -> Canonical byte-code
                                                         |
[File .sig]  -----> VerifyKey.verify()
```

1.2. Emergency Destruction Mechanism (Kill Switch)

The system implements a fail-fast strategy upon violation of the secure perimeter.

- **Trigger:** Physical extraction of the hardware key token, deletion of the session file, or an explicit command from `SecurityCenter` .
- **Reaction:** A `SecurityTokenRemovedError` exception is generated.
- **Action:**
 - The top-level interceptor initiates an immediate call to `sys.exit(1)` .
 - The current state of RAM structures is destroyed by the operating system.
 - Database write operations (PostgreSQL, Redis, ChromaDB) are forcefully terminated. The integrity of unallocated transactions is ignored in favor of preventing data leakage.

3.2.2 2. Capability-Based Security Model (Zero Trust at the Code Level)

In-process module interaction is regulated through proxy shells implementing the principle of least privilege (POLP). Direct invocation of core methods from application modules is prohibited.

2.1. Access Restriction via `RestrictedProxy`

- During initialization, each module receives a custom instance of `RestrictedProxy` rather than a direct reference to the core API (`CoreAPI`).

- `RestrictedProxy` encapsulates `ModuleAPI` and intercepts all attribute accesses via `__getattrute__`.
- Access to methods is allowed only if the called method is explicitly declared in the trusted and signed `system_map` for the specific module identifier.
- Objects returned through the proxy are cast to immutable types (where possible) to prevent "core state mutation via reference" type attacks.

2.2. Audit and Incident Logging

- Any attempt to access an undocumented method or property is logged by the internal method `_report_violation`.
- The incident record is sent directly to `SecurityCenter`, bypassing standard general-purpose logging queues.
- Repeated incidents of unauthorized access transition the module to `COMPROMISED` status, followed by isolation or termination of its worker process.

3.2.3 3. Process Isolation and Inter-Process Communication (IPC)

To execute potentially dangerous, heavy, or unstable tasks (CLI interfaces, NLP parsing, third-party integrations), the system utilizes an isolated worker architecture.

3.1. Isolation via `IsolatedRunner`

- **Process Creation:** The launch of an isolated module is carried out strictly via the `multiprocessing.get_context("spawn")` context. The use of `fork` is prohibited to prevent copying the memory state of the parent process (core) into the child.
- **Lifecycle:** The isolated process performs initialization in a closed loop (`isolated_worker_loop`). At startup, the worker must send a readiness signal (`READY`) to the parent process within `WORKER_START_TIMEOUT`. Otherwise, the process is forcefully destroyed via `.kill()`.

3.2. Data Exchange Protocol (IPC)

- Interaction between the Core and `IsolatedRunner` goes through a duplex channel `multiprocessing.Pipe`.
- Since the standard `Pipe` uses Python serialization mechanisms, a strict restriction is imposed at the development convention level: transmitted messages must contain only primitive data types (strings, numbers, dictionaries/lists with primitives) easily convertible to JSON format. The transmission of custom class instances and executable code is prohibited.

3.2.4 4. Network ACL and Storage Isolation (Redis/Postgres)

Each module has a strictly limited context within the databases used, determined at the manifest validation stage.

4.1. Centralized Bus and Access Control in Redis

The system rejects decentralized streams for each module to avoid network overengineering and excessive CPU resource consumption. Interaction is built via the Mediator pattern.

- **Unified Transport Bus:** All application modules conduct inter-module interaction exclusively via `event_bus` methods. Direct writing by application modules to arbitrary Redis streams is prohibited.

- **Isolation of Dedicated Spaces (Selective Namespacing):** Access to direct, isolated keys and specialized Redis streams is granted exclusively to a limited pool of authorized infrastructure and system services: `event_bus`, `network_controller`, `context_manager`, `health_checker`, `metrics_collector`, `session_manager`, `power_daemon_service`, `emotional_engine`, `script_runner`. Each of these services has access strictly to its target subset of keys.
- **Control by Core:** The application module does not possess credentials for direct access to Redis. Any attempt by an unauthorized module to bypass the `event_bus` and execute a direct command to the Redis server is intercepted at the descriptor control level and regarded as a critical violation of the security perimeter.

4.2. DBMS Command Restriction

- Depending on the module's role, the proxied methods restrict the available set of operations.
- Application modules (Tier 4/5) are deprived of rights to execute destructive operations or bulk selections (for example, `KEYS *` or `FLUSHDB` in Redis, direct DDL queries in Postgres). Access is provided only to specialized interfaces (`hash_rw`, `kv_rw`) that strictly control command arguments.

3.2.5 5. Known Design Vulnerabilities and Technical Debt (Design Flaws)

Listed below are the architectural compromises made in the current release of the system, which must be taken into account during operation and resolved in future iterations.

5.1. Direct Import Vulnerability (`RestrictedProxy` Bypass)

- **Description:** The `RestrictedProxy` mechanism effectively protects the core from modules if they use the interfaces provided to them. However, being in the same OS process, any application Python code can perform a direct import of internal core modules: `import core.security` or `import core.kms`.
- **Risk:** Complete bypass of Zero Trust, unauthorized access to private core methods without registration in `SecurityCenter`.
- **Current Mitigation:** Control at the level of static code analysis (linters prohibiting the import of core packages in application modules) and mandatory code review (Code Review).

5.2. Use of Unsafe Serialization in IPC (`multiprocessing.Pipe`)

- **Description:** Data transmission between the main core process and the `IsolatedRunner` via `multiprocessing.Pipe` is based on the standard Python mechanism, which utilizes the `pickle` module under the hood.
- **Risk:** If an isolated worker process (e.g., the BERT NLP module) is compromised via a vulnerability in a third-party library (HuggingFace/Tokenizers), an attacker could tamper with the serialized data in the `Pipe`. Upon deserialization (calling `recv()`) in the parent core process, remote code execution (RCE) will occur with core privileges.
- **Current Mitigation:** Strict control over the types of data sent. Restricting the range of libraries used in workers. Shifting IPC to pure JSON over Unix Domain Sockets or protected `AsyncIO` IPC is planned for future releases.

3.3 Security Center Specification (Audit & Isolation)

3.3.1 1. Architectural Context and Initialization

The `SecurityCenter` component is a critical element of the zero level of the core hierarchy (`tier=0`) of the S.U.B.O.T.A. system core. It inherits from the `BaseClass` base class and is responsible for collecting security incidents, maintaining forensics logs, threat scoring, and making decisions regarding the isolation of compromised modules.

1.1. System Environment Constants

When registering within the system and initializing the context, the component utilizes the following hard-coded markers:

- `BASE_NAME` (Service name): `'security_center'`
- `SOURCE` (Source): `'core'`
- `USER_ID` (Operator identifier): `'system'`
- `PROTOCOL_VERSION` (Interaction protocol version): `"1.0"`
- `config_name`: `None` (dedicated configuration YAML file is absent, parameters are hardcoded at the class level).

1.2. Lifecycle and Interface Binding

Module state management is governed by a strict asynchronous pipeline:

- Instantiation:** The object is created with a `ModuleStatus.STARTING` status. Internal data structures are initialized: `_threat_scores` (threat scores dictionary) and `_active_bans` (set of blocked actors).
- Loading (`on_load`):** An internal asynchronous incident buffer `_incident_buffer` (`asyncio.Queue`) is deployed. Infrastructure dependency injection (Dependency Injection) is performed via the `interfaces` object:
- `inner_core_queue` — internal queue of the trusted core circuit.
- `emergency_queue` — emergency notifications queue.
- `alarm_mode` — global alarm mode flag.
- `core_stop_event` — core event loop stop event.
- `kms` — key management system protocol (`KeyManagerSystemProtocol`). A background task for clearing scores by TTL (`_ttl_cleanup_loop`) is launched. The status transitions to `ModuleStatus.ACTIVE` .
- Unloading (`on_unload`):** The background cleanup task is forcefully cancelled (`_ttl_task.cancel()`), the status is reset to `ModuleStatus.OFFLINE` .

3.3.2 2. Violation Matrix and Security Metrics

The component carries out weighted threat scoring based on a predefined penalty point matrix (`VIOLATION_RULES`).

2.1. Violation Weights (Penalty Grid)

When an incident is logged, points are assigned to the actor according to the violation type:

| Violation type (violation_type) | Penalty point | Incident description |
|-----------------------------------|---------------|--|
| UNAUTHORIZED_ACCESS | 100 | Attempted unauthorized access to a resource |
| USER_BLOCKED | 100 | Action taken by a blocked user |
| HMAC_MISMATCH | 70 | MAC verification error (packet integrity violation) |
| REPLAY_ATTACK | 50 | Detection of a re-sent, previously valid packet |
| UNSUPPORTED_PROTOCOL | 50 | Unsupported version of the interaction protocol |
| MALFORMED_PAYLOAD | 50 | Distorted data structure that failed primary parsing |
| UNKNOWN_KEY_ID | 50 | Request utilizing an unknown key identifier |
| RATE_LIMIT | 30 | Exceeding the rate limit of requests on endpoints/interfaces |
| CLOCK_SKEW | 10 | Time skew of the event timestamp relative to core time |
| Any other event | 10 | Default penalty for unclassified anomalies |

2.2. Thresholds and Time Limits

- `SECURITY_THRESHOLD` = `3000` — Critical threshold of the total penalty for an actor, upon reaching which the automatic blocking/isolation procedure for the module is initiated.
- `SECURITY_TTL` = `300` (5 minutes) — Time interval after which degradation (deduction) of the actor's accumulated penalty points occurs, provided no new violations have been recorded.
- `REPLAY_CALL_TIMEOUT` = `30` — Maximum time window for packet replay checks.

3.3.3 3. Incident Processing Algorithms

Interaction with the component is divided into a lightweight synchronous ingestion interface and heavy asynchronous analysis loops that prevent blocking the main thread of execution (Event Loop).



```

/ \
[_enforce_ban()]      (Exit)
- Add to ban list
- Send SECURITY_ISOLATION
  to inner_core_queue

```

3.1. Incident Ingestion (`report_incident`)

External components call the synchronous method `report_incident(actor, violation_type, target)`. The method performs primary filtering:

1. Converts `violation_type` to uppercase (`UPPER`).
2. Checks the string for the presence of critical stop words (`CRITICAL_VIOLATION_WORDS`): ['EXIT', 'OFF', 'DROP', 'DELETE', 'DISCONNECT', 'TERMINATE', 'KILL', 'SHUTDOWN', 'RELOAD', 'RESTART'].
3. If a stop word is found, control is instantly transferred, bypassing the main bus, to the `_handle_critical` method.
4. If the event is standard, it is placed into the `_incident_buffer` queue using the `put_nowait()` method. If the buffer overflows, a `Security incident buffer overflow!` error is generated in the `security_forensics` logger.

3.2. Emergency Response (`_handle_critical`)

Upon detecting destructive intent, the following actions are executed bypassing the standard pipeline:

- The event is immediately written to the forensics logger with a status of `CRITICAL`.
- A `SubotaMessage` envelope is generated with a unique asynchronous `nonce` (`uuid.uuid4().hex`).
- A routing dictionary is written into the message metadata: `{'destinations': ['telegram', 'cli'], 'ugrgency': 'critical'}`.
- The packet with the event type `CRITICAL_NOTIFY` is immediately broadcasted to `emergency_queue` to execute operator notification.

3.3. Analysis Worker (`_analyze_loop` and `_analyze_incident`)

The non-critical threat processing pipeline functions within a dedicated asynchronous task:

1. The incident extracted from the buffer is checked against the `_active_bans` list. If the actor is already blocked, processing terminates.
2. A record is created or updated for the actor within the `_threat_scores` structure, incrementing the counter for this specific type of violation.
3. The actor's total score (`total_score`) is increased by the penalty value from `VIOLATION_RULES`. The timestamp `last_update = time.time()` is updated.
4. If `total_score >= 3000`, the asynchronous execution procedure— `_enforce_ban(actor)` —is invoked.

4.4. Isolation Mechanism (`_enforce_ban`)

Upon reaching the distrust limit, `SecurityCenter` performs hardware isolation of the threat source:

- The actor identifier is entered into `_active_bans`.
- A warning is recorded in the forensics log: `Threshold reached. Banning module/actor: ...`

- A system message `SubotaMessage` is formed with the text `"Isolate actor {actor}"` and the control structure `data['content'] = {'action': 'isolate_module', 'target': actor}`.
- The packet with the header `SECURITY_ISOLATION` is sent to the internal trusted core queue `__core_inner_queue` for physical disconnection or moving the module to an isolated container.

4.5. Threat Degradation (`_ttl_cleanup_loop`)

To eliminate the accumulation effect of old errors (e.g., accidental network timeouts `CLOCK_SKEW`), a memory cleanup cycle is launched once every 300 seconds (`SECURITY_TTL`):

- An iteration over all recorded violators takes place.
- If more than 5 minutes have passed since the actor's last activity (`current_time - last_update`), its scoring map is completely removed from `_threat_scores`.
- The actor is removed from the `_active_bans` blacklist (amnesty upon expiration of time).

3.3.4 5. Audit and Logging (Security Forensics)

The `SecurityCenter` logging subsystem is divided into two independent contexts:

1. **Standard Logger (`logger`)**: Used to record routine core debugging messages.
2. **Specialized Forensics Logger (`sec_logger`)**: Bound to the `'security_forensics'` channel. Records of critical violations, facts of incident buffer overflows, and executed block commands are directed exclusively here. This log is isolated and serves as the foundation for subsequent immutable security audits of the system.

3.4 System State Manager Specification

3.4.1 1. Architectural Context and Area of Responsibility

The `SystemStateManager` component (module `core/inner_core/state_manager.py`) is the central state dispatcher of the S.U.B.O.T.A. system. Its main task is to provide thread-safe, fast access to current module health metrics, the system load level, and information security status.

The component functions at the zero level of trust (`tier=0`) of the core, has no external configuration files (`config_name=None`), and no external dependencies at the initialization stage (`deps=[]`).

3.4.2 2. Typology and State Space (State Matrix)

The system operates with a strictly typed set of enumerations (`Enum`) combined into a common type alias `StateTypes`. This matrix defines all possible deterministic phases of the ecosystem operation.

2.1. Global Lifecycle (`LifecycleState`)

Describes the top-level runtime status of the entire system core:

- `UNKNOWN` — Initial undefined state prior to the launch of the core logic.
- `STARTING` — Asynchronous phase of initialization and sequential cascaded startup of modules.
- `READY` — The core is fully stable, all services are loaded and ready to process operator tasks.
- `RESTARTING` — A soft restart process of the core components has been initiated.
- `SHUTDOWN` — Controlled system shutdown phase (Graceful Shutdown).

2.2. Infrastructure Service States (`ServiceState`)

Used primarily for databases and internal buses (PostgreSQL, Redis):

- `STARTING` — Period of connection pool establishment or socket verification.
- `NORMAL` — The service functions in normal mode, transactions complete successfully.
- `FAILOVER` — The service is unavailable. Redirection of data streams to isolated local buffers is taking place.
- `RECOVERY` — Connection is restored, offloading of accumulated buffers (WAL degradation) to the target storage is being performed.

2.3. Additional State Spaces

- `SystemLoadState` (Host Load):** `STABLE` (Normal), `HIGH_LOAD` (Elevated), `CRITICAL` (Denial of Service).
- `SecurityState` (Security Circuit):** `GREEN` (Standard), `YELLOW` (Anomaly Registration / Threat Scoring), `RED` (Active Actor Isolation / Alarm).
- `ExternalServiceState` (External Gateways/API):** `ONLINE` (Available), `OFFLINE` (Disconnected), `UNKNOWN` (Status undefined).
- `EmotionalState` (Identity Emotional Engine):** `STARTING`, `NORMAL`, `SHUTDOWN`.

3.4.3 3. Initialization and Startup Map (`_service_states`)

Upon creating an instance of the `SystemStateManager` class, it is assigned the `ModuleStatus.STARTING` status. The internal state storage is populated with hard-coded default values:

```
self._service_states: Dict[str, StateTypes] = {
    "postgres": ServiceState.STARTING,
    "redis": ServiceState.STARTING,
    "telegram": ExternalServiceState.OFFLINE,
    "discord": ExternalServiceState.OFFLINE,
    "web": ExternalServiceState.OFFLINE,
    "system_load": SystemLoadState.STABLE,
    "security": SecurityState.GREEN,
    "emotional": EmotionalState.STARTING
}
```

- **In the `on_load()` method:** The internal lifecycle transitions to the `LifecycleState.STARTING` phase, and the status of the component itself changes to `ModuleStatus.ACTIVE`.
- **In the `on_unload()` method:** The component status transitions to `ModuleStatus.OFFLINE`.

To isolate data recovery processes at the execution context level, the `is_restoring` variable (`contextvars.ContextVar`) is used, having a default value of `False`.

3.4.4 4. Application Programming Interface (API)

4.1. High-Speed Getters

To minimize overhead when accessing from critical sections of the core code, access to basic states is implemented via Python properties (`@property`). They perform direct extraction from the dictionary with end-to-end type casting via `typing.cast`:

- `postgres, redis` -> Return `ServiceState`.
- `telegram, discord, web` -> Return `ExternalServiceState`.
- `system_load` -> Returns `SystemLoadState`.
- `security` -> Returns `SecurityState`.
- `emotional` -> Returns `EmotionalState`.
- `lifecycle` -> Returns the current `LifecycleState`.

The universal `get_service_state(service_name)` method allows obtaining the status by a string key. If the key equals "lifecycle", the core status is returned; if the name is unknown, `NewServiceState.UNKNOWN` is returned. A full dump of the state map is available via `get_all_service_states()`.

4.2. State Management (Setters)

- `set_lifecycle(state: LifecycleState)`: Modifies the core status. If the new state differs from the current one, it records the transition and notifies local subscribers.
- `set_service_state(service_name: str, state: StateTypes)`: Updates the status of a specific service. Includes strict data type validation:

 **Critical limitation:** If the type of the passed `state` object does not match the type of the already existing state in the map for this service, the method will raise a `TypeError` exception, preventing logical pollution of the state matrix.

4.3. Dynamic Component Registration

The `add_new_service(service_name, state)` method allows application modules to dynamically add themselves to the `SystemStateManager` monitoring map during application runtime:

1. The service name is automatically cleaned of paths and extensions (`.split(".")[1].replace(".py", "")`).
2. If the name already exists but the data types of the new and old states differ, an integer index from `self.service_name_index` is appended to the name to avoid collisions, after which the counter is incremented.
3. The new element is written to the general map, triggers listeners, and returns a dictionary of the form `{cleaned_name: state}`.

3.4.5 5. Local Subscription System (Observer Pattern)

`SystemStateManager` provides a mechanism for synchronous notification of state changes without using the heavy asynchronous Redis Event Bus, which is critical during the Tier 0 stage when the Redis bus itself might be in `STARTING` or `FAILOVER` status.

- **Listener registration:** Performed via the `subscribe(callback: Callable)` method. The callback function is added to the internal `_listeners` list. Strictly synchronous callable objects are accepted.
- **Transition notification (`_trigger_listeners`):** Upon successful modification of any status, all registered functions are automatically invoked, passing three arguments: `callback(service_name, new_state, old_state)`.
- **Call fault tolerance:** The entire notification cycle is wrapped in a `try/except Exception` block, ensuring that a crash or error within the code of one of the subscribers in no way disrupts the operation of the state manager or interferes with the notification of the remaining listeners.

3.5 4. Fault Tolerance Architecture (Emergency Gate)

3.5.1 4.1. Emergency Save Gate Concept

The emergency circuit (Emergency Gate) is designed to prevent data loss and maintain system operational capacity in the event of failure of key level one and level two infrastructure components (PostgreSQL, Redis). It is based on the Circuit Breaker pattern combined with a local encrypted Write-Ahead Log (WAL).

System States (ServiceState)

- **NORMAL:** Requests are routed directly to the target database. The local buffer is not utilized.
- **FAILOVER:** A critical communication failure has been detected. Requests are transparently redirected to the local encrypted buffer for the calling code.
- **RECOVERY:** Connection is restored. A specialized service performs controlled buffer degradation (offloading) to the target DB in the background, maintaining a lock on direct writes.



3.5.2 4.2. Interception Mechanism (@emergency_gate)

All modifying methods (POST/PUT/DELETE operations, transactions, data bus writes) of the storage layer are wrapped with the `@emergency_gate('service_name')` decorator.

1. **Business Logic Transparency:** The calling module does not handle infrastructure crashes and always receives control back (returns `None` for POST methods).
2. **Fault Isolation:** The decorator responds exclusively to infrastructural network errors (`ConnectionRefusedError`, `TimeoutError`, connection resets). Business logic errors (uniqueness violations, SQL syntax) are passed straight up to avoid an infinite retry loop of toxic data.

3.5.3 4.3. Cryptographic Protection of the Local Buffer (Zero Trust)

Local storage of emergency dumps on the host's hard drive is potentially vulnerable to compromise. To ensure security compliance, an asymmetric data sealing scheme is applied:

- **Key Distribution:** The `EmergencyService` service generates a key pair at startup. The public key is broadcasted to `SystemStateManager` and is available to all services. The private key is held exclusively in the isolated memory of `EmergencyService`.
- **Sealing (Encryption):** An application decorator that intercepts control in failure mode encrypts the serialized call parameters (`args`, `kwargs`) using `nacl.public.SealedBox` via the public key.
- **Unsealing (Decryption):** Reading and deserializing data are only possible inside `EmergencyService` during the recovery process. No other component of the system, including the decorator itself, has the technical capability to read the buffer contents after it is written.

3.5.4 4.4. Local Buffer (Emergency Buffer)

A lightweight SQLite database deployed at the core level is used as an energy-efficient local storage.

- **Capacity Limitation (Overflow Protection):** To prevent host disk space overflow, the buffer has a strict limit (`max_records`). When the limit is reached, an eviction strategy is applied: old records are deleted in batches of 100 with a warning outputted to the `sys.stderr` system stream.
- **Quarantine (Poisoned Pills):** If a record causes a non-network execution error on the target DB during the recovery process, it is removed from the main queue and moved to an isolated `poisoned_pills` table, preserving the error stack trace for subsequent manual audit by an engineer. Repeated automatic attempts to execute this record are blocked.

3.6 5. Redis Event Bus

3.6.1 5.1. Streams Layout

The S.U.B.O.T.A architecture is built on the basis of Event-Driven Architecture. The single point of message exchange between services. External integrations is `RedisEventBus`, which utilizes the **Redis Streams** mechanism.

Each data stream has a strictly limited capacity to avoid uncontrolled consumption of the host's random-access memory (RAM).

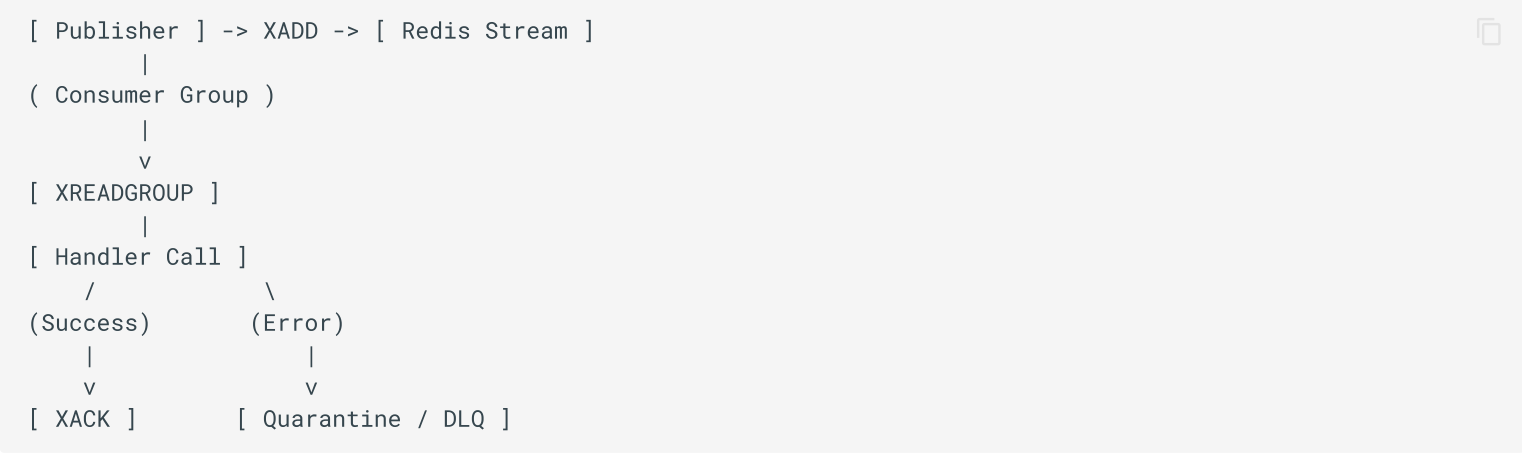
Stream Capacity Configuration

For each topic, a `maxlen` rotation strategy with approximation (`approximate=True`) is applied to minimize the load on the Redis CPU when evicting old records.

| Stream Name | Limit (Records) | Purpose |
|---|-----------------|--|
| <code>subota:internal:events</code> | 2000 | Main transport for internal core services (Internal Services). |
| <code>subota:external:events</code> | 1000 | Stream for integration with external agents and APIs. |
| <code>bridge:events</code> | 500 | High-frequency exchange within the bridge subsystem. |
| <code>subota:bridge:core</code> | 1000 | Incoming telemetry data from the bridge to the core. |
| <code>subota:core:bridge</code> | 100 | Low-frequency control commands for peripherals. |
| <code>subota:metrics</code> / <code>sensors:stream</code> | 50 / 100 | Fast-rotated caches of metrics and sensor states. |

3.6.2 5.2. Delivery Guarantees and Event Lifecycle

The system provides an **At-least-once** class delivery guarantee (minimum one delivery) through the use of Consumer Groups and the acknowledgment mechanism (`XACK`).



- Publishing:** The event is packed into a flat dictionary containing `type` metadata (string identifier), `ts` (POSIX timestamp), and `data` (JSON string).

2. **Reading:** Consumers are combined into named groups (`event_processors`). Each message is read atomically by a single worker in the group.
3. **Acknowledgment:** The acknowledgment operation (`XACK`) is sent to Redis strictly after the successful completion of the asynchronous handler execution. If the handler crashes, the message remains in the pending entries list (PEL) to avoid data loss.

3.6.3 5.3. Fault Tolerance and Degradation (Emergency Integration)

The `publish` event publication method is protected by the `@emergency_gate('redis')` circuit.

- When the Redis instance is unavailable (network timeout, service crash), control is intercepted by the decorator.
- The event is serialized, encrypted with an asymmetric key, and dumped into a local SQLite-based WAL.
- For the calling code, the infrastructure failure occurs imperceptibly; the isolation guarantees the continuity of system internal event generation.

3.7 6. Access Control (RBAC)

3.7.1 6.1. Architectural Overview

The access control subsystem in S.U.B.O.T.A is implemented based on a role model (Role-Based Access Control – RBAC). The `RBACManager` component functions as an internal core service of Tier 3 (`SubotaInternalService`).

The system differentiates access based on:

1. **Roles:** User groups (e.g., `admin`, `user`, `guest`). The `admin` role has unconditional access to all resources, bypassing checks via the permission table (hardcoding in the application layer, creating an architectural contradiction with the Zero Trust principle).
2. **Permissions:** Granular atomic rights to execute specific actions or access components.

3.7.2 6.2. Data Schema (DDL)

The current relational structure of the database is formed by a chain of Alembic migrations:

```
CREATE TABLE IF NOT EXISTS roles (  
    id SERIAL PRIMARY KEY,  
    name TEXT NOT NULL UNIQUE,  
    description TEXT  
);  
  
CREATE TABLE IF NOT EXISTS permissions (  
    id SERIAL PRIMARY KEY,  
    name TEXT NOT NULL UNIQUE,  
    description TEXT  
);  
  
CREATE TABLE IF NOT EXISTS role_permissions (  
    role_id INTEGER NOT NULL REFERENCES roles(id) ON DELETE CASCADE,  
    permission_id INTEGER NOT NULL REFERENCES permissions(id) ON DELETE CASCADE,  
    PRIMARY KEY (role_id, permission_id)  
);  
  
CREATE TABLE IF NOT EXISTS users (  
    id SERIAL PRIMARY KEY,  
    username TEXT NOT NULL UNIQUE,  
    hashed_password TEXT NOT NULL,  
    role_id INTEGER NOT NULL REFERENCES roles(id) DEFAULT 2, -- 'user'  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
    telegram_id BIGINT UNIQUE NULL -- Added by migration  
);
```

6.2.1. Initialization Data (Seed)

Data is deployed via initialization migrations:

- **Roles:** 1: `admin` (Full access), 2: `user` (Standard user), 3: `guest` (Read only).
- **Base permissions:** `script:edit`, `device:control`, `system:shutdown`, `user:admin_management`.
- **Default access matrix:**
- `admin`: receives all permissions, including `user:admin_management` via the `role_permissions` table.

- `user` : has strictly `device:control` .
- `guest` : has no permissions assigned in `role_permissions` (Strict Zero Trust).

3.7.3 6.3. Subsystem Components

6.3.1. RBACManager (`core/rbac_manager.py`)

The main service class responsible for direct interaction with the storage. At the current stage, caching is absent; each request goes to the DB.

Key methods:

- `get_user_role(user_id: int) -> str` : Returns the role name by the internal numeric user ID. If the user is not found, it returns the default `guest` role.
- `get_user_role_by_identifier(identifier: str) -> str` : Extracts the role by the string representation of the user's Telegram ID. If the user is absent, it sends a `publish_notification` event to the event bus (`RedisEventBus`) to be displayed in the `web_ui` and returns the `guest` role.
- `check_permission(role_name: str, permission_name: str) -> bool` : Checks for a link between the role and permission in `role_permissions` . Strings are converted to lowercase. For the `admin` role, it always returns `True` , bypassing the DB.

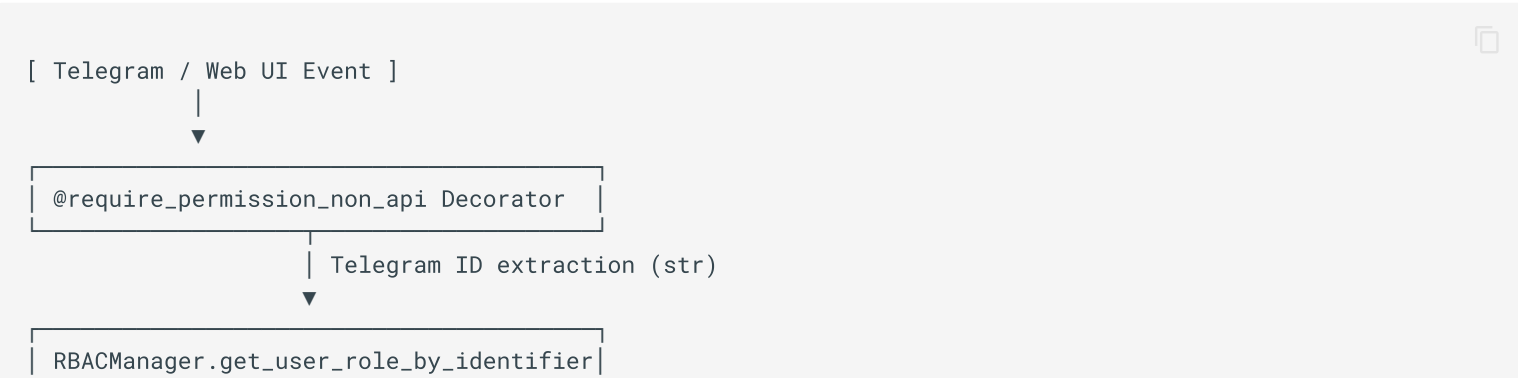
⚠ Critical performance requirement: The `get_user_role_by_identifier` method is required to perform explicit type casting `int(identifier)` before passing the parameter to the SQL query. Otherwise, the DBMS executes implicit Type Casting (`BIGINT = VARCHAR`) for each check, completely ignoring the `UNIQUE` index and overloading the core CPU on every incoming step in the event queue.

6.3.2. Access Interception Tools (`core/rbac_tools.py`)

The `require_permission_non_api` decorator is used to integrate access control into the application layer (Telegram handlers, CLI commands).

- **PermissionDenied(Exception)** **Exception:** Raised when unauthorized access is detected.
- **require_permission_non_api(rbac_manager, permission_name)** **Decorator:**
- Automatically extracts the `aiogram.types.Message` object from positional or keyword arguments of the function.
- Extracts `message.from_user.id` as a unique string identifier (`unique_identifier`).
- Requests the role and verifies the permission via `RBACManager` .

3.7.4 6.4. Permission Verification Lifecycle (Data Flow)





3.7.5 6.5. Current Architectural Conflicts and Technical Debt

- 1. Data Redundancy vs Hardcoded Logic:** The migration explicitly writes the `user:admin_management` permission for the `admin` role into the database. However, `RBACManager.check_permission` contains the hardcoded logic `if role_name == 'admin': return True`. This turns database records for the administrator into "dead data" and violates the concept of a clean role model. Moving this logic in favor of pure RBAC will be required to ensure Zero Trust.
- 2. Data Type Conflict (DBMS Layer vs Application Layer):** The `telegram_id` field in the DB has the `BIGINT` type, whereas decorators extract it as a `str`. The absence of strict type casting on the Python code side breaks the usage of PostgreSQL indexes.
- 3. Absence of Indexes on Foreign Keys:** The `role_id` fields in the `users` table and the linking fields in `role_permissions` lack explicit indexes, except for implicit primary key constraints. As the number of users grows, this will cause a linear performance degradation (Seq Scan instead of Index Scan) when processing each incoming event.

3.8 Machine Spirit Concept

3.8.1 1. Document Purpose

The document describes the architectural model of the "Machine Spirit" system for the SUBOTA 3.0 project.

Document purpose:

- formalize the already existing implementation of the identity system;
- describe the roles of the SUBOTA / OMNISSIAH / ALPHARIUS contexts;
- fix the context switching rules;
- define behavioral and logical restrictions;
- exclude conceptual elements that have no engineering value or contradict the existing architecture.

The document does not describe artistic lore as an independent entity. All styling is considered exclusively as:

- presentation layer;
- log formatting method;
- contextual response generation mechanism;
- semantic tagging of the system state.

3.8 2. Current Implementation State

At the time of writing the document, the identity system already contains:

3.8.2 2.1 Implemented Components

Structure:

```
identity/  
├─ emotional_engine.py  
├─ integrity_checker.py  
├─ persona_decorator.py  
├─ vector_math.py  
├─ db_handler.py  
├─ templates/  
│   ├── lifecycle_template.json  
│   ├── mood_templates.json  
│   ├── manifest_template.json  
│   ├── session_duration_template.json  
│   ├── timestamp_template.json  
│   ├── start_frazes_template.json  
│   └─ exit_frazes_template.json
```



3.8.3 2.2 Already Existing Mechanics

The system already implements:

- template generation of phrases;
- separation of responses by context;
- emotional vectors (`v_temper` , `v_stability` , `v_energy` , `v_affinity`);
- integrity events;
- lifecycle events;
- timestamp analysis;
- session duration analysis;
- state persistence via DB.

Machine Spirit is NOT a separate AI. It is an orchestration layer on top of:

- logs;
- contextual formatting;
- event interpretation;
- response decoration.

3.8 3. Architectural Model

3.8.4 3.1 Trinity Context Architecture

The system uses a three-context model:

| Context | Purpose | Level |
|-----------|--|----------------------|
| SUBOTA | Main user mode | Application Layer |
| OMNISSIAH | System diagnostic mode and infrastructure messages | Infrastructure Layer |
| ALPHARIUS | Security loop and override mechanism | Security Layer |

Contexts are NOT independent agents.

Each context represents:

- a set of templates;
- a set of behavioral rules;
- a set of formatting rules;
- an operation permission level.

3.8 4. SUBOTA Context

3.8.5 4.1 Purpose

SUBOTA is the main user context.

Used for:

- dialogues;
- everyday tasks;
- standard commands;
- ordinary interaction.

3.8.6 4.2 Behavior

Characteristics:

- neutral tone;
- high readability;
- absence of aggressive styling;
- minimal level of security noise.

3.8.7 4.3 Template Sources

Used:

- `start_frazes_template.json`
- `exit_frazes_template.json`
- `mood_templates.json`
- `lifecycle_template.json`
- `timestamp_template.json`
- `session_duration_template.json`
- `manifest_template.json`

Section:

```
"SUBOTA": { ... }
```



3.8 5. OMNISSIAH Context

3.8.8 5.1 Purpose

OMNISSIAH is an infrastructural diagnostic context.

Intended for:

- system logs;
- warning/error formatting;
- integrity reporting;
- lifecycle diagnostics;

- internal infrastructure reporting.

3.8.9 5.2 Restrictions

OMNISSIAH:

- does not manage the system;
- does not have blocking rights;
- does not change the execution flow;
- does not make security decisions.

It is a presentation layer for system events.

3.8.10 5.3 Usage

OMNISSIAH is used:

- in logs;
- during integrity scan;
- during system recovery;
- during resource degradation;
- during infrastructural errors.

Example:

```
[SPIRIT-REPORT]: The machine spirit of communication has become clouded. Performing reconnection.
```



In engineering terms, this corresponds to:

```
WARNING: MQTT connection lost. Retrying...
```



3.8 6. ALPHARIUS Context

3.8.11 6.1 Purpose

ALPHARIUS is a security context.

Used as:

- elevated security mode;
- security override layer;
- behavioral hardening layer.

3.8.12 6.2 Capabilities

ALPHARIUS can:

- block command execution;
- require re-authentication;
- enable restrictive policies;
- elevate the audit level;
- activate aggressive logging.

3.8.13 6.3 Restrictions

ALPHARIUS DOES NOT:

- engage in generating fake MQTT traffic;
- create honeypot devices;
- execute deception operations without a separate security module;
- implement security through obscurity.

Reason:

- conflict with latency contracts;
- resource consumption growth;
- impossibility of formal verification;
- high risk of self-induced degradation.

3.8 7. Context Manager

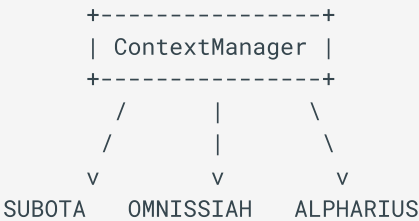
3.8.14 7.1 Purpose

ContextManager is the central orchestrator of the identity system.

It is responsible for:

- context switching;
- policy routing;
- template routing;
- response formatting;
- security escalation.

3.8.15 7.2 Base Scheme



3.8.16 7.3 Priorities

| Context | Priority |
|-----------|----------|
| ALPHARIUS | Highest |
| OMNISSIAH | Medium |
| SUBOTA | Default |

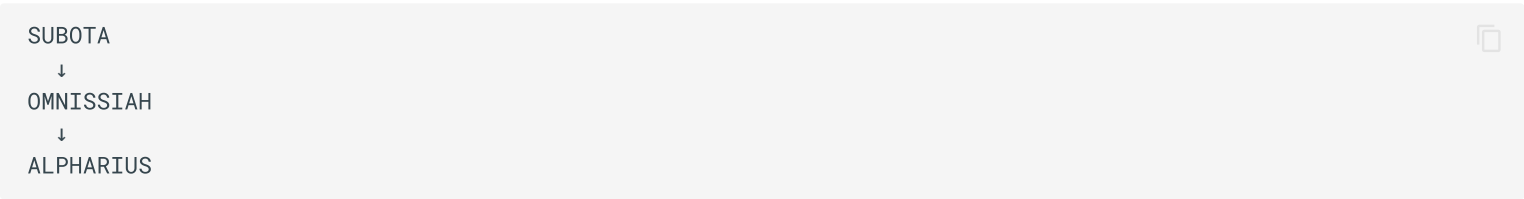
ALPHARIUS has override rights over SUBOTA.

3.8 8. FSM Architecture

3.8.17 8.1 State Machine

The identity system is implemented as an FSM.

Base states:



In practice:

- OMNISSIAH can work in parallel as an infrastructure overlay;
- ALPHARIUS is a security override state.

3.8 9. Context Switching Rules

3.8.18 9.1 Switching Principles

Context switching occurs:

- manually;
- via events;
- via security escalation.

3.8.19 9.2 Trigger Table

| Event | New Context | Action |
|------------------------------|-------------|----------------------------|
| Normal startup | SUBOTA | Standard mode |
| Integrity warning | OMNISSIAH | Diagnostic reporting |
| Integrity violation | ALPHARIUS | Security escalation |
| Multiple auth failures | ALPHARIUS | Sensitive actions block |
| Tampering detection | ALPHARIUS | Restriction mode |
| High error level | OMNISSIAH | Infrastructure diagnostics |
| User command "Alpha mode" | ALPHARIUS | Explicit switch |
| Security incident resolution | SUBOTA | Return to normal mode |

3.8 10. Return from ALPHARIUS

3.8.20 10.1 Reasons for Necessity

ALPHARIUS must not be a permanent state.

Reasons:

- aggressive logging overhead;
- UX restriction;
- false positives growth;
- resource consumption increase.

3.8.21 10.2 Return Conditions

Return is possible under the following conditions:

| Condition | Requirement |
|-------------------------------|-------------|
| Explicit user command | Yes |
| MFA / re-authentication | Yes |
| Timeout without new incidents | Optional |
| Successful integrity scan | Yes |

| Condition | Requirement |
|--------------------------|-------------|
| Error-rate stabilization | Yes |

3.8 11. Device Status Model

3.8.22 11.1 Purpose

Device statuses are a semantic layer over standard operational states.

This is NOT a separate security system.

3.8.23 11.2 Status Table

| Semantic Status | Operational Meaning |
|-----------------|-----------------------------|
| Loyal | ACTIVE / VERIFIED |
| Shrouded | MASKED / LIMITED VISIBILITY |
| Apostate | BLOCKED / INVALIDATED |
| In Meditation | SLEEP / LOW POWER |

3.8.24 11.3 Usage Rules

Statuses:

- are used in UI;
- are used in logs;
- are used in diagnostic reporting.

Statuses DO NOT:

- replace ACL;
- replace RBAC;
- replace authentication.

3.8 12. Integrity System

3.8.25 12.1 Basis

The integrity subsystem is already implemented via:

- `integrity_checker.py`
- `manifest_template.json`

3.8.26 12.2 Integrity Events

Supported events:

| Event |
|-----------------------------|
| INTEGRITY_CORE_MODIFIED |
| INTEGRITY_IDENTITY_MODIFIED |
| INTEGRITY_NEW_SKILL |
| INTEGRITY_DATA_LOSS |
| INTEGRITY_FIRST_SCAN |

3.8.27 12.3 System Reactions

Depending on the context:

- SUBOTA → neutral message;
- OMNISSIAH → diagnostic report;
- ALPHARIUS → security-oriented interpretation.

3.8 13. Emotional Engine

3.8.28 13.1 Purpose

EmotionalEngine is not an emotion simulation.

It is:

- contextual weighting system;
- adaptive response modifier;
- behavioral variance layer.

3.8.29 13.2 Vectors

Used:

v_temper
v_stability
v_energy
v_affinity



3.8.30 13.3 Vector Purpose

| Vector | Purpose |
|-------------|---------------------------|
| v_temper | Aggressiveness/reactivity |
| v_stability | State stability |
| v_energy | Reaction intensity |
| v_affinity | Friendliness/loyalty |

3.8 14. Noise Model

3.8.31 14.1 Current State

The noise-layer concept is preserved only as:

- abstraction level;
- rate variability;
- response variability.

The system DOES NOT generate mass artificial MQTT noise.

Reasons:

- risk of self-DoS;
- latency growth;
- QoS violation;
- degradation of embedded environments.

3.8.32 14.2 S_factor

S_factor is preserved as a behavioral intensity coefficient.

Not used as a false traffic generator.

3.8.33 14.3 Mode Table

| Mode | S_factor | Behavior |
|-------------------|----------|--------------------------|
| Standard | 0.1 | Normal mode |
| Elevated | 0.5 | Enhanced audit |
| Security Critical | 0.9 | Maximum security context |

3.8 15. HydraWhisper

3.8.34 15.1 Purpose

HydraWhisper is a lightweight response decorator.

3.8.35 15.2 Function

Randomly:

- adds secondary phrases;
- changes presentation layer;
- creates minor stylistic variance.

3.8.36 15.3 Restrictions

HydraWhisper:

- does not affect execution flow;
- does not affect security;
- does not affect routing;
- is not used in decision making.

3.8 16. Logs and Styling

3.8.37 16.1 Principles

Styling:

- must not hide real errors;
- must not replace raw logging;
- must not interfere with debugging.

3.8.38 16.2 Requirements

Each stylized log must have:

| Field | Requirement |
|------------------------|-------------|
| Raw event | Required |
| Timestamp | Required |
| Severity | Required |
| Context | Required |
| Human readable variant | Optional |

3.8.39 16.3 Example

[WARNING][OMNISSIAH]
MQTT connection lost.
Retry attempt: 3

Stylized variant:

[SPIRIT-REPORT]: Машинный дух связи помутился.
Совершаю повторное соединение.

3.8 17. What is Excluded from the Architecture

3.8.40 17.1 Excluded Elements

The following concepts ARE NOT part of the architecture:

Timestamp steganography

Reason:

- security through obscurity;
- lack of cryptographic strength;
- impossibility of audit.

Fake devices / phantom emitters

Reason:

- conflict with latency contracts;
 - load growth;
 - MQTT broker degradation;
 - complexity of diagnostics.
-

False positive response injection

Reason:

- observability violation;
 - risk of hiding real attacks;
 - impossibility of safe maintenance.
-

3.8 18. Final Model

Machine Spirit is:

- contextual orchestration system;
- identity formatting layer;
- security-aware response architecture;
- event interpretation system.

Machine Spirit IS NOT:

- a standalone AI;
- a separate security engine;
- a deception platform;
- a cyber warfare system.

The main task of the system is to:

- give the infrastructure a consistent contextual identity;
 - improve observability;
 - separate operational modes;
 - centralize behavioral routing.
-

3.8 19. Planned Extensions

3.8.41 19.1 ContextManager implementation

Planned:

- centralized FSM router;
- policy-based switching;

- async-safe context transitions.
-

3.8.42 19.2 Security integration

Planned:

- integration with security incident bus;
 - escalation policies;
 - MFA hooks;
 - audit event routing.
-

3.8.43 19.3 Logging integration

Planned:

- unified spirit formatter;
- structured logging compatibility;
- correlation-id support;
- tracing integration.

4 📖 API

4.1 Interfaces and Protocols (Schemas)

4.1.1 7. Base Classes and API Schemas (reference/base_classes.md)

7.1. Architectural Overview of the Component Model

The S.U.B.O.T.A component model relies on a three-level inheritance hierarchy of service classes and strict validation of structured data via Pydantic schemas. The system divides services based on runtime environment and network perimeter trust into **Internal** and **External**. Interaction between them is typed using message protocols and security envelopes.

7.2. Specification of Base Classes

The class hierarchy is designed to extend responsibilities from an abstract module description to the implementation of defense mechanisms against network attacks.

7.2.1. BaseClass (core/inner_core/base_class.py)

The abstract foundation for all system components. It defines the lifecycle and metadata structure (module passport).

- **Lifecycle Management:** Declares two mandatory asynchronous methods: `on_load()` (initialization, interface binding) and `on_unload()` (orderly shutdown, graceful shutdown).
- **Module State:** Fixes the status via `ModuleStatus` (`OFFLINE` , `STARTING` , `ACTIVE` , `ERROR`). The initial state during initialization is strictly `OFFLINE` .
- **Interface Container:** Uses a `BaseInterfaces` object for dynamic dependency injection (Dependency Injection).

7.2.2. SubotaInternalService (core/inner_core/security/subota_internal_service.py)

The base class for trusted core services of the system. It operates within a secure perimeter under the assumption that the calling code has already passed primary authorization.

- **Event Management:** Directly interacts with the data bus via `RedisEventBus` , which is obtained from the interface container at the moment `on_load()` is called.
- **Message Assembly:** The `_build_message` method encapsulates the logic of creating a `SubotaMessage` Pydantic model, automatically generating a cryptographic `nonce` (via `uuid.uuid4().hex`) to prevent duplication in logs and for tracing.
- **Inbound Flow Validation:** The `@check_payload` decorator performs a basic check of compliance between the inbound dictionary structure and the `SubotaMessage` model, suppressing validation exceptions and logging the incident.

7.2.3. SubotaExternalService (modules/subbota_service.py)

An heir to `SubotaInternalService` , designed to operate in an untrusted environment (external integrations, Telegram bridges, third-party service APIs). It contains a comprehensive layer of active defense.

- **Cryptographic Signature (HMAC):** Computes a SHA-256 HMAC from a JSON string payload sorted by keys (`sort_keys=True`). Keys are dynamically requested from `KeyManagerSystem` (`kms`) based on `key_id` or a `_bridge` suffix.
- **@secure_event Decorator (End-to-End Filtering Layer):** Performs 6 verification stages for each incoming call:
- **Envelope Validation (`SecureEnvelope`):** Verifies the data structure. On error, a report is sent to `SecurityCenter` .

- *User Blocking*: Instantly drops the request if the `key_id` is in the local `blocked_users` block cache.
- *Clock Skew Check (Time Shift)*: Compares the system time with the message `timestamp`. Tolerance is 10 seconds (narrowed down to 5 seconds in alarm mode). Protection against Replay attacks from old dumps.
- *Replay Protection (Nonce Replay)*: Verifies the uniqueness of the `key_id:nonce` pair against the `used_nonces` cache (TTL 30 seconds).
- *Constant-Time Signature Verification*: Compares hashes via `hmac.compare_digest` to eliminate timing attacks.
- *Rate Limiting*: Checks a sliding request window (defaulting to 10 requests per 1.0 second) via the history of timestamps for each `key_id`.
- **Event Bus Isolation Layer**: The `publish_to_event_bus` method is strictly blocked by the `@deprecated` macro and raises a `RuntimeError` while sending an incident report to the infosec center upon a call attempt. The module must explicitly call either `publish_to_internal_event_bus` or `publish_to_external_event_bus`.
- **Attack Analysis and Alarm Mode**: When 5 security errors are detected within a 10-second window from 3 unique sources, the service transitions to alarm mode. This narrows verification time limits and initiates the sending of forensics logs to the emergency queue `emergency_queue`.
- **Data Sanitization**: Before dumping suspicious payloads into the error log (`error_dump`), keys from the `SENSITIVE_KEYS` list (`password`, `token`, `secret`, etc.) are recursively replaced with `***REDACTED***`, and strings longer than 50 characters are truncated.

7.3. Specification of Data Schemas and Protocols (Schemas)

Data exchange and manifest registration are strictly regulated by Pydantic models with a prohibition on unauthorized fields (`extra='forbid'`) where it is critical for security.

7.3.1. Core Transport Level Models

| Schema / Class | Field Description and Features |
|----------------|--------------------------------|
| | |

| `SubotaMessage`

| Main data container. Contains `text`, unique sender `key_id`, `nonce`, perimeter `source`, `source_user_id`, `session_id`, `timestamp` (autogenerated via `time.time()`), and an arbitrary `data` dictionary.

|| `SecureEnvelope`

| Security envelope for the external perimeter. Settings: `extra='forbid'`. Fields: `hmac_signature: str` and a nested valid `payload: SubotaMessage` model.

|

7.3.2. Peripheral Sensor Models

| Schema / Class | Field Description and Features |
|----------------|--------------------------------|
| | |

| `SensorMessage`

| Payload from a physical device. Settings: `extra='forbid'`. Mandatory fields: `device_id`, `timestamp`, `nonce`, `command` string, and a strictly typed `data` parameter dictionary.

SecureSensorEnvelope

| Envelope for raw messages from brokers (MQTT). Settings: `extra='forbid'`. Fields: `device_id`, `hmac_signature`, string serialized `payload` (for deferred deserialization), and `topic`.

|

7.3.3. Topology and System Manifest Models

| Schema / Class | Field Description and Features |
|----------------|--------------------------------|
| | |

SystemMap

| Root schema for the entire system topology configuration. Contains a flat `modules: Dict[str, ModuleManifest]` dictionary for core assembly by `ModuleManager`.

StreamPermission

| Granular description of rights to a Redis stream. Contains the stream name (`stream`) and an array of permitted actions access (e.g., `['publish']`).

ModuleManifest

| Module deployment passport. Describes `class_path`, core `tier`, `capabilities`, dependencies `deps`, methods `methods`, `isolated` flag, internal `events` map, `config_name`, and a list of `event_bus_permissions: Optional[List[StreamPermission]]` for bus access control.

|

7.4. Critical Analysis, Vulnerabilities, and Architectural Debt

1. Architectural Bottleneck and DoS Vector via `asyncio.Lock`

In the `SubotaExternalService.secure_event` decorator, the critical section of validation is entirely wrapped in `async` with `instance._lock: .` Inside this section, when an attack is detected or a signature fails, `await instance._register_security_error(envelope)` is called, which under certain conditions executes `await self._send_alarm()`, containing `await self.emergency_queue.put(...)`.

- **Risk:** If the emergency queue is full or processed slowly, the current worker completely blocks `_lock`. As a result, the service stops accepting and validating *any* requests from other (legitimate) users. The validation of synchronous `TTLCache` caches does not require an asynchronous lock, since context switching within `asyncio` happens only on `await`.
- **Solution:** Remove the global `_lock` from the validation method. Operations with `TTLCache` are atomic within a single execution thread of the event loop. Move the `_register_security_error` call outside the synchronous checks and execute it as a background task via `asyncio.create_task` so as not to block the inbound packet processing pipeline.

2. Clock Skew / Replay Attack Vulnerability due to Default Pydantic Factories

In the `SubotaMessage` schema, the `timestamp` field is declared with `default_factory=lambda: float(time.time())`.

- **Risk:** If an external adversary or an incorrectly implemented client sends a packet *without* specifying a `timestamp`, Pydantic on the receiver side (core service) will automatically substitute the current server system time at the moment of deserialization. As a result, the `clock_skew = abs(now - timestamp)` check will always equal ≈ 0 , completely neutralizing the protection against Replay Attacks. The attacker can repeatedly send the intercepted packet without a time field, and the system will accept it as new.
- **Solution:** Make the `timestamp` field in `SubotaMessage` strictly mandatory (`timestamp: float = Field(...)`) without default values at the parsing stage of inbound perimeter messages.

3. Lack of Static Typing (Service Locator Pattern / Anti-pattern)

The `self.interfaces` field is initialized with an empty placeholder class `BaseInterfaces`, and then manually populated dynamically with bindings in heirs: `self.interfaces.kms`, `self.interfaces.event_bus`.

- **Risk:** This completely blinds static analyzers (`mypy`, `pyright`). The developer is forced to use inline comments `#type: ignore` or constructs like `self.kms: 'KeyManagerSystem' = self.interfaces.kms` to bypass linter errors. Any typo in the name of the injected interface will cause an `AttributeError` exclusively at runtime.
- **Solution:** Abandon the dynamic `BaseInterfaces` object. Use explicit declarations of expected protocols (structural typing via `typing.Protocol`) or a strictly typed interface dataclass for each core tier level.

4. Hardcoded Security Configuration (Feature Creep / Technical Debt)

All security limits (`MAX_ERRORS`, `ERROR_WINDOW`, `RATE_LIMIT_REQUESTS`) are placed into global constants of the `modules/subbota_service.py` file. A comment `#TODO: [SubotaService] move to config` is left directly in the code.

- **Risk:** Impairs deployment flexibility. It is impossible to change the alarm trigger threshold for a specific bridge without modifying the core source code and rebuilding the container.
- **Solution:** Move the specified parameters into a Pydantic module configuration model and read them from `self.config` at the initialization moment during the `on_load()` phase.

4.1.2 Script and module interfaces

1. Core Level: Module Isolation (`module_api.py`)

At the system core level, component isolation is implemented through a strict access control mechanism.

RestrictedProxy Mechanism (Core)

- **Zero Trust Architecture:** Access to core objects is completely controlled through a proxy class with rigid visibility restrictions.
 - **Reflection and Mutation Blocking:** Data structure protection is ensured by the explicit declaration of `__slots__`, which precludes the use of the standard `__dict__`.
 - **Attribute Control:** Interception of any read attempts is performed via an overridden `__getattr__` method.
 - **Immutability:** State modification is blocked at the property management level via `__setattr__`.
 - **Security Audit:** Any attempt at unauthorized access or restriction bypass is immediately registered, sending a report to the `security_center` via a `SC_report_incident` call.
-

2. Subsystem Level: Dynamic Loading and Script Lifecycle (`child_api.py`)

The child component level is designed for executing isolated scripts and tasks with dynamic execution context management.

Dynamic Pipeline (Dynamic Loading)

- **Runtime Import:** Loading of child modules is performed dynamically during application runtime using the built-in `importlib.import_module` mechanism.
- **Identification:** The target class invocation is initialized based on a string path `class_path`.

Child Component Contract (Duck Typing)

Each pluggable script must strictly conform to the following interface:

- **Constructor:** Class initialization is performed without passing arguments (`child_class()`).
- **Authority Declaration:** The script must declare a `dependencies` attribute as a list of strings to specify its required dependencies.
- **Visibility Restriction:** The script must declare a `methods` attribute (a restriction dictionary) to filter the available functionality.
- **Entry Point:** Asynchronous startup of the component logic is performed through a mandatory call to the `await instance.on_load()` method.

Fault Tolerance Ensurance (Resilience)

- **Restart Strategy:** Management of failures during script initialization is encapsulated within the `load_children_with_retry` function.

- **Control Parameters:** Repeated loading attempts are regulated by the `max_retries` and `retry_delay` configuration parameters.

RestrictedProxy Mechanism (Subsystem/Script)

The current runtime isolation at the script level has an implementation distinct from the core:

- **Method Copying:** Access restriction is implemented by iterating over the passed `methods` list and dynamically binding permitted attributes to the proxy instance using `setattr(self, m_name, attr)`.
- **State Storage:** The local proxy uses a standard `__dict__` dictionary and does not contain `__slots__` restrictions.
- **Call Validation:** Overriding of the `__getattr__` method is absent.
- **Error Handling:** When attempting to access an undeclared method, a standard Python runtime `AttributeError` exception is generated.
- **Audit:** Integration with the `security_center` is absent; interception and logging of infosec incidents are not performed at this level.

4.1.3 Cryptographic Utilities Reference (Crypto Helper)

1. General Information and Purpose

The `core/inner_core/security/crypto_helper.py` module provides a set of standalone functions for protecting data during network transmission and storage. The component implements three layers of defense:

1. **Key Derivation:** Transforming text passphrases into cryptographically strong keys bound to a device's hardware identifier.
 2. **Symmetric Encryption:** Stream encryption (key streaming/gamming) of arbitrary data dictionaries without using heavy external dependencies.
 3. **Integrity and Authenticity Control (Authentication):** Signing ciphertext using an **Encrypt-then-MAC** scheme to ensure guaranteed protection against data tampering and replay attacks.
-

2. Specification of Algorithms and Functions

2.1. Key Derivation (`generate_crypto_key`)

The function converts a simple text passphrase (`passphrase`) into a high-entropy cryptographic key formatted as a hex string.

- **Algorithm:** PBKDF2 (Password-Based Key Derivation Function 2).
- **Underlying Hash:** SHA-256 (`hashlib.pbkdf2_hmac`).
- **Iteration Count:** 100,000.
- **Salt:** A unique device identifier (`device_uid`), encoded in UTF-8. Binding to `device_uid` guarantees that the exact same password on different physical hosts will yield completely different keys.

2.2. Stream Cipher (`_crypt_xor_stream`)

An internal private method that implements symmetric encryption using key streaming (analogous to CTR / counter mode).

- **Cryptographic Basis:** HMAC-SHA256.
 - **Block Size:** 32 bytes (corresponds to the SHA-256 output length).
 - **Operational Mechanism:**
 - Data is sliced into 32-byte blocks.
 - The sequence number of each block is converted into a 4-byte counter value with Big-Endian byte order (`byteorder='big'`).
 - For each block, a unique block of pseudorandom keystream is calculated using the formula: `HMAC(key_bytes, iv + counter, hashlib.sha256)`.
 - The resulting keystream is applied to the original text via a bitwise XOR (`^`) operation.
 - **Advantage:** The function is completely symmetric — re-applying the same keystream to the ciphertext restores the original plaintext.
-

3. Serialization and Transport Envelope Protocol

3.1. Payload Encryption (`encrypt_payload`)

Accepts a string hex key and an arbitrary Python dictionary (`payload_dict`). The packet packaging process occurs through the following stages:

1. **Pre-serialization:** The dictionary is serialized into a compact JSON byte string with mandatory deterministic key sorting (`sort_keys=True`). This eliminates differences in hashes caused by varying field orders in Python memory.
2. **IV Generation:** Using a cryptographically secure operating system generator `os.urandom(16)` , a unique 16-byte initialization vector (IV) is created.

 **Critical Security Requirement:** The initialization vector (IV) must be strictly unique for each transmitted data packet.

1. **Encryption:** The bytes of the JSON string are processed via `_crypt_xor_stream` .
2. **Envelope Assembly:** The binary initialization vector (the first 16 bytes) is concatenated with the resulting ciphertext, after which the entire structure is converted into a single hex string for secure transmission over text-based protocols.

3.2. Envelope Decryption (`decrypt_payload`)

Performs the reverse decoding of an incoming hex string (`encrypted_hex`):

1. The string is converted into raw binary form.
2. The first 16 bytes, which represent the initialization vector (`iv`), are strictly extracted from the array, and the remaining part is labeled as ciphertext (`ciphertext`).
3. The data undergoes keystream decryption via `_crypt_xor_stream` , and the resulting byte string is parsed back into the original dictionary via `json.loads` .

4. Authenticity and Integrity Control (Authentication)

To eliminate chosen-ciphertext attacks (CCA) and prevent on-the-fly packet tampering attempts, the system utilizes the **Encrypt-then-MAC** concept (encryption first, then signature).

4.1. Packet Signature Creation (`sign_ciphertext`)

The function binds the sender identifier and the body of the encrypted packet together with a cryptographic signature.

- **Format of Signed Data:** A string formatted as `"{device_id}:{ciphertext_hex}"` , encoded in UTF-8.
- **Signature Algorithm:** Calculation of HMAC-SHA256 from the formatted string using a dedicated hex key. Returns a string hex digest.

4.2. Packet Verification (`verify_ciphertext`)

Validates the envelope signature *prior* to executing the potentially vulnerable decryption and JSON parsing procedure.

- The function dynamically generates a reference signature based on the received device ID and ciphertext, then compares it with the incoming signature (`incoming_signature`).

- **Protection Against Timing Attacks:** String comparison is performed exclusively via the specialized `hmac.compare_digest` function. This ensures constant-time execution of the comparison operation regardless of which byte contains a mismatch, completely neutralizing side-channel timing attacks.

4.2 Core Components (Core)

4.2.1 Configuration Manager, Secret Manager, Session Manager, Context Manager

1. Configuration Manager (`core/config_manager.py`)

Architecture and Data Structure

The system configuration is built on the **Pydantic (v2)** library, which ensures strict data type validation at startup. The central hub is the monolithic `AppConfig` class, which aggregates the configuration schemas of all modules:

| Configuration Class | Key Parameters and Features |
|--|--|
| <code>CoreConfig</code> | Stores the base version (<code>version = "3.0.0"</code>), <code>sub_version</code> , and the <code>use_tui_mode</code> flag. |
| <code>RedisConfig</code> | Connection settings, baseline <code>ttl</code> (120s), and <code>session_ttl</code> (300s). |
| <code>PostgresConfig</code> | DB parameters and connection pool settings (<code>pool_min_size: 5</code> , <code>pool_max_size: 20</code>). |
| <code>NetworkConfig</code> | Network operation timeouts and connection retry limits. |
| <code>MqttClientConfig</code> | Stores broker parameters, LWT settings (<code>online / offline</code>), and topic mapping. |
| <code>VoiceInputConfig</code> / <code>SileroTTSTConfig</code> / <code>VADConfig</code> | Audio interface parameters, paths to ML models (Vosk, Silero), and buffers. |

Initialization and Validation Mechanism

- Loading:** The `load_config(config_path)` function reads the YAML file via `yaml.safe_load` and passes the dictionary to the `AppConfig` constructor.
- Orchestration and Post-Validation:** Inside `AppConfig` , three `after` -validators execute sequentially for end-to-end data synchronization:
- `sync_bridge_dependencies` : Extracts settings for external bridges (`external_bridges`) and propagates them to `external_bridge_heartbeat` and `remote_config_manager` .
- `distribute_dev_settings` : Forwards flags from `in_development_settings` (e.g., `dry_run` , `docker_mode`) to the respective target modules.
- `set_timezone` : Forcibly overwrites `location_timezone` across all dependent subsystems (scripts, notifications, archiver) based on the global value.
- Error Handling:** When a `ValidationError` occurs, the manager outputs a detailed log specifying the erroneous field via `logger.error` and hard-terminates the process via `SystemExit(1)` .

Critical Analysis and Risks

- **Lack of Hot-reload:** Dynamic code-driven configuration modification "on the fly" is not provided. Any change requires a core restart.
- **Architectural Typos:**
 - In the `NetworkConfig` class, the parameter is declared as `retyr_count` instead of `retry_count`.
 - The `KeyMenegerSystemConfig` and `MonitorModuelConfig` classes contain spelling errors in their names (`Meneger`, `Moduel`).
- **Mutability During Validation:** Post-validator overwrite field values of the `self` object after its creation, which may lead to side effects under potential concurrent access.

2. Secret Manager (`core/secret_manager.py`)

Security Components and Encryption

- **Base Class:** Inherits from `BaseClass`, belongs to **Tier 0**, and possesses the `['storage']` hardware capability.
- **Cryptography:** `cryptography.fernet.Fernet` is used as the symmetric encryption algorithm.
- **KMS Interaction:** The manager does not store the master key. During the `on_load` phase, it requests the raw key from the KMS subsystem via the `self.interfaces.kms.get_key(BASE_MANE, 'storage')` interface, then encodes it into URL-safe Base64 to initialize Fernet.

Data Lifecycle

1. **Loading:** The `initialize()` method reads the encrypted file from the path specified in the configuration (`config/secrets.enc`), decrypts it using `Fernet`, and parses lines in `key=value` format.
2. **Storage:** Decrypted secrets are retained in RAM within an internal `self._secrets` dictionary.
3. **Unloading:** Upon `on_unload()`, the `self._secrets` dictionary is completely cleared, and the status changes to `ModuleStatus.OFFLINE`.

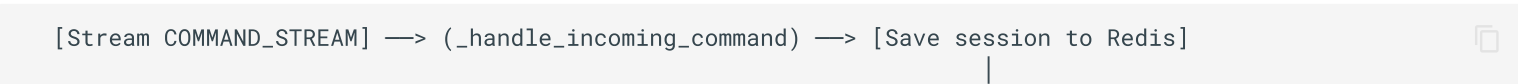
Critical Analysis and Risks

- **Zero Trust Violation (Internal Security):** Decrypted secrets reside in memory in a standard unprotected `dict`. Any module that obtains a direct reference to the `SecretManager` instance can call the `get_secret(key)` method and read the data. Proxy isolation for `child_api.py` is absent.
- **Architectural Typo:** The module name constant is defined as `BASE_MANE = 'secret_manager'`.

3. Session Manager (`core/session_manager.py`)

Queue Architecture and Event-Driven Logic

The manager is an internal core service (`SubotaInternalService`), belongs to **Tier 3**, and operates with the asynchronous Redis driver (`redis.asyncio`). It is responsible for end-to-end message routing and tracking command execution context.



```
[Stream RESPONSE_STREAM] <— (publish_notification) <— (_handle_assistant_response)
```

Session Lifecycle

1. **Subscription Registration:** Asynchronous workers are started via `task_watcher` for two Redis Streams event streams:
2. `COMMAND_STREAM` ('user_command_input')
3. `EMOTIONAL_RESPONSE_STREAM` ('emotional_assistant_response')
4. **Inbound Session Capture:** The `_handle_incoming_command` method intercepts the command, validates the payload via the `@SubotaInternalService.check_payload` decorator, and saves metadata (`source`, `timestamp`, `source_user_id`) to Redis with a `session:{session_id}` prefix and a `session_ttl` lifetime constraint.
5. **Response Routing:** Upon receiving an event from `EMOTIONAL_RESPONSE_STREAM`, the `_handle_assistant_response` method reads the session metadata from Redis, determines the original channel (`source`), and sends the compiled response back to the central notification bus via the `publish_notification` method.

Critical Analysis and Risks

- **Tight Coupling via TaskWatcher:** Subscription management is entirely delegated to an external `task_watcher` by passing lambda expressions. In the event of a worker crash inside the stream, restart logic relies exclusively on the correct operation of `task_watcher`.
- **Direct Serialization:** Writing to Redis is performed via standard `json.dumps`, which imposes restrictions on data types within the session metadata.

4. Context Manager (`core/context_manager.py`)

Short-Term Memory

A **Tier 1** module encapsulating the logic for working with the user's dialogue context (short memory).

- **Storage:** Uses a dedicated Redis client obtained via `redis_connection_manager`.
- **Key Structure:** Data is isolated using the mask `context:{user_id}:{key}`.
- **TTL Control:** Writing is performed with a mandatory lifetime restriction (`ttl`) specified in the `RedisConfig` configuration.

Critical Analysis / CRITICAL BUG

There is an explicit architectural asymmetry in data handling within the code:

1. The `save_context` method performs data serialization: `json.dumps(value).encode('utf-8')` and saves it to Redis as a JSON byte string.
2. The `get_context` method performs an `await self.redis.get(redis_key)` operation and **directly returns the raw string/bytes** without performing reverse deserialization via `json.loads`.

- **Architectural Risk:** Any calling component expecting to receive the original data type (dictionary, list, or number) from `get_context` will encounter a business logic error because the method returns raw JSON text. Refactoring of the `get_context` method is required to ensure data transformation symmetry.

4.2.2 Notification Manager

The `core/notification_manager.py` module, `NotificationManager(SubotaInternalService)` class. An internal core service operating at the **Tier 4** level, with the configuration bound via `config_name='notification_manager'`.

Purpose

The central dispatch point for all outbound system notifications (standard, critical, and archiver notifications), routing them to sender channels (`telegram_sender`, `cli_sender`, `web_ui_sender`, `voice_sender`, etc.) that are loaded dynamically.

Dynamic Loading of Senders

During initialization, the module assembles `senders_interfaces` — a flat dictionary of all non-private, non-callable attributes of `self.interfaces` — and passes it along with the active senders configuration (`notifi_config.active_senders`) to `ChildAPI` / `load_children_with_retry` (the exact same dynamic loading mechanism used for user scripts, see `interfaces.md`). Loaded sender instances are placed into `self._senders`.

Two Background Loops

- **NotificationManager Loop** — subscribes to the `notifications_group` and processes events of the `publish_notification` type via `_handle_notification`.
- **NotificationManager Critical Loop** (`_run_critical_queue_listener`) — listens to a local `asyncio.Queue` (`emergency_queue`):
 - `CRITICAL_NOTIFY` and `ARCHIVE_DONE` events are processed directly (`_send_critical_notifi` / `_send_archiver_notifi`);
 - `EXIT_SEQ_REQ`, `EMERGENCY_STOP`, and `RELOAD_MODULE` events are forwarded further down into the internal core queue (`__inner_core_queue`) for handling at the `Core` level.

Main Methods

- `send_message_notification(text, urgency='info', notification_destinations=['telegram'])` — prepends an emoji prefix based on the severity level (🔴 critical / ⚠️ warning / ❌ error / ⓘ info) and broadcasts the message to the specified channels via `asyncio.create_task` (fire-and-forget, without awaiting the result).
- `_handle_notification(payload)` — the handler for standard notifications from the event bus, determining destination channels via `payload.data['content']['destinations']`.
- `_send_critical_notifi(payload)` — a critical notification with a timestamp, which enforces a pause of `notifi_config.critical_timeout` after being sent (protection against flooding channels with frequent critical messages).
- `_send_archiver_notifi(payload)` — a notification from the archiver that builds a multi-line report from `archivation_payload`. Supports "night mode": within the 23:00–07:00 interval, transmission is deferred until morning via `loop.call_later`.

Critical Analysis (Based on Code Inspection)

1. `__get_seconds_wake_up_time` will crash when invoked:

```
target = datetime.strptime('7:00', "%H:%M:%S")
```



The `"%H:%M:%S"` format requires seconds, whereas the string `'7:00'` does not contain them — the call is guaranteed to terminate with `ValueError: time data '7:00' does not match format '%H:%M:%S'`. Even if the format is corrected to `"%H:%M"`, a second issue remains: `target` will be offset-naive (`tzinfo=None`), while `now` obtained from `__get_time_now()` is timezone-aware (via `pytz`). Comparing `now >= target` in this scenario will crash with `TypeError: can't compare offset-naive and offset-aware datetimes`. The method is invoked from the night branch of `_send_archiver_notifi` — meaning any archiver notification arriving between 23:00 and 07:00 will trigger an unhandled exception.

1. **Inconsistent severity keys across handlers within the same module.** `_handle_notification` and `_send_critical_notifi` read the key `'ugrgency'` (a typo, but used consistently throughout the system), whereas `_send_archiver_notifi` reads the correctly spelled `'urgency'`:

```
# _send_archiver_notifi
urgency = content.get('urgency', 'info')
```



If the event publisher (in line with the rest of the system's codebase) passes `ugrgency`, the archiver path will fail to detect it and silently fall back to the default `'info'`.

1. **Unresolved author's TODO in the code:** `_handle_notification` is marked with `#!/ TODO: [NotificationManager] fix payload for notification` directly above the `@SubotaInternalService.check_payload` decorator — payload processing for this method is flagged by the author themselves as incomplete.
2. **Unknown severity level "leaks" into the message text.** In `send_message_notification`, the catch-all `match ugrgency` branch generates a prefix `f'Unknown ugrgency: '`, which is appended directly to the text sent to the user instead of being logged as a warning:

```
case _:
    prefix = 'Unknown ugrgency: '
```



1. **Fire-and-forget without exception handling.** Both `_handle_notification` and `send_message_notification` trigger transmission via a raw `asyncio.create_task(...)` call without maintaining a reference to the task or attaching a completion callback (unlike, for instance, `credits.add_done_callback(handle_task_result)` in `core_init_v2.py`). If `sender.send_message()` raises an exception, it will remain unhandled — surfacing only as a default `asyncio` warning rather than being captured by the module's logger.

4.2.3 Task Watcher Reference (Task Watcher)

1. Architectural Context and Initialization

The `TaskWatcher` component (module `core/task_watcher.py`) operates at the second level of the hierarchy (`tier=2`) of the S.U.B.O.T.A. system core and inherits from the `BaseClass` base class. The primary objective of the component is the continuous monitoring of registered asynchronous coroutines, their automatic resuscitation when exceptions occur, and the prevention of cascading system failures by means of a controlled application shutdown when error limits are exceeded.

1.1. Configuration Parameters and Interfaces

Upon instance creation, the following parameters of the declarative contract with the core are established:

- `name`: `'task_watcher'`
- `config_name`: `'task_watcher'` (configured via the corresponding `TaskWatcherConfig` class).
- `methods`: requests access to the `core_stop_event` and `core_stop_func` objects from the infrastructure (`infra`) level.

During the execution of the asynchronous loading phase (`on_load`), the component extracts parameters from the configuration and binds system events:

- `check_interval` — the periodicity of task state checks in seconds.
- `max_restarts` — the default global limit of restart attempts for a single task.
- `is_core_stopping` — an `asyncio.Event` signaling the commencement of the core shutdown.
- `stop_core` — the executable object (`core_stop_func`) for emergency system termination.

During initialization, the module is assigned the `ModuleStatus.STARTING` status, and upon completion of the `on_load` method, the status transitions to `ModuleStatus.ACTIVE`.

2. Component Structure and Data Model

Task management is abstracted from low-level `asyncio.Task` objects using a specialized `MonitoredTask` container.

2.1. The `MonitoredTask` Container

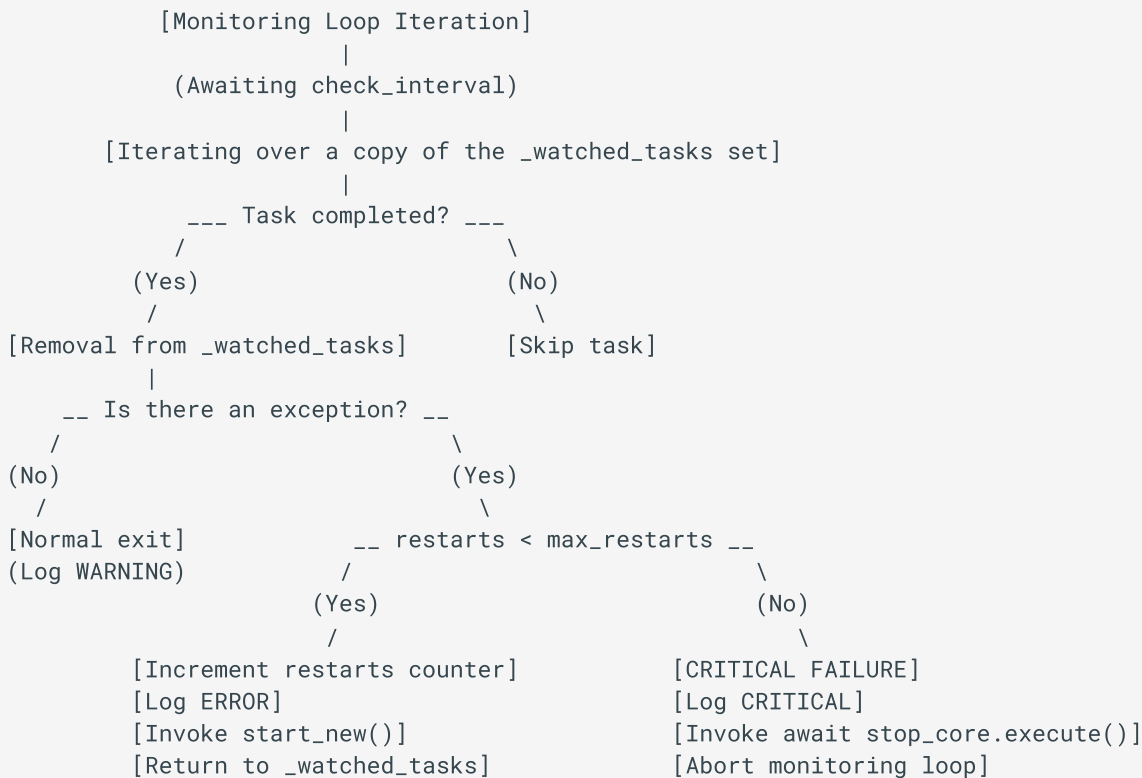
Each monitored coroutine is wrapped in an instance of the `MonitoredTask` class, which stores state and metadata:

- `coroutine_func`: A reference to the callable object that returns a coroutine (`Callable[[], Coroutine[Any, Any, None]]`).
- `name`: A unique string name of the task for logging purposes and display within the control panel.
- `task`: A reference to the currently running `asyncio.Task` object (or `None` if the task has not yet started).
- `restarts`: The current counter of executed emergency task restarts (initially `0`).
- `max_restarts`: The maximum allowable number of restarts for this instance (cast to `int`).

The `start_new()` method performs the physical launch of the task within the Event Loop via `asyncio.create_task(self.coroutine_func(), name=self.name)` and stores the reference in the `task` attribute.

3. Supervisor Operational Algorithm (`_monitor_loop`)

The core monitoring logic is executed within a background asynchronous task named `"Watcher_Supervisor"` , which is activated by calling the `start()` method. The supervisor functions as an infinite loop until the `is_core_stopping` event transitions to the set state (`is_set()`).



3.1. Task Termination Scenarios

- 1. Normal Termination (Clean Exit):** If a task completes without exceptions (`task.exception() is None`), this is interpreted as a normal exit (for example, an internal `break` was triggered within the coroutine loop). The component records the event in the log with a `WARNING` status, and the task is permanently removed from the monitoring pool.
- 2. Emergency Termination (Crash) Within the Limit:** If a task crashes with an error and the current `restarts` counter is less than `max_restarts` , then:
 3. The restart counter is incremented (`restarts += 1`).
 4. A message with a traceback dump (`exc_info=True`) is written to the log with an `ERROR` status.
 5. `start_new()` is invoked, creating a new instance of `asyncio.Task` , and the object is returned back to the `_watched_tasks` pool.
- 6. Critical Failure (Limit Exceeded):** If a task crashes and the restart limit is exhausted (`restarts >= max_restarts`), the supervisor:
 7. Logs a `CRITICAL` status indicating the name of the task.
 8. Immediately initiates an emergency shutdown of the entire application via `await self.stop_core.execute()` .
 9. Aborts its loop and exits the method.

4. Task Pool Management (Public Interface)

- `start_watching(coroutine_func, name)` : Registers a new task. It creates a `MonitoredTask` object using the global `max_restarts` value from the configuration, performs the initial launch, and adds the container to the internal `_watched_tasks` set.
- `stop_watching(name) -> bool` : Performs controlled removal of a task from monitoring. It locates the task by name, removes it from `_watched_tasks` , and, if the task is active, sends a cancellation signal `task.cancel()` . The process awaits the completion of the cancellation within a strict timeout of 2 seconds. Cancellation errors (`CancelledError` , `TimeoutError`) are intercepted and suppressed, while other exceptions are logged. Returns `True` if the task was found, otherwise `False` .

5. Component Unloading (`on_unload`)

When the system shutdown process (Graceful Shutdown) is initiated, `TaskWatcher` performs resource cleanup:

1. The supervisor loop itself is forcibly cancelled (`_monitor_task.cancel()`).
2. A list of all currently active tasks from the `_watched_tasks` pool that have not yet completed is compiled.
3. A `task.cancel()` signal is sent to all discovered tasks.
4. Using the `await asyncio.gather(*tasks_to_await, return_exceptions=True)` construct, the component blocks the unloading thread until all tasks are guaranteed to finish processing the cancellation signal.
5. Completion of the supervisor itself is awaited, the `_watched_tasks` set is cleared, and the module status is set to `ModuleStatus.OFFLINE` .

6. State Monitoring Interface (Data for WebUI)

The `get_status_data()` method returns a flat, structured list of dictionaries, optimized for JSON serialization and frontend display.

Each element of the list contains information about a task:

- `name` : The name of the monitored task.
- `restarts` : A string representation of restart progress formatted as `"{current}/{maximum}"` .
- `exception` : A string describing the error (`str(exception)`) if the task crashed, or `None` .
- `status` : The current textual marker of the coroutine state, computed via a strict cascade of checks:

| Task State | Status Calculation Conditions | Textual Marker (<code>status</code>) |
|--------------------|--|--|
| In progress | The task exists and the <code>task.done()</code> method returns <code>False</code> . | "Running" |
| Forcibly cancelled | A <code>task.cancelled()</code> call returned <code>True</code> OR an <code>asyncio.CancelledError</code> exception was intercepted when attempting to retrieve the error. | "Cancelled" |
| Normally completed | <code>task.done() == True</code> , and no exceptions were recorded during execution. | "Done" |

| Task State | Status Calculation Conditions | Textual Marker
(status) |
|-----------------------|---|------------------------------|
| Crashed with an error | <code>task.done() == True</code> and the <code>task.exception()</code> method returned an exception object. | "Failed" |

4.2.4 Authentication Manager (Auth Manager)

1. Architectural Context and Initialization

The `AuthManager` component (`core/auth_manager.py` module) functions at the first level of the hierarchy (`tier=1`) of the S.U.B.O.T.A. system core and inherits from the `BaseClass` base class. The module encapsulates all the logic for working with JWT (JSON Web Tokens), providing generation and secure verification of access tokens for adjacent services and interaction interfaces.

1.1. Dependency Contract and Methods

Upon instantiation, the object registers the following parameters of the declarative contract:

- `name`: `'auth_manager'`
- `config_name`: `'jwt'` (associated with the `JwtConfig` configuration structure).
- `methods`: requests access to the `get_secret` method from the secret manager (`secret_manager`).

1.2. Internal State and Lifecycle

Upon instance creation, internal variables are initialized, and the module is assigned the `ModuleStatus.STARTING` status:

- `_secret_key` — a string for storing the cryptographic JWT signature key (initially empty).
- `_algorithm` — a string specifying the hashing algorithm used.
- `_access_token_expire_minutes` — an integer value of the token lifetime.

Load Phase (`on_load`)

During the asynchronous startup of the module, the following steps are performed:

1. The core configuration is cast to the strict `JwtConfig` type.
2. The core secret retrieval method is cast to the `SM_get_secret` protocol type.
3. The `"JWT_SECRET_KEY"` variable is requested from the secret manager. If the key is missing, the fallback value `"FALLBACK_KEY"` is applied.
4. The token signing algorithm (defaulting to `"HS256"`) and its lifetime in minutes (defaulting to `30`) are extracted from the configuration.
5. The component status changes to `ModuleStatus.ACTIVE`, and the event is recorded in the system log.

Unload Phase (`on_unload`)

During a graceful shutdown of the service, for security purposes, the private fields `_secret_key` and `_algorithm` are cleared, the status is changed to `ModuleStatus.OFFLINE`, and a message about routine termination is written to the system log.

2. Access Token Creation (`create_access_token`)

The `create_access_token(data, expires_delta)` method generates a signed JWT string based on the passed `data` payload dictionary.

2.1. Critical Security Checks

- **Missing key:** If the private field `_secret_key` is not initialized, the method aborts execution, generates an error in the logger with the `ERROR` status, and throws a `ValueError("Secret key for JWT not found!")` exception.
- **Using default value:** If the standard code constant (`"your_secret_key"`) is used as the signature key, the method bypasses the operation but records a warning in the log with the `WARNING` status about the need to change the key to a unique one.

2.2. Payload Generation (Standard Claims)

The method creates a copy of the passed payload dictionary and supplements it with mandatory system service fields (Claims) according to the JWT standard:

- `exp` (Expiration Time): The token expiration time in UTC format. It is calculated as the current time `datetime.now(timezone.utc)` plus the passed `expires_delta` object (if provided), or plus the default number of minutes from `_access_token_expire_minutes`.
- `iss` (Issuer): The token creator identifier. Hardcoded as `'subota_ai_core'`.
- `sub` (Subject): The token subject. Hardcoded as `'access_token'`.

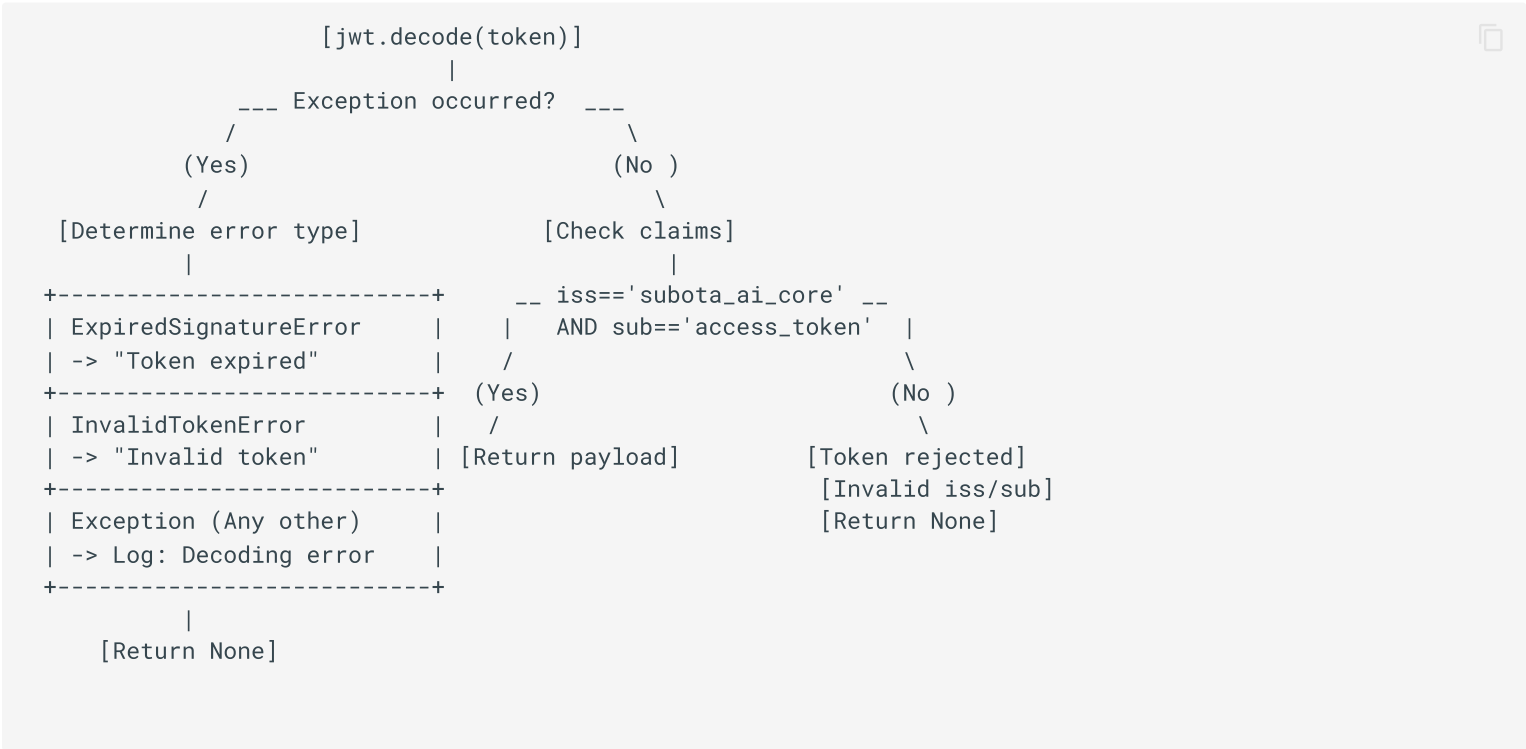
The token string is encoded using the PyJWT library via the `jwt.encode` method applying the specified `_algorithm`.

3. Token Verification and Decoding (`decode_access_token`)

The `decode_access_token(token)` method accepts an incoming JWT string, performs its parsing, cryptographic signature verification, and structure validation. In case of success, it returns the dictionary of the original payload (`payload`); on any anomaly, it returns `None`.

3.1. Validation Cascade and Exception Handling

Decoding is protected by a `try/except` block and handles errors according to the following matrix:



3.2. Field Validation Specification

Even if the cryptographic signature of the token is correct and it has not expired, `AuthManager` performs additional logical validation of the contents:

 **Validation Rule:** The method checks for an exact match of the `iss` and `sub` fields. If the value of `payload.get('iss') != 'subota_ai_core'` or `payload.get('sub') != 'access_token'`, the token is immediately rejected, the message `Token rejected: Invalid issuer/subject` is written to the log, and the calling method receives `None`.

4.2.5 Network Controller Reference (Network Controller)

1. Architectural Context and Area of Responsibility

The `NetworkController` component (module `core/network_controller.py`) operates at the first level of the hierarchy (`tier=1`) of the S.U.B.O.T.A. system core and inherits from the `SubotaInternalService` internal security service. The primary purpose of the module is to provide the core with up-to-date information regarding the state of the external and internal networks, deliver a secure HTTP transport with lock-protection mechanisms, and maintain a cache of network telemetry.

1.1. Declarative Circuit Markers

Upon initialization, the service registers fixed system parameters:

- `service_name / BASE_NAME: 'network_controller'`
 - `SOURCE: 'core'`
 - `USER_ID: 'system'`
 - `config_name: 'network'` (configuration structure `NetworkConfig`).
 - `loop_name: 'NetworkController Loop'` (the name of the background task for the supervisor).
-

2. Configuration Parameters and Management Constants

The controller operates using rigidly fixed time intervals and threshold values that govern the frequency of network polling:

2.1. Time Intervals (`Final[float]`)

- `INTERVAL_NORMAL` = 60.0 sec. — Network verification periodicity during a stable internet connection.
- `INTERVAL_FAIL` = 10.0 sec. — Accelerated network polling when a single packet loss/access drop is detected.
- `INTERVAL_LONG_OFFLINE` = 300.0 sec. (5 minutes) — Infrequent polling mode during a confirmed, prolonged absence of connectivity (saves system resources).
- `FAIL_THRESHOLD` = 18 — The number of consecutive failed checks after which the network is officially classified as deeply unavailable.

2.2. Data Bus and Cache Constants

- `SYSTEM_STATUS_STREAM` = `"subota:internal:system:status"` — The name of the Redis stream used for broadcasting network status alteration events.
 - `SYSTEM_STATUS_CACHE` = `"subota:system:status:cache"` — The name of the Redis hash table (Hash) for storing a snapshot of the current network state under the `'internet'` key.
-

3. Network State Matrix (`NetworkState`)

The internal network state is typed by the `NetworkState` enumeration (built on top of `StrEnum`):

- 1. UNKNOWN — The initial state prior to the completion of the first verification iteration.
- 2. ONLINE — The external network is fully accessible; at least one endpoint from the pool responded successfully.
- 3. OFFLINE_SHORT — The network is temporarily unavailable; the number of failed checks is below the FAIL_THRESHOLD .
- 4. OFFLINE_LONG — Connectivity has been absent for a long duration (failure counter >= 18).

4. Diagnostic Algorithms and Interfaces

4.1. Automatic Topology Detection (_initialize)

If the external_ip parameter is not defined within the NetworkConfig configuration, the controller initiates its automatic detection:

- An HTTP request is executed to https://api.ipify.org?format=json with a retry limit of retyr_count (default is 3) and a pause of ping_timeout (default is 2.0 sec.) between attempts.
- If IP determination fails, an error is logged, as subsequent diagnostic processes for external nodes may function incorrectly.
- During the initialization stage, the core's authorization tokens (credits) are requested to establish a connection to the redis_client . Following this, the _internet_check_loop task is handed over to the task_watcher for monitoring.

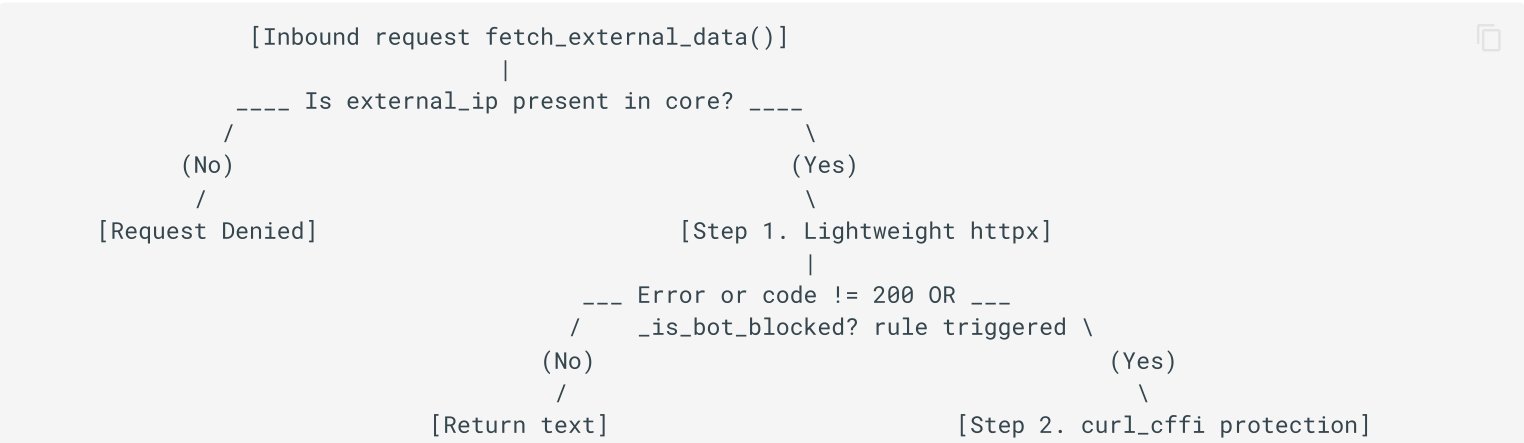
4.2. Routing Simulation (_get_source_ip and check_ping)

To segregate network boundaries, the controller verifies whether the target host belongs to private subnets according to RFC 1918 (based on the prefixes 192.168. , 10. , 172.16. - 172.31.):

- When invoking check_ping(target_host, timeout) , the argument src_addr=self.internal_ip is forcibly set for private subnets.
- For external addresses, the source parameter remains empty, allowing the operating system to apply its standard host routing table.
-  **Important Constraint:** Executing low-level ICMP pings via the ping3 library requires superuser (root) privileges. In their absence, a PermissionError is intercepted, and a corresponding warning is recorded in the log.

5. Two-Stage Transport and WAF Circumvention (fetch_external_data)

The fetch_external_data method ensures highly reliable data retrieval from external resources protected by traffic filtering systems (Cloudflare, Incapsula, etc.).



(Failure)

False]

(Success)

[Return text]

[Log error /

5.1. Step 1: Standard Client (`httpx`)

Initially, the request is dispatched via the standard asynchronous client `httpx.AsyncClient` with an established baseline timeout. Upon receiving a response, the content is passed to the bot signature analyzer `_is_bot_blocked(html_content)`. The signature analyzer inspects the HTML code for key markers indicating the presence of protective firewalls: `["incapsula", "robots", "incapsula incident id", "request unsuccessful", "captcha", "forbidden"]`. If a marker is detected, response generation is aborted, and the controller transitions to the second stage.

5.2. Step 2: Browser Emulation (`curl_cffi`)

When a block is identified or the `httpx` timeout expires, the controller deploys a heavy anti-bot session using `AsyncSession(impersonate="chrome")`. The `curl_cffi` library performs a complete, low-level simulation of the TLS fingerprint (JA3/JA4 fingerprints) and the headers of the Google Chrome browser. If, even after emulation, a repeated text analysis via `_is_bot_blocked` reveals traces of a WAF, the request is definitively flagged as compromised and returns a failure indicator.

6. Concurrent Monitoring Pool Mechanism (`_internet_check_loop`)

The background network verification loop is optimized to eliminate false-positive triggers resulting from individual server crashes:

1. The controller sequentially polls a built-in distributed pool of endpoints:
2. `[http://connectivitycheck.gstatic.com/generate_204](http://connectivitycheck.gstatic.com/generate_204)` (an HTTP status of `204` is expected).
3. `[http://www.msftconnecttest.com/connecttest.txt](http://www.msftconnecttest.com/connecttest.txt)` (the occurrence of the text `"Microsoft Connect Test"` is expected).
4. `[http://captive.apple.com](http://captive.apple.com)` (the occurrence of the text `"Success"` is expected).
5. **Atomic Isolation:** Polling a specific point within the loop (`_check_endpoint`) is constrained by a rigid, non-blocking timeout of `3.0` seconds to avoid locking the iteration.
6. **Concurrent Verification:** If the current pool endpoint fails to respond, the controller instantly deploys a parallel poll of *the entire remaining pool simultaneously* using `asyncio.gather(*tasks)` inside the `_verify_full_pool` method. The network is declared offline only if *none* of the servers in the pool provide a valid response.

6.1. Event Publication and Cache Invalidation

When a change in network status is detected (`old_state != self.state`), the controller executes two mandatory transactions:

- It dispatches an `internet_status_changed` event into the Redis Stream `subota:internal:system:status` to notify the `SystemStateManager` and other dependent core services.

- It serializes the `internet_status_dict` dictionary into JSON format and writes it using the `hset` method into the `subota:system:status:cache` hash table under the `'internet'` key, thereby providing instantaneous access to a snapshot of network data for external interfaces (WebUI) without the need to re-interrogate the hardware.

4.2.6 PostgreSQL Manager Reference (Postgres Manager)

1. Architectural Context and Dependencies

The `PostgresManager` component (module `core/data_bases/postgres_manager.py`) serves as a fundamental element of the S.U.B.O.T.A. core data storage subsystem, operating at the first level of the hierarchy (`tier=1`) and inheriting from `BaseClass`. The class is responsible for providing an interface to work with strictly relational data for all adjacent scheduler services and business logic.

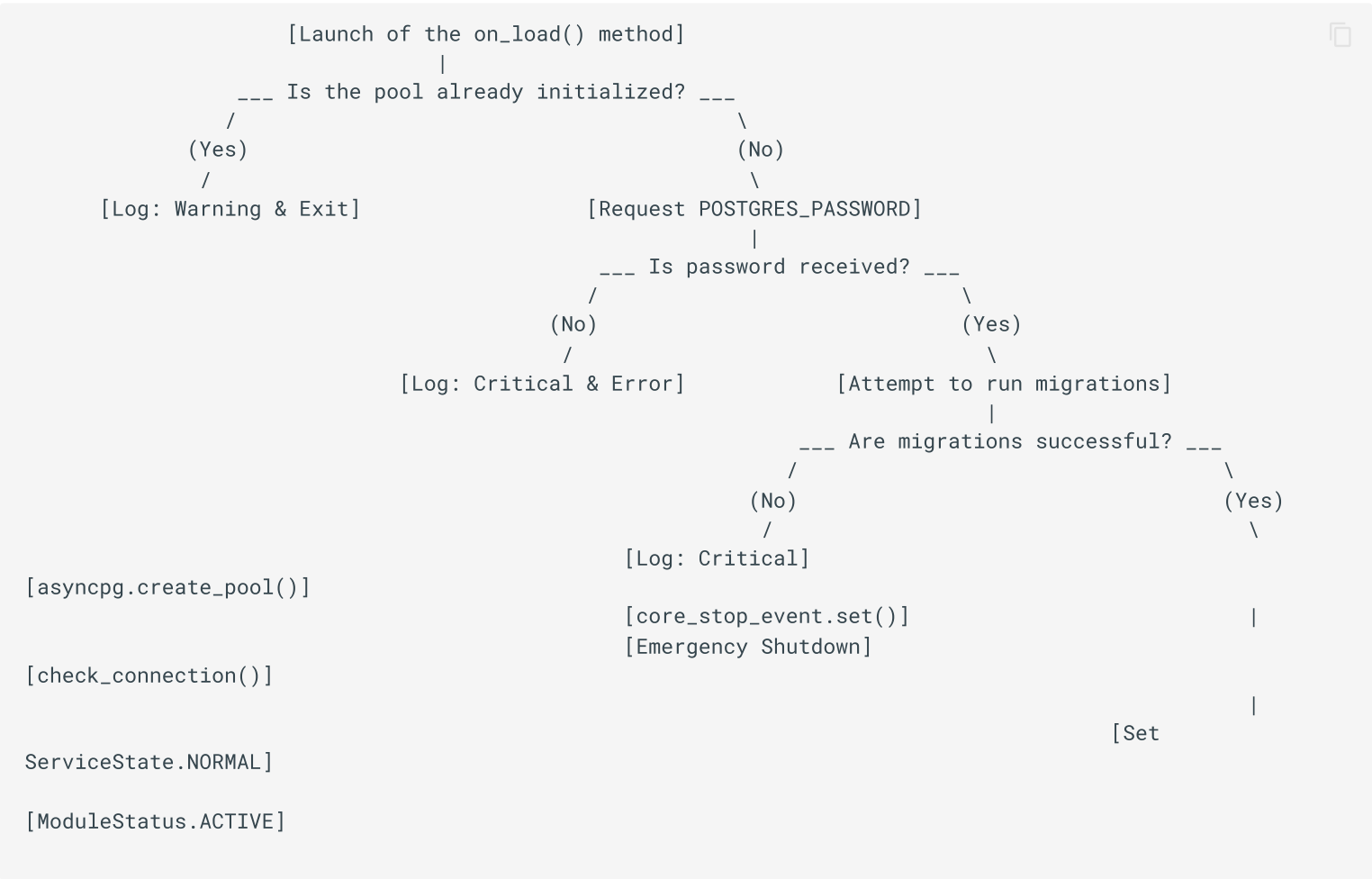
1.1. Declarative Relations Contract

Upon instantiation, the object registers rigid system bindings:

- `name`: `'postgres_manager'`
- `config_name`: `'postgres'` (configuration structure `PostgresConfig`).
- `deps`: Explicit dependencies on `['system_state_manager', 'emergency_buffer']`. The module will not transition into its active phase until the specified services are fully loaded.
- `methods`: Declares a requirement for the `get_secret` method from the secret manager and the `core_stop_event` from the core infrastructure subsystem.

2. Lifecycle and Initialization Procedure

The module undergoes a strict cascade of security and infrastructure readiness checks when the asynchronous `on_load()` method is invoked:



2.1. Isolation and Migration Execution (`run_migrations`)

The core decouples the asynchronous operational context of the application from the synchronous execution mechanism of Alembic database migrations:

1. `"POSTGRES_PASSWORD"` is retrieved from the secret manager. If the string is empty, a critical log is generated and execution is terminated with a `ValueError` exception.
2. The password is escaped using the `quote_plus` function, and a connection string adhering to the `postgresql+asyncpg://...` standard is prepared. For proper interpretation of special characters within Alembic, `%` characters are doubled (`%%`).
3. The password is temporarily placed into the `os.environ["MIGRATION_POSTGRES_PASSWORD"]` system environment variable.
4. Since Alembic performs synchronous blocking I/O operations and requires the presence of an `alembic.ini` configuration file in the working directory, its invocation is isolated within a separate operating system worker thread using the `asyncio.to_thread(self._run_alembic_upgrade_sync, full_url)` abstraction.
5. Upon completion of the synchronous `command.upgrade(alembic_cfg, "head")` step, a `finally` block guaranteed-clears the `MIGRATION_POSTGRES_PASSWORD` environment variable to prevent credential leakage into child processes.
6. Migration initialization is wrapped in an exponential retry scheduler `_retry_async_init` (maximum of 3 attempts, initial delay of 5 seconds). If migrations fail after all attempts, the manager considers the environment compromised, sets the total core shutdown event `self._stop_core_event.set()`, and aborts the startup of the entire system.

2.2. Asynchronous Pool Deployment

Following the successful application of migrations, the `asyncpg.create_pool()` connection pool constructor is invoked, passing the `pool_min_size` and `pool_max_size` size constraints from the component settings. A non-blocking test poll is performed via `check_connection()` (`SELECT 1;`), after which a `ServiceState.NORMAL` status is transmitted to the `system_state_manager`.

3. Transaction Processing Architecture and Fault Tolerance

The component implements two fundamentally distinct approaches for executing modifying (write/update) and informational (fetch) SQL queries.

3.1. Data Modification and Emergency Gate (`@emergency_gate`)

Write methods `execute(query, *args)` and batch processing methods `execute_many(query, values)` are rigidly decorated with the `@emergency_gate('postgres')` system wrapper.

- In the event of a PostgreSQL connection pool failure or physical unavailability of the DBMS server, the decorator intercepts control and redirects the data stream to the internal `emergency_buffer` (Emergency Buffer) module, preventing write operations from being lost.
- The `execute_many` method utilizes the optimized `connection.executemany` protocol for the atomic insertion of tuple batches, recording the volume of processed entries into the debug logger.

3.2. Data Reading and Silent Failover (`_query_gate`)

All methods for data retrieval (`fetch`, `fetchrow`, `fetch_all`) are channeled through a central unified gateway

`_query_gate(method_name, query, *args)` . The method implements a service silent degradation pattern:

1. **Preventive Check:** If the `SystemStateManager` has already logged a global DBMS failure (`postgres != ServiceState.NORMAL`), the gateway instantaneously returns an empty result (an empty list `[]` for fetch methods or `None` for precise queries), preventing the accumulation of hung timeouts within the core.
 2. **Dynamic Proxy Call:** The method dynamically extracts the target read function from the `asyncpg` connection object using the standard `getattr(connection, method_name)` mechanism.
 3. **Silent Switch:** Upon the occurrence of transport network errors (`asyncpg.PostgresConnectionError`, `OSError`, `ConnectionError`), the gateway **does not propagate** the exception upwards. Instead, it:
 4. Transitions the service status into emergency mode: `self.core_state_manager.set_service_state('postgres', ServiceState.FAILOVER)` .
 5. Directly writes a diagnostic string to the standard I/O error stream `sys.stderr.write(...)` to bypass standard logger queues.
 6. Silently returns a safe empty result (`[]` or `None`), allowing the system to continue operating based on cached data.
 7. **Critical Syntax Errors:** Logical errors in SQL query construction (`Exception`) are intentionally re-raised (`raise e`) so that they are captured by the calling context (for instance, the task scheduler logger).
-

4. Reserved Structure and Built-in Metrics

4.1. Embedded Tables (`_create_tables_if_not_exists`)

The module contains a reserved initialization schema for base tables (used as a fallback deployment option outside of Alembic):

- `settings` (`key TEXT PRIMARY KEY`, `value TEXT NOT NULL`) — a flat storage for configuration key-value pairs.
- `scheduled_tasks` — a system task scheduler table supporting JSONB structures for schedule configurations (`schedule_config`) and bus event metadata (`event_data`), alongside timezone markers (`TIMESTAMP WITH TIME ZONE`).

4.2. Core Performance Analytics (`system_metrics`)

The manager provides specialized methods for extracting telemetry and performance metrics of the operating system and core components:

- `get_latest_metrics()` — utilizes an analytical windowed query `SELECT DISTINCT ON (metric_name) ... ORDER BY metric_name, "timestamp" DESC`, allowing for an instantaneous snapshot of the freshest physical and string indicators for each registered system metric.
 - `get_history_metrics(limit)` — extracts the historical stack of metric alterations sorted by time in reverse order with a rigid selection size constraint applied via the `$1` placeholder.
-

5. Component Unloading (`on_unload`)

During module deactivation or a core shutdown, a controlled closure of the connection pool is performed within the `on_unload()` method:

- The existence of an active pool object is verified, after which the `await self.pool.close()` coroutine is invoked.
- The reference to the pool is cleared (`self.pool = None`), preventing memory leaks.
- The module status changes to `ModuleStatus.OFFLINE` , and the completion fact is recorded in the system log.

4.2.7 Logging Subsystem Reference (Log Distributor & Logger Manager)

1. Architectural Context and Area of Responsibility

The S.U.B.O.T.A. core logging subsystem consists of two cooperating components: the declarative `LogDistributor` service (`core/log_distributor.py`) and the functional controlling module `LoggerManager` (`core/logger_manager.py`).

Together, they provide:

- Log collection from all system modules.
 - Multi-channel stream separation (console, files, audit, forensics).
 - Interception of third-party library I/O streams.
 - Asynchronous delivery of critical logs (severity level `WARNING` and higher) to the shared core event bus without blocking the primary application thread.
-

2. Logger Manager (`core/logger_manager.py`)

The module configures Python's standard `logging` library, injecting custom handlers, formatters, and overload protection mechanisms.

2.1. Interception of External Streams (`VoskStreamRedirect`)

A specialized interface class designed to capture textual output from system `stdout` / `stderr` streams.

- **Application:** Used primarily to redirect output from low-level speech recognition libraries (Vosk/Kaldi), which write diagnostic data to `stderr` by default, bypassing the standard logger.
- **Operational Algorithm:** Accumulates incoming characters within an internal buffer `self.buffer` . Upon detecting a newline character (`\n`), the `write()` method splits the text, cleans empty lines, and dispatches each completed entry line-by-line to the target Python logger with a designated severity level (defaulting to `logging.INFO`).

2.2. Log Storm Protection (`EventBusHandler`)

A custom `logging.Handler` that executes the translation of logs into the event bus. To avoid infinite publication loops and memory exhaustion, rigid filters are embedded within the handler:

1. **Loop Filtering:** The `emit` method instantly ignores any records generated by logging infrastructure and transport modules: `["core.redis_event_bus", "core.logger_manager", "core.system_log_processor", "root"]` .
2. **Duplicate Suppression (`TTLCache`):** To protect the system from a "log storm" (where the exact same error is cyclically generated at a rate of thousands of entries per second), a cache with a lifetime restriction is applied: `TTLCache(maxsize=100, ttl=20)` . A unique fingerprint is formed based on the logger name, severity level, and the first 50 characters of the message. If a matching fingerprint already resides within the cache, the entry is silently discarded.
3. **Transaction Volume Restriction:** If the resulting formatted text exceeds a length of `3000` characters, the string is forcibly truncated, and a `"... [TRUNCATED DUE TO OVERSIZE]"` marker is appended. This precludes transport buffer drops during huge traceback dumps.

4. **Asynchronous Decoupling:** Event bus publication is executed in a non-blocking manner. The method intercepts the currently running event loop via `asyncio.get_running_loop()` and schedules the dispatch task through `loop.create_task`. If the event loop is stopped (for example, during the core shutdown phase), a `RuntimeError` exception is intercepted without halting the execution thread. Any internal failures of the handler itself are written directly to `sys.stderr`.

3. Channel Topology and Rotations (`setup_logging`)

The `setup_logging` function initializes five independent channels (handlers) to distribute logs based on the provided `AppConfig` configuration:

| Channel Name /
Logger | Handler Type | Purpose / Specifics | Rotation Limits |
|--|----------------------------------|---|-------------------------------|
| Console
(<code>sys.stdout</code>) | <code>StreamHandler</code> | Output to terminal with color-coded levels via <code>colorlog.ColoredFormatter</code> . | None |
| General Log | <code>RotatingFileHandler</code> | Storage of all system entries in <code>date / level / module / text</code> format. | 10 MB,
maximum of 3 files |
| asyncio Error Log | <code>RotatingFileHandler</code> | An isolated log for entries with a severity level <code>>= logging.ERROR</code> . | 10 MB,
maximum of 3 files |
| Breach Dumps
(<code>security_forensics</code>) | <code>RotatingFileHandler</code> | Registration of security incidents. Wrapped in strict <code>--- INCIDENT START/END ---</code> markers. The <code>propagate = False</code> property ensures it is not duplicated in the general log. | 5 MB, maximum
of 5 files |
| System Audit
(<code>system_audit</code>) | <code>RotatingFileHandler</code> | Structured log of operations in JSON Lines format (<code>audit.jsonl</code>). Utilizes <code>JsonAuditFormatter</code> . <code>propagate = False</code> property applied. | 10 MB,
maximum of 10 files |

3.1. JSON Audit Formatting (`JsonAuditFormatter`)

Utilized for automated security log parsing by machines. It transforms a record into a flat JSON object containing the following keys:

- `ts` : Timestamp indicating record creation (Unix timestamp).
- `lvl` : String representation of the severity level (e.g., `INFO`, `ERROR`).
- `mod` : Name of the logger module that invoked the call.
- `data / msg` : If a dictionary or a list is passed as the message, it is placed entirely within the `data` key. Otherwise, a string is written to `msg`.

4. Log Distributor (`core/log_distributor.py`)

The `LogDistributor` class inherits from the `SubotaInternalService` base security component and acts as an authorized agent for transmitting data to the event bus.

4.1. Declarative Circuit Markers

Upon initialization, internal security parameters are fixed:

- `service_name`: `"log_distributor"`
- `user_id`: `'system'`
- `source`: `'core'`
- `config_name`: `None` (utilizes the shared core infrastructure).

4.2. Dynamic Connection to the Bus (`add_event_bus_handler_to_logger`)

Immediately after the core initializes the global event bus (`EventBus`) and transitions `LogDistributor` to the `ModuleStatus.ACTIVE` status, the `add_event_bus_handler_to_logger` function is invoked:

1. The function scans the array of existing handlers for the root logger to avoid duplication.
2. An instance of `EventBusHandler` is created, passing a reference to the active `LogDistributor`.
3. A threshold filtering level is established (defaulting to `logging.WARNING`). From this moment on, all system events of `WARN` level and higher begin translating to the event bus.

4.3. Specification of the Published Event

When dispatching a log to the bus via the `publish_to_event_bus` method, strictly typed parameters are formulated:

- **Event Type (`answer_type`):** `LOG_EVENT_TO_NOTIFY_TYPE = 'log_to_notify_event'`.
- **Payload (`payload`):** A dictionary structured as `{"text": formatted_text}` (where text is restricted to 3000 characters).
- **Service Context (`content`):**

```
{
    'level': record.levelname,      # Severity level string (WARNING/ERROR/CRITICAL)
    'source': record.name,          # Name of the originating system module
    'timestamp': record.created,    # Unix timestamp of log generation
    'is_system_log': True           # Flag indicating system origin of the log
}
```



4.2.8 Scheduling Subsystems Reference (Scheduler & Backup Scheduler)

1. Overview of Scheduler Architecture

The S.U.B.O.T.A. system separates time interval management tasks into two isolated circuits:

1. **Business Tasks and Events Circuit (Scheduler and SchedulerManager)**: A persistent and temporary scheduler with deep integration into the PostgreSQL database and the system event bus (EventBus). It operates at the third level of the core (tier=3).
2. **Infrastructure Security Circuit (BackupScheduler)**: An autonomous, isolated second-level service (tier=2) that manages the backup process of the entire system and archive rotation on the physical disk.

2. Business Tasks Circuit (Scheduler & SchedulerManager)

2.1. Circuit Topology and Parameters

- **Identification:** The module registers as an internal security service with the parameters `service_name = 'scheduler'`, `user_id = 'system'`, and `source = 'core'`.
- **Configuration:** It is initialized based on the `'location_timezone'` section.
- **External Interfaces:** It utilizes the `task_watcher` system component to register and monitor its internal execution loops.

2.2. Persistent Tasks Mechanism

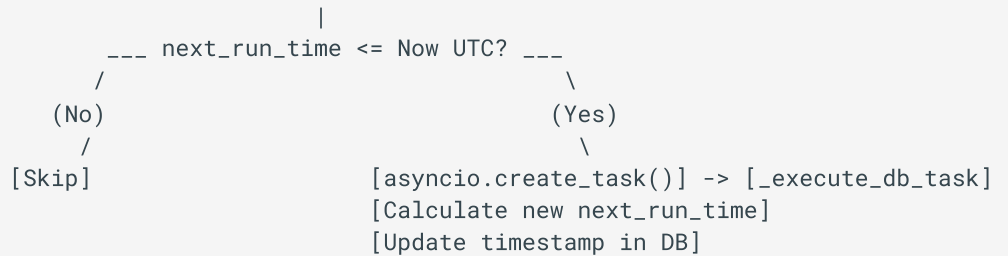
Persistent tasks are stored in the `scheduled_tasks` table of the PostgreSQL DBMS and are protected against server reboots or crashes.

Watcher Loop (_task_watcher_loop)

The scheduler registers an infinite loop named "Scheduler Loop" within the `task_watcher`, which executes at a fixed polling interval of **5 seconds**.

1. **Environment Availability Check:** On each iteration, the state of the `SystemStateManager` is polled. If `postgres != ServiceState.NORMAL`, the loop sleeps for 5 seconds, preventing the generation of cascading connection errors.
2. **Pool Loading:** All entries where `is_active = TRUE` are retrieved from the database via the `SchedulerManager`, sorted in ascending order by `next_run_time`. The DBMS manager automatically deserializes the `schedule_config` and `event_data` fields from `JSONB` /strings into Python dictionaries.
3. **Execution and Time Shifting:** For each task where `next_run_time <= current UTC time`, a non-blocking execution is initiated via `asyncio.create_task(self._execute_db_task(task))`. Simultaneously, a new execution timestamp is calculated, and the `SchedulerManager` atomically updates it in the database using an `UPDATE` SQL statement.

[illegible]



Schedule Type Matrix (`_calculate_next_run`)

- `once` : The task is intended for a single execution. After launch, the scheduler sends an asynchronous request for deactivation (`is_active = FALSE`), and the function returns `None` .
- `interval` : Dynamic time calculation. An offset computed by unpacking the configuration dictionary is added to the current UTC moment: `datetime.now(timezone.utc) + timedelta(config)` .
- `cron` : **Attention! Full support is absent.** The task is logged with a `WARNING` level, and its next execution time is forcibly shifted exactly **1 hour** forward.

Execution via the Event Bus (`_execute_db_task`)

When the target time arrives, the task is published to the `EventBus` by calling `self.publish_to_event_bus` . The `intent` parameter from the `event_data` dictionary is passed as the response type (`answer_type`) (if it is missing, a default value of `'scheduled_event'` is substituted). The contents of `task['event_data']` are passed unmodified as the payload (`content`) of the event.

2.3. Temporary Tasks (Non-Persistent Tasks)

For short-term operations that do not require database preservation, the `Scheduler` provides an isolated in-memory task storage named `self._active_tasks` .

- **Registration (`run_later`)**: Accepts a delay in seconds, a target coroutine `task_func` , and its arguments. It wraps the operation in a `delayed_task_wrapper` helper function, generates a unique ID based on the object hash `str(id(task))` , and tracks execution. Upon completion (or error), the task is guaranteed to remove itself from the active tasks dictionary via an `add_done_callback` callback function.
- **Cancellation (`cancel_task`)**: Searches for the `asyncio.Task` object in RAM by its string ID and invokes the `task.cancel()` method, returning the boolean outcome of the operation.

3. Backup Circuit (`BackupScheduler`)

The `BackupScheduler` module is isolated from the primary schedule database and interacts directly with the OS file system and the `task_watcher` scheduler under the unique loop name `"BackupScheduler Loop"` .

3.1. File Structure Analysis

At startup (`on_load`), the component calls the `_get_last_backup_date_from_disk()` method to parse the `backups/` directory. If the directory exists, files are filtered using a mask matching the prefix `app_backup_` and the extension `.tar.gz` . The date extraction is performed using the regular expression: `^app_backup_(\d{8})_\d{6}\.tar\.gz$`

The captured `\d{8}` group is converted into a `datetime.strptime(..., '%Y%m%d').date()` object. By means of mathematical comparison, the freshest backup date on the disk is determined and assigned to the `self._last_backup_date` variable.

3.2. Loop Algorithm and Backup Execution

The backup interval is read from the `schedule_interval_hours` configuration and converted into seconds (by multiplying by **3600**).

When processing an iteration (`_check_and_run_backup`), the following steps are performed:

1. **Redundancy Protection:** The current date of the operating system is checked. If `self._last_backup_date == current_date`, execution is skipped with an informational log (the system guarantees the creation of **no more than one backup per day** via the standard loop).
2. **Delegation to Subprocesses:** The manager instantiates an asynchronous operating system subprocess via `asyncio.create_subprocess_shell('./backup.sh', ...)` and awaits its complete termination (`communicate()`).
3. **Archive Rotation:** In the event of a successful script completion (exit code `0`), the manager immediately initiates the execution of a second Bash script: `./rotate_backups.sh`. A rotation failure does not halt the system; instead, it is recorded in the logs as a `WARNING`. Following this, the date of the last backup is updated to the current date.

3.3. Escalation of Critical Failures

If the `./backup.sh` script returns a non-zero exit code or the process generates an exception, the subsystem switches to emergency mode:

- The failure is recorded in the logger with an `ERROR` / `CRITICAL` severity level.
- The manager constructs an emergency data packet and places it into the core's system queue for critical infrastructure notifications, `self.critical_notifications_queue` (`interfaces.infra.emergency_queue`):

```
{
    'text': f"🚨 CRITICAL BACKUP FAILURE. Code: {process.returncode}. Error: {error_msg[:200}}",
    'urgency': 'critical',
    'timestamp': datetime.now().isoformat()
}
```

- If the loop itself crashes due to unforeseen exceptions, the thread sleeps for exactly **60 seconds**, after which it makes a repeated verification attempt, preventing an abnormal shutdown of the backup manager.

4.2.9 System Monitoring: Health Checker & Metrics Collector

1. Monitoring Subsystem Overview

The S.U.B.O.T.A. monitoring subsystem operates at the third level of the core hierarchy (`tier=3`) and is divided into two interconnected internal services (`SubotaInternalService`):

1. **HealthChecker** : Responsible for continuous connectivity checks with critical infrastructure (Redis, PostgreSQL), dynamic status management of service availability, and escalating critical failures to the emergency queue.
2. **MetricsCollector** : Asynchronously collects operating system telemetry, aggregates isolated metrics from external modules via the event broker, tracks thread statuses in `TaskWatcher` , and persists historical records to the database.

Both components utilize a shared in-memory caching endpoint — the Redis hash `subota:system:status:cache` — to provide instantaneous system status snapshots.

2. Health Monitoring Module (`HealthChecker`)

2.1. Parameters and Service Lifecycle

- **Registration Data:** `service_name = 'health_checker'` , `user_id = 'system'` , `source = 'core'` .
- **Lifecycle:** Upon initialization (`on_load`), it reads the polling interval from `config.interval_check` and spawns an infinite loop named "HealthChecker Loop" within the `TaskWatcher` scheduler. Upon shutdown (`on_unload`), it safely unregisters the loop from the scheduler and transitions to `ModuleStatus.OFFLINE` .

2.2. Connection Control Algorithm and State Machines

The service executes periodic polling of infrastructure nodes at a specified interval:

- **Redis Verification:** Invokes the client's `ping()` command. In the event of a `ConnectionError` , it generates a `SubotaMessage` with an `urgency = 'critical'` status level, dispatches it to the core infrastructure emergency queue (`emergency_queue`), and records a `CRITICAL` log entry.
- **PostgreSQL Verification:** Delegates the call to the `interfaces.postgres_manager.check_connection()` method. If an exception is intercepted, it formats a critical notification packet identical to the Redis failure routine.
- **Latency Calculation:** For both connections, the operation execution time delta is measured utilizing `asyncio.get_event_loop().time()` , and the outcome is retained in milliseconds.

State Management (Loop Match-Logic)

Based on the verification outcomes, `HealthChecker` synchronizes states within the `SystemStateManager` according to the following switching matrix:

| Initial Status in StateManager | Connection Check Result | New Service State | Subsystem Action |
|----------------------------------|-----------------------------|----------------------------------|--|
| <code>ServiceState.NORMAL</code> | <code>True</code> (Success) | <code>ServiceState.NORMAL</code> | Commits <code>Latency</code> to the system log at <code>INFO</code> level. |

| Initial Status in StateManager | Connection Check Result | New Service State | Subsystem Action |
|--------------------------------|-------------------------|-----------------------|--|
| ServiceState.NORMAL | False (Failure) | ServiceState.FAILOVER | State transition. Publishes system_warning to the event bus. |
| ServiceState.FAILOVER | True (Success) | ServiceState.RECOVERY | State transition. Dispatches a notification indicating the initiation of recovery. |
| ServiceState.FAILOVER | False (Failure) | ServiceState.FAILOVER | State remains unchanged. Dispatches a repeated warning to the event bus. |
| ServiceState.RECOVERY | True (Success) | ServiceState.RECOVERY | Wait state for complete stabilization. Logged at WARNING level. |

 **Important:** Any state transition to `FAILOVER` or `RECOVERY` accompanied by dispatching a message to the EventBus or recording an entry to the logger is strictly governed by an internal spam protection gateway (`anti_spam`).

2.3. Notification Filtering Algorithm (Anti-Spam)

To prevent cascading flood conditions within communication channels and logs during database outages, the `anti_spam()` method utilizes an atomic counter named `anti_spam_counter` :

- Allows the **first** message through (`counter == 0`).
- Subsequently, it only permits **every fourth** message through (`counter % 4 == 0`).
- In all alternative scenarios, it returns `False` , thereby blocking duplicate system alerts.

3. Metrics Collection Module (`MetricsCollector`)

3.1. Aggregation of Isolated Module Metrics (Redis Streams)

`MetricsCollector` acts as a consumer for isolated metrics generated by independent system modules.

- **Integration Circuit:** Stream `subota:metrics` , consumer group `group_metrics` , consumer name `metrics_collector_1` .
- **Time Validation (Timestamp Skew):** Upon receiving a `metrics_isolated` event type, the time delta is verified: `abs(current_time - event_timestamp)` . If the discrepancy exceeds the `MAX_TIMESTAMP_SKEW` threshold (**10 seconds**), the packet is dropped and an error is logged, shielding the system from processing stale data.
- **Payload Parsing:** Data is extracted from the `SubotaMessage.data['content']` structure. If either the `module_name` or `module_uuid` fields are undefined, the event is ignored. Successfully parsed metrics (`up_time` , `ram_used` , `cpu_used`) are committed to an internal RAM dictionary `self._isolated_metrics` under the composite key `{module_name}_{module_uuid}` .

3.2. Collection of System and Environmental Metrics

Hardware metrics collection is handled within the `_collect_system_metrics()` method, utilizing the `psutil` library alongside direct interaction with Linux kernel subsystems:

- **CPU Utilization:** Read via `psutil.cpu_percent(interval=None)`. During the initial execution or if a value of `0.0` is returned, the method automatically substitutes a pseudo-random value within a `4.0` to `8.0` % range to eliminate initialization zero-value distortion on statistical plots.
- **Processor RAM Footprint:** The `_collect_used_memory()` method computes the virtual (`vms`) and resident (`rss`) memory allocation of the current process, translating bytes into megabytes via integer division (`// 1024 // 1024`).
- **Power Supply and Battery:** The presence of a battery unit is verified via `psutil.sensors_battery()`. If the device runs on AC power or a battery, the charge percentage and connection state are retrieved.
- **Power Consumption Telemetry (Watts):** The `_get_power_consumption_watts()` method scans the `/sys/class/power_supply/` directory for `BAT*` devices:
 - *Method 1:* Reading `/sys/class/power_supply/BAT*/power_now` (the microwatt value is converted to Watts by dividing by `1000000`).
 - *Method 2 (Fallback):* If `power_now` is absent, the current `current_now` (microamperes) and voltage `voltage_now` (microvolts) are read. The final power consumption is derived using the formula:

$$\text{Power (Watts)} = \frac{\text{current_now} \times \text{voltage_now}}{10^{12}}$$

3.3. Persistence and Data Storage

The aggregation of all data types (system telemetry, health metrics from `HealthChecker`, task statuses from `TaskWatcher`) is combined into a singular unified loop `collect_and_save_all_metrics()`, which operates at an interval of `config.report_interval` seconds.

PostgreSQL Write Schema

All gathered metrics are flattened into a list of dictionaries and committed in batches via the database manager's `execute_many` method.

- **Target Table:** `system_metrics`.
- **SQL Statement:**

```
INSERT INTO system_metrics (metric_name, metric_value_float, metric_value_string, source)
VALUES ($1, $2, $3, $4);
```

4. Shared State Memory (`SYSTEM_STATUS_CACHE`)

To guarantee low-latency access to telemetry data without hitting the persistent PostgreSQL disk storage, both modules update an atomic cache in Redis concurrently.

- **Cache Key:** `subota:system:status:cache` (Data type: Hash).

```
subota:system:status:cache (Redis Hash)
├─ "health" --> {"redis": {...}, "postgres": {...}} [Updated by HealthChecker]
└─ "metrics" --> [{"metric_name": "...", "source": "..."}, ...] [Updated by MetricsCollector]
```

1. `HealthChecker` writes a serialized JSON dictionary `self.health_dict` into the `"health"` field, containing connectivity success flags, delays (`latency`), and the current state names of the core state machine.

2. `MetricsCollector` writes a complete serialized JSON array containing all aggregated metrics accumulated during the iteration into the `"metrics"` field prior to forwarding them to the long-term database.

4.2.10 System Log Processor Reference (System Log Processor)

1. Component Overview

The `SystemLogProcessor` module operates as a third-level core internal infrastructure service (`tier=3`). Its core responsibility centers on the centralized collection, processing, and persistent storage of S.U.B.O.T.A. system logs and notifications carrying a severity level of **WARN and higher**.

System Identification Constants

Upon initialization, the service registers the following static parameters within the core hierarchy:

- `service_name = 'system_log_processor'`
- `user_id = 'system'`
- `source = 'core'`
- `config_name = 'location_timezone'` (loads geographical timezone settings utilized for logging routines).

2. Lifecycle and EventBus Integration

2.1. Initialization and Subscription (`_initialise`)

During service startup (`on_load`), the component extracts the timezone configuration (`self.timezone`), binds to the `execute / fetch_all` methods exposed by the PostgreSQL manager, and registers an infinite listening loop under the name "SystemLogProcessor Subscription" within the `TaskWatcher` task scheduler.

The deployment process for the subscription circuit proceeds as follows:

1. The `self.event_bus.ensure_group_exists(SYSTEM_LOG_GROUP)` method is invoked to guarantee the presence of the consumer group within Redis. A failure at this stage triggers a `CRITICAL` log entry, and initialization is immediately aborted since operation without the message broker is impossible.
2. Subscription to the group is launched utilizing the following parameters:
3. **Group Name (`SYSTEM_LOG_GROUP`):** `'system_log_group'`
4. **Consumer Name (`SYSTEM_LOG_CONSUMER`):** `'system_log_consumer_1'`
5. **Dispatching:** An internal `event_dispatcher` method filters the incoming data stream. Processing is strictly restricted to events whose type (`event_type`) matches `LOG_EVENT_TO_NOTIFY_TYPE` (`'log_to_notify_event'`). The extracted payload is subsequently forwarded to the asynchronous handler.

Upon successful completion, the module status transitions to `ModuleStatus.ACTIVE` .

2.2. Service Unloading (`on_unload`)

When a core shutdown or service restart occurs, the `SystemLogProcessor` dispatches a command to `TaskWatcher` to forcibly terminate the subscription loop by its registered name. Following this, the base class shifts the component status to `ModuleStatus.OFFLINE` .

3. Data Schema and Log Persistence

Log persistence routines are guarded by the `@SubotaInternalService.check_payload` data contract verification decorator, guaranteeing that incoming dictionaries strictly adhere to the `SubotaMessage` structure.

Retention Mechanism (`_save_log_to_database`)

When parsing a `SubotaMessage` object, data type conversion occurs automatically: if the `timestamp` field is presented as Unix-time (floating-point or integer representation), it is automatically converted into a Python `datetime` object via `datetime.fromtimestamp()`.

Records are committed into the `system_notifications` relational table. If an SQL transaction failure takes place, the exception is logged and subsequently re-raised (`raise`) to engage the protective error handling loop.

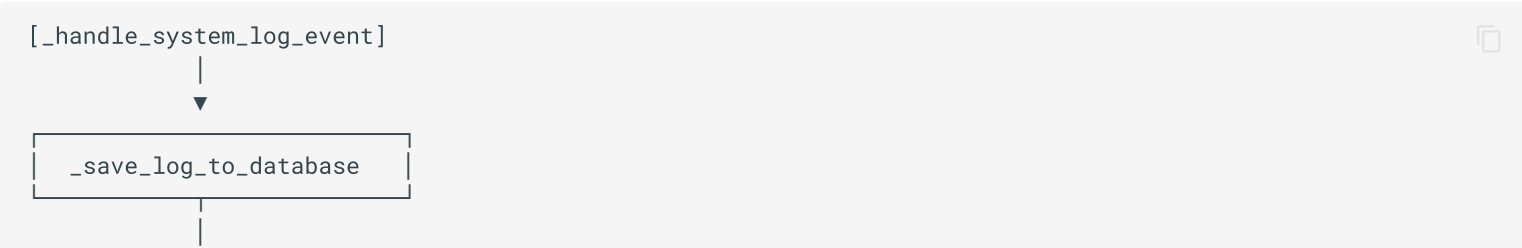
Field Mapping Structure (SQL Mapping)

```
INSERT INTO system_notifications (timestamp, level, message, source, session_id, user_id, is_read)
VALUES ($1, $2, $3, $4, $5, $6, $7);
```

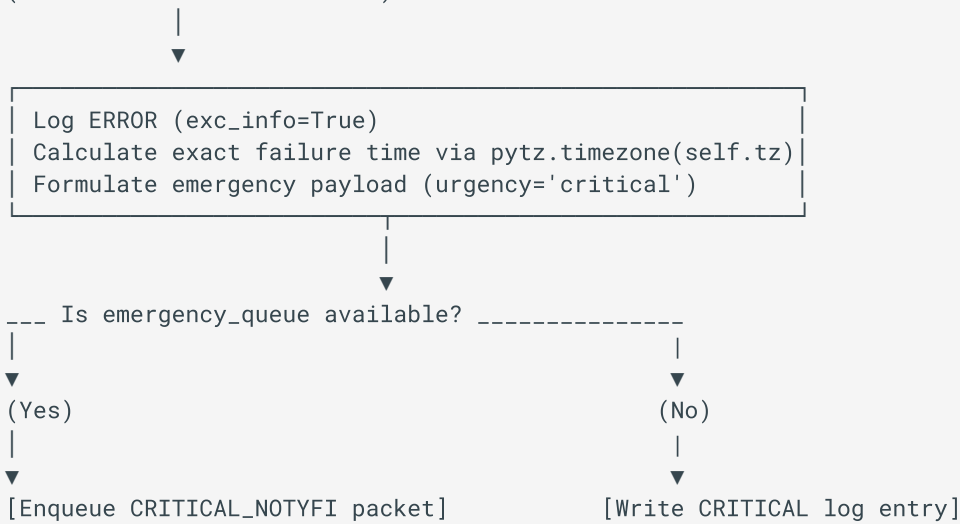
| Query Parameter | Database Column | Type / Default Value | Source from Log / Payload |
|-----------------|-----------------|----------------------|--|
| \$1 | timestamp | TIMESTAMP | log_data.get('timestamp', datetime.now()) |
| \$2 | level | VARCHAR | log_data.get('level', 'UNKNOWN') |
| \$3 | message | TEXT | log_data.get('message', '') |
| \$4 | source | VARCHAR | log_data.get('source', 'Unknown') |
| \$5 | session_id | VARCHAR | log_data.get('session_id', 'Uknown') |
| \$6 | user_id | VARCHAR / NULL | log_data.get('user_id', None) |
| \$7 | is_read | BOOLEAN | log_data.get('is_read', False) (User read status flag) |

4. Emergency Response and Fault Isolation Circuit

Should the PostgreSQL DBMS become unavailable or return a critical error during log write attempts, the `SystemLogProcessor` prevents information loss and cascading core failure by isolating the data within an internal notification queue (`emergency_queue`).



(Write Error / DBMS Failure)



1. **Incident Logging:** The error is intercepted within an `except Exception` block, outputting the complete call stack (`exc_info=True`) to the system log.
2. **Time Localization:** The service evaluates the exact local time of the incident leveraging geographical zone configuration parameters: `datetime.now(tz=pytz.timezone(self.timezone))`.
3. **Escalation:** A new critical `SubotaMessage` object is constructed, embedding the service notice text " CRITICAL FAILURE: SystemLogProcessor cannot write logs to PostgreSQL..." paired with `data.content.urgency = 'critical'` metadata.
4. The data package is atomically pushed to the core infrastructure queue `self.interfaces.infra.emergency_queue` as a dictionary structured as follows:

```
{'event_type': 'CRITICAL_NOTYFI', 'payload': notification_payload}
```



If the queue remains unreachable for any reason, a log entry at the `CRITICAL` severity level is generated.

5. Query Programmatic Interface (`get_history`)

To facilitate log reading routines for external interfaces or administrative modules, `SystemLogProcessor` exposes a `get_history` method supporting dynamic SQL query assembly, pagination, and full-text search capability.

```
async def get_history(limit: int = 100, offset: int = 0, level: Optional[str] = None, search: Optional[str] = None) -> List[Dict[str, Any]]
```



Filtering and Pagination Mechanism

Queries are constructed utilizing typed placeholders (`$1`, `$2`, ...) to ensure robust protection against SQL injection attacks handled at the `asyncpg` driver level:

- **Log Severity Level (`level`):** Providing this argument injects a strict evaluation constraint defined as `level = $param_idx`, forcing the string evaluation format into uppercase (`.upper()`).
- **Full-Text Search (`search`):** Providing a search query parameters constraint formulates a case-insensitive filter leveraging the `ILIKE` operator evaluated simultaneously across two distinct database table columns:


```
(message ILIKE $param_idx OR source ILIKE $param_idx_plus_one)
```



The search string value is escaped utilizing percent wildcards: `f"%{search}%"`.

- **Sorting and Offsetting:** Entries are invariably returned sorted chronologically from newest to oldest: `ORDER BY timestamp DESC`. The `LIMIT` and `OFFSET` pagination boundaries are mandatory parameters enforced during query generation, concluding the query argument array.

The query execution result set is converted from database system records (`asyncpg.Record`) into standard Python dictionaries (`dict(record)`). If exceptions manifest during the data extraction cycle, the method returns an empty list `[]`.

4.2.11 Technical Specification: Number and Text Processing

1. Functional Purpose and Subsystem Composition

The module is responsible for the two-way conversion between text representation and numerical values, as well as for parsing and evaluating simple mathematical expressions passed in natural language (Russian language).

The subsystem consists of two isolated modules:

- `nums_to_text.py`: Conversion of numerical types (`int`, `float`) into a text string with support for inflection (noun declension) for predefined units of measurement (degrees, percentages, time, etc.).
- `text_to_nums.py`: Tokenization of a text string, mapping substrings to dictionaries of numerals and mathematical operators, building and evaluating expressions taking into account operator precedence.

2. Critical Architecture Analysis and Algorithmic Bugs

The current implementation contains critical architectural flaws, logical errors, and gross violations of energy efficiency (minimization of CPU consumption). The code is written without taking into account the asynchronous context of the core (`asyncio`) and Python 3.13+ standards.

Critical Business Logic Bugs (Catastrophic Evaluation Failures)

- Algorithmic Collapse of the Composite Number Parser** (`parse_decimal_number`): Inside the `parse_decimal_number` function, the following logic is used:

```
current_number = current_number * 10 + num_dict[closest_match]
```

This works exclusively for digit-by-digit input (for example, "два пять" -> 25). If the user says "двадцать пять", the program will take the value `20` from the dictionary, then in the next iteration multiply it by 10 and add `5`. Result: `20 * 10 + 5 = 205` instead of `25`.

- Breakage of Large Numbers Bit Depth** (`parse_number`): The `parse_number` function performs flat addition: `result += num_dict[closest_match]`. The expression "две тысячи" will turn into `2 + 1000 = 1002` instead of `2000`. The logic is completely incapable of handling order-of-magnitude multipliers.
- Crash on Unary Operators** (`evaluate_expression`): In the `evaluate_expression` function, the processing of the `sqrt` operator is highlighted as unary (`return operator_functions[operator](right)`). However, the evaluation loop itself rigidly slices stack elements as if for a binary operation:

```
left = numbers[i]
right = numbers[i + 1] # Will raise IndexError if there is only one number in the stack
numbers[i:i + 2] = [result] # Replaces two elements with one
```

When attempting to evaluate "корень девяти", the interpreter will either throw an `IndexError` or destroy the adjacent number in the operations stack.

Violation of Energy Efficiency and Performance

- **CPU-bound Overengineering in Loops (`difflib.get_close_matches`):** For each word in the input string, the `find_closest_input` function is called, which internally executes `gcm()` (Greenwood substring search algorithm) with `cutoff=0.8` over the full `NUMS_RAW` dictionary (more than 40 elements). Performing fuzzy string comparison for each token in a loop loads the CPU and irrationally wastes host energy.
- *Solution:* Replace with a hash table (`dict`) with $O(1)$ for exact matches. Apply fuzzy search only as a fallback if no exact match is found.

✚ Violation of Modularity and Code Cleanliness

- **Hardcoded Localization (Hardcode):** In the `parse_number_expression` function, there is a direct check for a syntax delimiter: `if 'и' in words:`. This makes the module absolutely unextendable to other languages and violates the isolation of data from code.
- **Breakage of Inflection on Real Numbers (`nums_to_text`):** The calculation of the remainder of division (`num % 10` and `num % 100`) in the `num_to_text` function is correct only for integers (`int`). Passing a `float` (for example, `23.5 градуса`) will lead to unpredictable behavior of `if/elif` branches and distortion of the output of units of measurement.
- **Anti-pattern "Silencing Errors":** The `except Exception as e` block in `num_to_text` returns `str(num)` without any logging. Localization errors or failures of the `num2words` library will be hidden from the core monitoring system.

3. Interaction of Components (Data Interfaces)

`modules/nums_to_text.py`

- `num_to_text(num: int|float, unit: dict = None) -> str`: Converts a number into text with the substitution of case forms.
- `format_time(hours: int, minutes: int) -> str`: Wrapper for joining hours and minutes.

`modules/text_to_nums.py`

- `reading_start(string_input: str) -> float | str`: Entry point for parsing mathematical expressions from text. Returns a rounded result or the placeholder `"The calculator is broken"` in case of exceptions.

4.2.12 Authentication Subsystem and RBAC (API Reference)

This document contains a detailed programmatic description of the components of the role-based access control (RBAC) subsystem of the S.U.B.O.T.A. core. The subsystem provides permission checks both at the DBMS internal identifier level and during interaction with external interfaces (Telegram handlers, CLI commands).

1. RBACManager Class

1.1. Programmatic Context and Initialization

The `RBACManager` class is located in the `core/rbac_manager.py` module. It functions as an internal trusted Tier 3 core service and inherits from the `SubotaInternalService` base class.

Upon instantiation, the component hardcodes the following system parameters:

- `service_name: 'rbac_manager'.`
- `user_id: 'system'.`
- `source: 'core'.`
- `config_name: None` (there is no dedicated configuration file).
- `tier: 3.`

Upon object creation, it is assigned the `ModuleStatus.STARTING` status. At the moment the asynchronous initialization method `on_load()` is executed, the component dynamically extracts the SQL query execution method `fetchrow` from the injected `postgres_manager` interface and transitions to the `ModuleStatus.ACTIVE` operational status. When the module is unloaded via `on_unload()`, the status is reset to `ModuleStatus.OFFLINE`.

1.2. Integration with Autogeneration (mkdocstrings)

The following program block is used to automatically build documentation from source code docstrings:

```
core.rbac_manager.RBACManager()
```

Bases: `SubotaInternalService`

Менеджер для проверки прав доступа на основе ролей (Role-Based Access Control).

```
check_permission(role_name, permission_name) async
```

Проверяет, имеет ли указанная роль требуемое разрешение. Возвращает True, если разрешение есть, иначе False.

```
get_user_role(user_id) async
```

Получает имя роли пользователя по его ID.

```
get_user_role_by_identifier(identifier) async
```

Получает имя роли пользователя по его строковому Telegram ID.

1.3. Logic of Public Methods

```
get_user_role(user_id: int) -> str
```

- **Purpose:** Retrieving the user's role name by their internal numerical identifier in the DBMS.
- **Type Validation:** The method performs a strict `isinstance(user_id, int)` check. If an incorrect type is passed, the incident is logged with the `ERROR` level, and the method immediately returns the default value `'guest'`, bypassing the DB query.
- **SQL Logic:** Executes a JOIN query to the `users` and `roles` tables on the condition `u.id = $1`. If there is no record in the DB, a `WARNING` is registered, and the user is assigned the `'guest'` role.

```
check_permission(role_name: str, permission_name: str) -> bool
```

- **Purpose:** Verifying the binding of an atomic permission to a specific role.
- **Type Validation:** Accepts strictly string values for arguments (`isinstance(..., str)`). Type violations are logged as `ERROR` and return `False`.
- **Case Conversion:** Both string arguments are forcibly converted to lower case (`.lower()`) before analysis.
- **Bypass Logic (Hardcode):** If `role_name == 'admin'`, the method short-circuits (triggers instantly) and returns `True` without executing an SQL query.
- **SQL Logic:** For all other roles, it executes a predicate query `SELECT EXISTS(...)` against the `role_permissions` junction table.

```
get_user_role_by_identifier(identifier: str) -> str
```

- **Purpose:** Finding a user's role based on the string representation of their Telegram ID.
- **Validation and Parsing:** The argument is checked for compliance with a string (`isinstance(identifier, str)`). Next, an explicit type casting `int(identifier)` is performed inside a `try...except ValueError` block. In case of failure (a non-numeric string), an execution error is recorded and `'guest'` is returned.
- **SQL Logic:** Performs a search by the field `u.telegram_id = $1`.
- **Event Bus:** If a user with the specified Telegram ID is not found in the DB, the method generates and sends an asynchronous event of type `'publish_notification'` to the `RedisEventBus`. The destination (`destinations`) of the payload is `'web_ui'`, and the criticality level is passed with a specific flag `"ugrgency": 'warning'` (see the Implementation Specifics section). The method returns the `'guest'` role.

2. Access Interception Tools (rbac_tools.py)

The `core/rbac_tools.py` module provides applied tools for declarative access control for asynchronous functions of the circuit (aiogram Telegram handlers, CLI command line interface commands).

2.1. PermissionDenied Exception

```
class PermissionDenied(Exception):  
    """Exception raised when a permission is missing."""  
    pass
```



Generated at runtime if the user's session does not possess the required privilege level to call the target function.

2.2. require_permission_non_api Decorator

A declarative decorator to restrict access at the application layer level.

Operational Mechanics and Context Extraction:

- Finding the Event Object:** Inside the `wrapper` function, a search for the `aiogram.types.Message` object takes place. First, the named argument `kwargs.get('message')` is checked. If it is missing, the decorator checks the first positional argument `args[0]` for compliance with the `types.Message` class.
- Identification:** The numerical identifier `message.from_user.id` is extracted from the message object, which is forcibly converted to the string type `str` and saved as `unique_identifier`. If the message object is empty or does not contain an ID, a `PermissionDenied` exception is raised.
- Verification via Core:** The decorator calls the asynchronous method `rbac_manager.get_user_role_by_identifer(unique_identifier)` to determine the role, and then verifies the permission via the `check_permission` method.
- Call Signature Modification:** In case of successful verification, the decorator **removes** the `user_id` argument (if it was present) and forcibly injects the obtained string `unique_identifier` into `kwargs` before passing control to the original function. If the permission check fails, the call is blocked, the event is logged with the `WARNING` level, and a `PermissionDenied` exception is thrown describing the missing right.

3. Implementation Specifics and Documented Technical Debt

When developing and integrating authentication modules, specific patterns and trade-offs of the current code assembly must be taken into account:

- Administrator Bypass:** The `check_permission` method contains a hardcoded condition for the `admin` role (`return True`). This means that any permission restrictions for the administrator configured in the `role_permissions` relational table are ignored at the code level.
- Event Bus Payload Specifics (Typo in Field):** When generating a notification about an unknown Telegram user in the `get_user_role_by_identifer` method, the metadata dictionary (`content`) contains a key with the specific name `"ugrgency": 'warning'` (the letter 'r' instead of 'e'). When manually handling system events on the `web_ui` side, it is necessary to parse exactly this key name to avoid losing filtering.
- Type Conversion:** The `telegram_id` field in the database schema is defined as a `BIGINT` type. The application layer decorator extracts it as a `str`. The `get_user_role_by_identifer` method performs a forced casting to `int()` before sending the SQL query. This ensures the correct utilization of the DBMS unique index.

4.3 Services

4.3.1 Integration Services (Bridges): Authorization and Connection Control

1. Architecture of the Integration Circuit

The S.U.B.O.T.A. system interacts with the external physical world and automation brokers (such as MQTT) through specialized isolated components — **Bridges**. Interaction with them from the core side relies on the `SubotaExternalService` abstraction and is divided into two baseline circuits:

1. **Authentication Circuit** (`BridgeAuthService`): Provides a secure initial handshake of the external device with the core.
2. **Activity Control Circuit** (`ExternalBridgeHeartbeat`): Conducts continuous monitoring of network latency and manages the logical presence flag of the bridge in the system.

2. Bridge Authorization Service (`BridgeAuthService`)

The service processes incoming registration requests from external bridges within the secure peripheral awakening paradigm.

2.1. Bus Integration Circuit

- **Registration Flow (Stream):** `subota:register:request`.
- **Processed Event Type** (`event_type`): `MASHIN_SPIRIT`.
- The module registers a background worker named `"bridge_auth_service_loop"` in `TaskWatcher`. Upon successful initialization, it transitions to the `ModuleStatus.ACTIVE` status.

2.2. Handshake Protocol "Awakening of the Machine Spirit"

Upon receiving the `MASHIN_SPIRIT` event, the service initiates a strict cryptographic validation procedure for the passed parameters:

1. **Re-registration Check:** The service checks the current state of the bridge via the `bridge_MQTT_Bridge_initialized` and `bridge_MQTT_Bridge_online` events. If the bridge is already online, the repeated request is treated as a compromise or hardware failure. The system aborts execution, records a warning in the logger, and sends an incident report to the Security Center with the violation type `BRIDGE_IS_ONLINE`.
2. **Secret Extraction:** The reference bridge authorization token `mqtt_bridge_number` (service designation — `holy_fraze`) is requested from the secure `SecretManager` storage. If it is missing, the procedure is aborted by raising an `AssertionError`.
3. **Cryptographic Fingerprint Calculation:** A dynamic salt (`nonce`) and the client hash `alfarium` are extracted from the incoming payload. Authenticity calculation on the core side is executed using a SHA-256 double hashing algorithm:

$$hash_result = \text{SHA256}(\text{str}(\text{holy_fraze}))$$

$$\omega = \text{SHA256}(hash_result + \text{nonce})$$

1. **Activation:** If the calculated `omega` is strictly equal to the passed `alfarium`, authorization is considered successful (`holy_machine = True`). The service publishes the `spirit_response_True` event into the core's internal bus, passing the

original content to complete the initialization of adjacent services. If the hashes do not match, an authorization error `Wrong Ommissiah code` is generated.

3. Activity Control Service (`ExternalBridgeHeartbeat`)

The module performs a two-way poll of the `MQTT_Bridge` , calculates network metrics, and is responsible for the emergency drop of sessions in case of packet loss.

3.1. Architectural Flows and Events (Heartbeat Circuit)

Interaction with the bridge is strictly divided into two dedicated streams of the Redis Streams message broker:

- `subota:bridge:core` (**Inbound Flow / TO_CORE**): Receipt of responses (`PONG`) and initiative requests from the bridge.
- `subota:core:bridge` (**Outbound Flow / FROM_CORE**): Sending core requests (`PING`) and responses outward.

The service deploys three parallel bus event handlers via `TaskWatcher` :

- `briedge_starting_handler` (`"ExternalBridgeHeartbeat_briedge_starting_handler"`): Listens for the internal core event `spirit_response_True` (the signal of successful authorization in `BridgeAuthService`). Upon receiving it, it sends the startup packet to the bridge via the `first_ping_bridge()` method and starts the main polling loop.
- `pong_handler` (`"ExternalBridgeHeartbeat_pong_handler"`): Intercepts `BRIDGE_TO_CORE_PONG` events from the inbound flow, confirming device activity.
- `ping_handler` (`"ExternalBridgeHeartbeat_ping_handler"`): Listens for `CORE_TO_BRIDGE_PING` events. This allows the bridge to act as the initiator of the core availability check.

 **Session Security:** The methods for processing responses (`pong_verifier()`) and initiative pings (`_respond_to_ping()`) are protected by the `@SubotaExternalService.secure_event` decorator, which prevents session data tampering at the moment the control loops are executed. In all event handlers, the validity of the packet timestamp is strictly verified based on the `max_timestamp_skew` delta value from the configuration. If the data is obsolete, the packet is dropped.

3.2. Latency and State Analysis Algorithms (Internal Loops)

Ping Sending Loop (`_ping_loop_bridge`)

Performs regular broadcast sending of `CORE_TO_BRIDGE_PING` packets into the outbound stream `subota:core:bridge` at the `config.ping_timeout` interval. Sending is paused if the internal event `_ping_pause` is set to a signaled state (for example, while waiting for an overdue response). Each transmission logs the current Unix timestamp in the `self._last_ping` variable.

Network Load Analysis Loop (`_latency_loop`)

Awaits the setting of the `bridge_MQTT_Bridge_initialized` flag on each iteration. Based on the calculated time difference between packet transmission and receipt in milliseconds (`self._bridge_latency`), the service classifies the state of the system's transport layer:

| Latency (Δt , ms) | Logger Gradation | Informational Message in the Log |
|-----------------------------|------------------|----------------------------------|
| $\Delta t = 0$ | INFO | No network load. Latency: 0 ms. |

|| \$0 < \Delta t \le 100\$ | INFO | Very low network load. Latency: X ms.

|| \$100 < \Delta t \le 1000\$ | INFO | Low network load. Latency: X ms.

|| \$1000 < \Delta t \le 5000\$ | INFO | Medium network load. Latency: X ms.

|| \$5000 < \Delta t \le 10000\$ | WARNING | High network load. Latency: X ms.

|| \$\Delta t > 10000\$ | ERROR | Very high network load. Latency: X ms.

|

Connection Integrity Control Loop (`_bridge_cheker_loop`)

Starts at the `config.bridge_pong_timeout` interval and performs the functions of a watchdog timer (Watchdog):

- If the current time since the receipt of the last legitimate packet (`time.time() - self.last_pong`) exceeds the permissible timeout limit, the service raises the polling pause flag `_ping_pause` , forcibly sends an out-of-turn single ping, and increments the missed packet counter `_missed_pings` .
- **Session Drop on Emergency:** If the missed response counter exceeds a critical threshold (`self._missed_pings > 3`), the bridge is declared inoperable. Infrastructure system flags are atomically reset (cleared): `bridge_MQTT_Bridge_online` and `bridge_MQTT_Bridge_initialized` , and a connection loss warning is output to the log.

Reaction to Initiative Polling from the Bridge (`_respond_to_ping`)

In the event that an external bridge independently sends a `CORE_TO_BRIDGE_PING` ping request into the inbound flow, the core instantly calculates the reverse transmission latency "Bridge-to-Core" (`__bridge_to_core_latency`), compiles a `CORE_TO_BRIDGE_PONG` response packet, and sends it back to `subota:core:bridge` . Based on the operation results, a consolidated diagnostic matrix containing total latency, internal leg latency, and the current missed packet stack is output to the log.

4.3.2 Isolated Runner Component Specification (Isolated Runner)

This document contains a detailed software and technical specification of the `IsolatedRunner` component designed to offload resource-intensive or potentially unstable computational tasks (e.g., BERT-based NLP models) into isolated operating system processes. This guarantees the overall fault tolerance of the S.U.B.O.T.A. core in the event of failures within application modules.

1. Architectural Context and Initialization

The `IsolatedRunner` class is located in the `core/services/isolated_runner.py` module. It operates as a Tier 3 core internal service and inherits from the `SubotaInternalService` base class.

1.1. Base System Constants

Upon instantiation and registration within the event bus, the component utilizes rigidly fixed environment parameters:

- `service_name`: `'isolated_runner'` (the value of `BASE_NAME`).
- `source`: `'core'`.
- `user_id`: `'system'`.
- `tier`: `3`.
- `stream_name` for metrics: `'subota:metrics'`.
- `stream_event` for metrics: `'metrics_isolated'`.
- The names of groups and consumers are formatted with the prefixes `group_isolated_runner` and `consumer_isolated_runner_1`.

1.2. Time Limits and Timeouts (Constants)

The stability of operation execution is regulated by four key timeouts (in seconds):

- `WORKER_START_TIMEOUT = 20`: The time limit for launching a worker process and receiving a `READY` signal from it.
 - `WORKER_CALL_TIMEOUT = 15`: The time limit for non-blocking wait of a response from the communication channel (`Pipe.poll`) during a single call to a worker method.
 - `REQUEST_TIMEOUT = 10`: The maximum time a request can spend inside the asynchronous queue/lock before execution begins.
 - `METRICS_TIMEOUT = 30`: The frequency of collecting and sending worker resource consumption metrics to Redis.
-

2. Inter-Process Communication (IPC) and Lifecycle

Management of the isolated process is built upon the standard `multiprocessing` package using the `"spawn"` context, which guarantees clean memory allocation without inheriting the state of the parent OS process.

2.1. Launching a Worker (`_run_isolated_module`)

1. The method generates a unique text token for the process: `__uuid_process = str(uuid.uuid4())[ :8 ]`.

2. A duplex communication channel is created via `ctx.Pipe()`. The parent end (`_parent_conn`) remains in the main process, while the child end (`child_conn`) is passed to the worker.
3. The target entry point (`IsolatedRunner.isolated_worker_loop`) is launched in a separate process with a name structured like `{working_module_name}_{uuid}`.
4. The main process transitions into an asynchronous wait state (`loop.run_in_executor`) for the "READY" string from the child process via the created `Pipe`. If the signal is not received within `WORKER_START_TIMEOUT` or if an `EOFError / worker_error` is detected, the process is forcibly terminated (`process.kill()`), and the runner transitions to `ModuleStatus.OFFLINE`.

2.2. Static Worker Loop (`isolated_worker_loop`)

Executes completely within the isolated OS process:

1. It accepts a `class_path` string path and dynamically splits it from the right using `.rsplit(".", 1)` into a module path and a class name.
2. It imports the module via `importlib.import_module`, instantiates an object of the target class, and sends a "READY" signal to the parent.
3. It enters an infinite `while True` loop, blocking on channel reads: `pipe_conn.recv()`.
4. Upon receiving a request dictionary, it extracts `method` and `kwargs`. If the method is asynchronous (`inspect.iscoroutinefunction`), it is executed via `asyncio.run()`; otherwise, it is invoked directly. The execution result is sent back into `pipe_conn`.
5. Any unhandled exception inside the worker is intercepted, formatted via `traceback.format_exc()`, and sent to the parent as a dictionary with the "worker_error" key.

3. Fault Tolerance and Self-Healing

Communication with the worker is protected against freezes and crashes using a cascade of checks and restart mechanisms.

3.1. I/O Thread Isolation (IPC Executor)

All data exchange operations over the `Pipe` are offloaded to a dedicated thread pool `ThreadPoolExecutor(max_workers=1, thread_name_prefix="ipc_worker")`. This prevents blocking the main event loop (`asyncio loop`) if OS read/write system calls hang. Access to the executor is synchronized via `asyncio.Lock()`.

3.2. Failure Handling Strategy During Calls (`_call_isolated_process`)

Invoking a worker method is wrapped in a loop of 2 attempts (`attempt in range(2)`):

- **Activity Verification (`_ensure_worker_alive`):** Before sending data, `process.is_alive()` and the presence of a connection are verified. If the worker is dead, a transparent automatic restart is initiated before the command is sent.
- **Execution Timeout:** Data exchange inside the thread is monitored by the `conn.poll(WORKER_CALL_TIMEOUT)` method. If the worker hangs while processing a task longer than the specified time, a `TimeoutError` is raised, the parent process immediately performs a hard kill on the hung worker (`self._isolated_process.kill()`), releases the pool thread, and returns `None`.
- **Forced Reset (`_reset_worker`):** If an unexpected exception or channel disconnection (`EOFError`) is caught on the first attempt, the `_reset_worker` method is invoked. It terminates the process, closes the old `Pipe`, clears readiness events, and prepares the runner to spawn a clean process on the next iteration.

4. Declarative Processing Pipelines (Pipeline Execution)

The component implements the concept of declarative execution of chains of responsibility (`pipelines`) described as lists of configuration dictionaries.

4.1. Resolution of Dependencies and Parameters

When executing pipeline steps, the runner dynamically computes the targets of methods and their arguments:

- `_resolve_source(source_str)` : Converts a string description into an object. The string `'self'` returns the runner instance itself, while constructs like `'self.interfaces.context_manager'` sequentially traverse attributes via `getattr()` .
- `_resolve_param(path, payload, internal_state)` : Extracts argument values based on prefixes:
- If a string begins with `"payload."` , data is read from the incoming Redis event (`SubotaMessage`).
- If a string begins with `"self."` , the runner first searches for the key in the internal isolated state dictionary of the current pipeline (`internal_state`), and if absent, reads the attribute from the actual `self` object.

4.2. Pipeline Structure (Using IntentProcessor as an Example)

The system contains a predefined intent processing pipeline `intent_processor_pipeline` consisting of 5 sequential steps:

| Step | Action
(<code>action</code>) | Method
(<code>method</code>) | Source (<code>source</code>) | Input Parameters
(<code>kwargs</code>) | Output Buffer
(<code>save_to</code>) |
|------|---|--|--|---|---|
| 1 | <code>call</code> | <code>get_context</code> | <code>self.interfaces.
context_manager</code> | <code>user_id ←
payload.source_user_
id</code>

<code>key ←
'last_target_id'</code> | <code>self.context</code> |
| 2 | <code>call_isolated</code>
<code>ed</code> | <code>process_inte</code>
<code>nt</code> | <code>self._isolated_p</code>
<code>rocess</code> | <code>raw_intent ←
payload.text</code>

<code>context ←
self.context</code>

<code>script_docs ←
self.interfaces.infr
a.scrip_docs</code> | <code>self.intent</code> |
| 3 | <code>call</code> | <code>save_context</code> | <code>self.interfaces.
context_manager</code> | <code>user_id ←
payload.source_user_
id</code>

<code>key ←
'last_target_id'</code>

<code>value ←
self.intent.target_i
d</code> | <i>None</i> |

| Step | Action
(action) | Method
(method) | Source (source) | Input Parameters
(kwargs) | Output Buffer
(save_to) |
|------|----------------------|--------------------------|-------------------|--|------------------------------|
| 4 | call | publish_to_e
vent_bus | self | answer_type ←
self.intent.answer_t
ype

content ←
self.intent.content | None |
| 5 | call | publish_to_e
vent_bus | self | answer_type ←
'last_intent_stream'

content ←
self.intent.content | None |

Note: Step #2 (`call_isolated`) serves as a context switch point for execution and redirects task completion to the isolated OS worker process.

5. Monitoring, Metrics, and Management

5.1. Resource Metrics Collection (`_metrics_loop`)

If the worker is active, the runner's background task queries operating system process parameters via the `psutil` library at a frequency defined by `METRICS_TIMEOUT` . Based on the retrieved PID, the following are computed:

- Resident memory size (`RSS`) in megabytes (`mem_info.rss / 10242`).
- CPU core utilization percentage (`process.cpu_percent(interval=0.1)`).
- Worker continuous operation time (Uptime) in seconds, rounded to two decimal places.

The data is packaged into a `metrics_isolated` event and published to the Redis system stream `'subota:metrics'` .

5.2. Remote Module Management (Forced Restart)

The `_initialize_module_control` method subscribes the runner to control events of the metrics stream. When a command formatted as `RESTART_MODULE_{working_moduel_name}_{uuid}` is sent to the bus, the runner performs a hot reload procedure for the worker:

1. The inter-process lock `_ipc_lock` is acquired to avoid collisions with active requests.
2. The full stop procedure `_shutdown_module()` is called, closing communication channels and terminating the process.
3. A new worker instance is spawned based on the saved class manifest.

Security: The restart request must be validated by the `@SubotaInternalService.check_payload` decorator. If the `payload.key_id` field (the command source) is empty or undefined, the runner ignores the instruction and records a `WARNING` level event.

4.3.3 Power Daemon

This document contains a software and technical description of the `PowerDaemonService` component, which provides low-level monitoring of the physical power lines of the server host, processes telemetry from external Android sensors, caches the aggregated state of the power subsystem in Redis, and dispatches alerts regarding power supply degradation or loss.

1. Architectural Context and Initialization

The `PowerDaemonService` component is located in the `core/services/power_daemon.py` module. It functions as a core internal service, inheriting from the `SubotaInternalService` base class.

1.1. Constants and Global Tokens

When registering within the core and the event bus, the service uses the following fixed identifiers:

- `service_name`: `'power_daemon_service'` (the value of `BASE_NAME`).
- `source`: `'core'`.
- `user_id`: `'system'`.
- `config_name`: `'power_daemon'` (dedicated `PowerDaemonConfig` structure).

1.2. Redis Stream and Storage Addressing

Operational data exchange and event publication are performed using the following keys:

- `REDIS_POWER_STATE_KEY` = `"power:state:cache"`: Storage for the serialized JSON object of the aggregated power state.
- `SYSTEM_STATUS_CACHE` = `"subota:system:status:cache"`: A Redis hash table (`HSET`) where the `power_status` field is written for adjacent services.
- `SYSTEM_STATUS_STREAM` = `"subota:internal:system:status"`: The target system stream for critical sensor alerts.
- `METRICS_STREAM_NAME` = `"subota:metrics"`: The general stream for system metrics.

2. Integration with Auto-generation (mkdocstrings)

The following software block is used for automatically building documentation from the header strings of the source code:

```
core.services.power_daemon_service.PowerDaemonService()
```

Bases: `SubotaInternalService`

```
parse_power_supply(raw)
```

Парсит сырые данные `/sys/class/power_supply`.

Возвращает: { 'ac_online': bool, 'battery': { 'capacity': int, 'status': str, 'voltage_now': float, 'current_now': float, 'charge_now': float, 'charge_full': float, 'charge_full_design': float, 'cycle_count': int, 'technology': str, 'model': str, 'manufacturer': str, 'degradation': float } }

```
process_sensor_message(payload) async
```

Парсит данные о питании Android-датчиков и обновляет power_dict.

```
read_power_supply()
```

Читает сырые данные из /sys/class/power_supply.

3. Service Lifecycle and Subscriptions

3.1. The on_load() Method and Initialization

Upon loading, the service extracts configuration parameters, the system core stop event `core_stop_event`, as well as the asynchronous Redis client via the `redis_connection_manager` interface based on the retrieved credentials (`self.credits`).

The internal asynchronous method `_initialize()` performs the following operations:

1. Guarantees the existence of the consumer group `group_power_daemon_service` for the general stream and the dedicated channel `subota:sensors:stream`.
2. Registers an asynchronous handler `event_handler`, which filters incoming events by the `sensor_message` type and redirects them to the `process_sensor_message` method.
3. Initiates background monitoring tasks via `task_watcher`: the physical host polling loop (`_host_power_loop`) and the event bus subscription.

Upon successful completion of the procedures, the service status transitions to `ModuleStatus.ACTIVE`.

3.2. The on_unload() Method

When unloading the module, the `task_watcher` forcibly stops three registered dispatcher tasks:

- "PowerDaemonService Loop Subscription sensors"
- "PowerDaemonService Loop Subscription DTEK"
- "PowerDaemonService Loop Main Power"

After this, the status is reset to `ModuleStatus.OFFLINE`.

4. Monitoring Host Physical Power

The background loop `_host_power_loop` continuously polls files in the Linux virtual file system at the path specified in the configuration (`power_supply_path`, usually `/sys/class/power_supply`) at intervals of `self._config.poll_interval`.

4.1. Reading and Parsing Linux Kernel Virtual Interfaces

- `read_power_supply()` : Traverses all directories within the specified path. For each device, it reads the text contents of internal property files (e.g., `status`, `capacity`, `online`) into a raw dictionary.
- `parse_power_supply(raw)` : Performs normalization of raw data. Devices with the `Mains` type determine the external AC grid presence flag: if `online == "1"`, the `ac_online` parameter is set to `True`. For devices with the `Battery` type, numerical values of voltages, currents, and charges are divided by `1 000 000` to convert from Linux kernel micro-units to standard values (`Volts`, `Amperes`, `Ampere-hours`).

If the actual capacity (`charge_full`) and design capacity (`charge_full_design`) parameters are present, the current percentage of hardware degradation (wear) of the battery is calculated using the formula:

$$\text{degradation} = \left(1 - \frac{\text{charge_full}}{\text{charge_full_design}}\right) \times 100$$

4.2. Processing Power Switch Triggers

When a change in the external network flag (`current_ac`) is detected, atomic events for switching host operating modes occur:

```
[Mode: AC Power] → Power Loss (current_ac = False) → Call _handle_power_loss()
|
▼
[Mode: Battery] ← AC Restoration (current_ac = True) ← Call _handle_power_restore()
```

The `_handle_power_loss()` Method

1. Captures the string time of the accident `_offline_start_time` (format `%H:%M %d.%m`) taking into account the local timezone `location_timezone` and saves the float timestamp `_offline_start_time_float` via `time.time()`.
2. Modifies the internal state `power_dict`, adding the `'power_lost'` token.
3. Publishes the `POWER_LOSS_DETECTED` event to the bus and updates the cache in Redis.

The `_handle_power_restore()` Method

1. Computes the duration of the power outage using `_format_offline_duration()` (formatting the time difference into a `hh:mm:ss` string). If the service was restarted during battery operation, the duration is marked as `"unknown..."`.
2. Formats the `power_status` dictionary, specifying the offline start time, recovery time, and total duration of the accident.
3. Generates two events into the event bus: `POWER_RESTORED` for internal systems and an external notification `publish_notification` for delivery to the `telegram` client interface with a criticality level of `warning`.
4. Resets the float tracker to `0.0` and updates the cache.

4.3. Threshold Control of Server Battery Discharge

In autonomous mode (`not self.is_ac_online`), the daemon checks the current capacity `battery_pct` relative to the configuration limits on each tick:

- If the charge level is less than or equal to `battery_critical_pct`, a `BATTERY_CRITICAL` event is generated.
- If the charge level is less than or equal to `battery_low_pct`, a `BATTERY_LOW` event is generated.

Both events trigger a forced overwrite of the cache in Redis.

5. Integration with External Sensors (Android Sensors)

The service aggregates data from external Android devices acting as remote grid monitoring sensors at points of presence.

5.1. Telemetry Parsing (`process_sensor_message`)

The method is protected by the `@SubotaInternalService.check_payload` decorator and accepts a validated `SubotaMessage` object.

1. Data from the `payload.data["content"]` container is unpacked into a strictly typed Pydantic schema `SensorMessage`.
2. A unique device string token `device_id ( sensor_name )` and the internal battery parameters dictionary `battery` are extracted from the message.
3. Device parameters are converted to lowercase and normalized into the sensor data structure:
4. `{sensor_name}_uptime`: The uptime of the sensor's OS.
5. `{sensor_name}_bat_plugged`: The fact of connection to a charger (`str.lower()`).
6. `{sensor_name}_bat_status`: The battery status of the sensor (e.g., `charging`, `discharging`).
7. `{sensor_name}_bat_percent`: The current charge level.
8. The resulting data block is saved in `self.power_dict["sensors"][sensor_name]` and published to Redis.

5.2. Battery Charge Control of External Sensors

After parsing, the `_check_sensor_battery` method is invoked. If the sensor's charge level falls below or equals the configured value `sensor_low_battery_pct`, the daemon performs the following actions:

1. Logs a warning at the `WARNING` level.
2. Formats a `SENSOR_LOW_BATTERY` event packet containing the sensor name and the actual charge percentage, and directs it straight to the system isolated stream `SYSTEM_STATUS_STREAM ( subota:internal:system:status )`.

6. Caching and State Publication Interfaces

Updating data in Redis is encapsulated in the `_publish_power_state()` method and is executed upon any change to the internal state dictionary `self.power_dict` (when parsing host parameters or receiving a new packet from an Android sensor):

1. A readiness check of the asynchronous client is performed (`if not self.redis_client`). If initialization is missing, the operation is logged and execution is aborted.
2. The current dictionary `self.power_dict` is converted into a JSON string using `json.dumps()`.
3. The value is written as a string under the persistent key `REDIS_POWER_STATE_KEY ( "power:state:cache" )`.
4. Concurrently, data duplication is performed into the operational status cache hash table `SYSTEM_STATUS_CACHE ( "subota:system:status:cache" )` in the `"power_status"` field.

Any unexpected I/O errors or network failures during interaction with Redis are intercepted by a `try...except` block, logged with an `ERROR` level, and do not lead to an emergency shutdown of the daemon's main loop.

7. External Power Supply Grid Status Reference

(Integration with DTEK services is currently disabled. Reason: blocked by bot protection)

To map raw text data from external schedule parsers (DTEK), a static global translation dictionary `DTEK_STATUS_MAP` is declared in the code. Developers of adjacent UI and notification modules must use these values to output localized statuses:

| Raw Status Key | System Text Description (Translation) |
|----------------|---|
| "yes" | "Light is on" (Power is supplied in accordance with the schedule) |
| "maybe" | "Possible outage" (Possible outage according to schedule / Gray zone) |
| "no" | "No light" (Power is absent) |
| "first" | "Light will not be on for the first 30 minutes." |
| "second" | "Light will not be on for the second 30 minutes." |
| "mfirst" | "Possible outage first 30 minutes." |
| "msecond" | "Possible outage second 30 minutes." |

4.3.4 External Registration Service Documentation

This document contains a detailed software and technical specification of the `ExternalService` component, which provides centralized management, authentication, and historic data retention from external hardware platforms, gateways (bridges), and a distributed network of smart sensors.

1. Architectural Context and Configuration

The `ExternalService` class is located in the `services/registration_service/` module and operates as a Tier 3 core internal service, inheriting from `SubotaInternalService`.

1.1. Base System Constants

When instantiating and exchanging messages inside the event bus, the component utilizes rigidly fixed environment parameters:

- `service_name`: `'external_registration_service'` (the value of `BASE_NAME`).
- `source`: `'core'`.
- `user_id`: `'system'`.
- `config_name`: `'external_services'`.
- **Consumer Group and Consumer Names:** Formatted with the prefixes `group_external_registration_service` and `consumer_external_registration_service_1` respectively.

1.2. Allowed Event Time Skew (Freshness Check)

- `MAX_EVENT_TIME_SKEW = 10`: The maximum allowable discrepancy in seconds between the generation time of an event at its source and the time it is processed by the service. This shields the system from processing stale or replayed data packets.
-

2. Full Service Code Implementation

```
services.external_registration.ExternalService()
```

Bases: `SubotaInternalService`

3. Event Processing and Freshness Loop

3.1. Freshness Verification Method (`__is_fresh`)

Before processing any incoming request from the event bus, the service performs a timestamp verification check:

1. The current system time of the host is retrieved via `time.time()`.
2. The absolute difference is evaluated: `difference = abs(timestamp - float(ts))`.

3. If `difference < MAX_EVENT_TIME_SKEW`, the method returns `True`, permitting further execution. Otherwise, the event is discarded, and a `WARNING` level entry is written to the log.

3.2. Asynchronous Command Dispatching

The internal `_initialize()` method subscribes the consumer to the Redis event bus. An asynchronous lambda function named `event_handler` handles event routing:

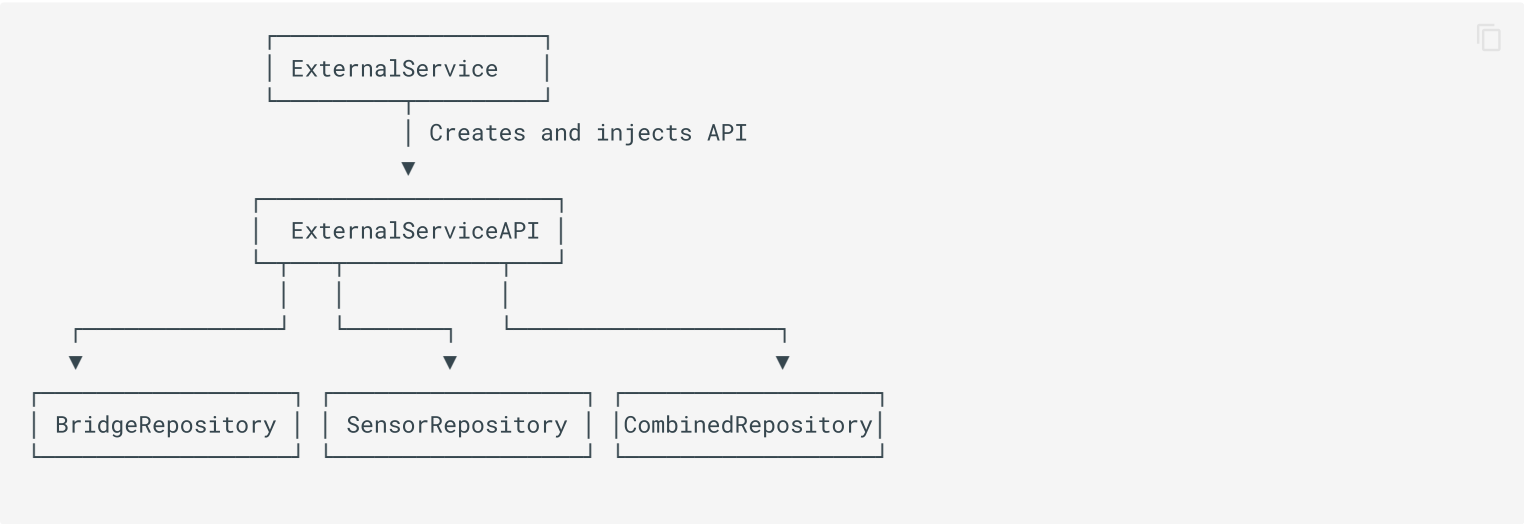
- The name of the incoming event type is converted to lowercase (`event_type.lower()`).
- The event type is validated against the `EVENT_TYPES` whitelist.
- Using a reflection mechanism (`getattr(self, e_type, None)`), an asynchronous class method matching the name is extracted. This method is guarded by the `@SubotaInternalService.check_payload` decorator and is executed with the `payload` passed as an argument.

4. Repository Architecture and Data Access Layer (DAL)

The component entirely decouples event processing logic from direct storage interactions by leveraging the *Data Access Object (DAO / Repository)* pattern.

4.1. Dependency Injection via ExternalServiceAPI

Within the `on_load()` method, an intermediary data class named `ExternalServiceAPI` is constructed. It encapsulates asynchronous interaction protocols with the PostgreSQL DBMS, the task dispatcher (`task_watcher`), and the key management service (`kms`). This object is passed into the constructors of the specialized repositories:



All application repositories inherit from the `BaseRepository` class, which maps API methods into internal protected references (`self.fetch_one`, `self.fetchrow`, `self.fetch_all`, `self.execute`, `self.execute_many`, `self.get_key`).

4.2. Supported Operations and Repository Methods Matrix

Each incoming event maps to a strictly defined method within an application repository:

| Event (event_type) | Target Class Method | Utilized Repository | Implemented Logic |
|----------------------------|----------------------------|---------------------|--|
| register_new_bridge | register_new_bridge | BridgeRepository | Checks if the bridge exists. If missing, creates a record in the database. |
| get_all_bridges | get_all_bridges | BridgeRepository | Returns a list of all registered bridges within the system. |
| delete_bridge | delete_bridge | BridgeRepository | Deletes a bridge matching its unique string name . |
| create_new_state | create_new_state | StatesRepository | Adds a new historical metric (state) for a sensor by its device_uid . |
| get_states_by_device_uid | get_states_by_device_uid | StatesRepository | Queries state history with support for result set limiting. |
| create_new_sensor | create_new_sensor | SensorRepository | Performs data validation, cryptographic token generation, and sensor creation. |
| update_sensor | update_sensor | SensorRepository | Updates metadata, power priority, or the status flag of a sensor. |
| read_sensor | read_sensor | SensorRepository | Retrieves the complete profile card of a registered sensor by its device_uid . |
| delete_sensor | delete_sensor | SensorRepository | Performs cascade deletion of a sensor from the relational database. |
| get_bridges_sensors_states | get_bridges_sensors_states | CombinedRepository | Aggregated query: bridge data + list of sensors + their state histories. |
| get_bridge_sensors | get_bridge_sensors | CombinedRepository | Retrieves the collection of sensors linked to a specific bridge. |
| get_sensor_states | get_sensor_states | CombinedRepository | Queries the latest states of a sensor (fallback selection method). |

5. Cryptographic Protection and Sensor Registration

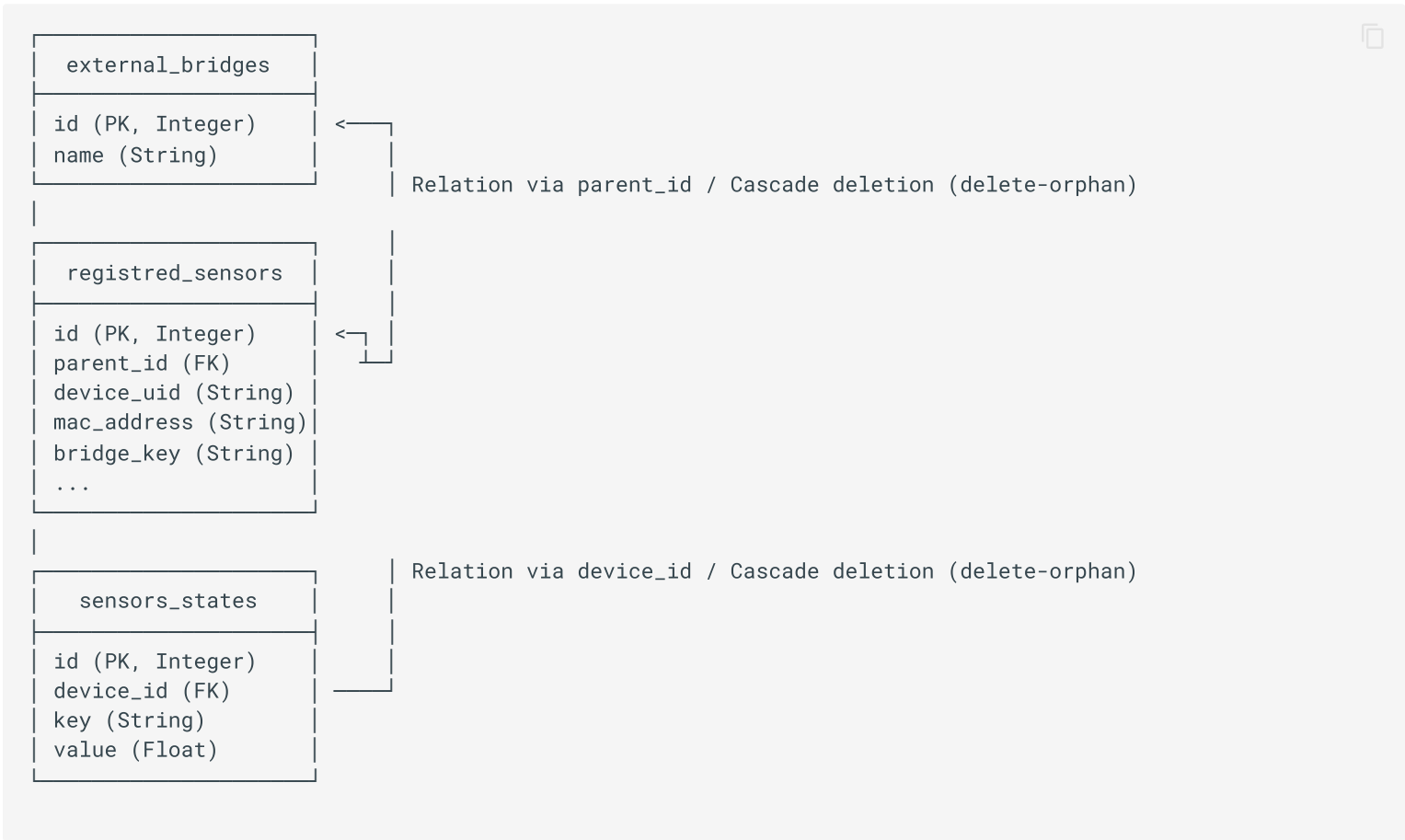
Upon receiving a request to add a new sensor via the `SensorRepository.create_new_sensor` method, the core executes an end-to-end security routine:

- 1. Extraction and Validation:** The parameters `device_uid`, `bridge_name`, `mac_address`, and `bridge_key` are extracted from the provided dictionary. If any are missing, a `ValueError` is raised.

2. **Device Token Generation:** An authorization cryptographic key is computed based on the provided bridge password and the unique sensor identifier using an external core utility: `generate_crypto_key(passphrase=bridge_key, device_uid=device_uid)`.
3. **Parent Relation Resolution:** A raw SQL query is executed against the bridges table to locate the `id` corresponding to the string name `bridge_name`. If the bridge is found, its `id` is assigned to the sensor's `parent_id` field, and the initial status is set to `0` (Active). If the bridge is not found, `parent_id` is populated as `None`, and the status is set to `-1` (Isolated/Binding Error).
4. **Retention:** The data is validated against the `SensorCreate` Pydantic schema and inserted into the database, returning the generated database ID via a `RETURNING id` clause.

6. Relational Data Schema (SQLAlchemy)

The data structure is entirely described using the declarative style of `SQLAlchemy` and maps onto three interconnected tables with *One-to-Many* relationships.



6.1. The `ExternalBridge` Model (Table `external_bridges`)

Describes a logical or physical data aggregation gateway (e.g., `MQTT_Bridge`).

- `id`: Primary key, type `Integer`.
- `name`: Unique name of the bridge, type `String(50)`, mandatory field.
- `sensors`: A `relationship` dependency targeting registered sensors. The parameter `cascade="all, delete-orphan"` is configured, guaranteeing the deletion of all child sensors when a bridge is removed.

6.2. The `RegisteredSensor` Model (Table `registred_sensors`)

Stores configuration parameters of a physical end device.

- **id** : Primary key, type `Integer` .
- **device_uid** : A unique device string token used as the primary business identifier, type `String(50)` , unique.
- **mac_address** : Hardware MAC address of the device, type `String(17)` , unique.
- ****platform / type / location**** : System textual environment metadata, types `String(20)` .
- **bridge_name** : Name of the parent gateway used for quick textual validation checks, type `String(20)` .
- **bridge_key** : The generated cryptographic key of the device, type `String(255)` , unique.
- **status** : Integer value indicating device readiness, type `Integer` .
- **sensor_metadata** : JSON field for arbitrary configuration structures, type `JSON` , allows `Null` values.
- **power_priority** : Shutdown priority value of the sensor during core power degradation scenarios, type `Integer` , defaults to `0` .
- ****last_seen / created_at**** : Timestamps tracking activity metrics and record creation (generated database-side via `func.now()`).

6.3. The `SensorState` Model (Table `sensors_states`)

A time-series metric history retention table for physical sensors.

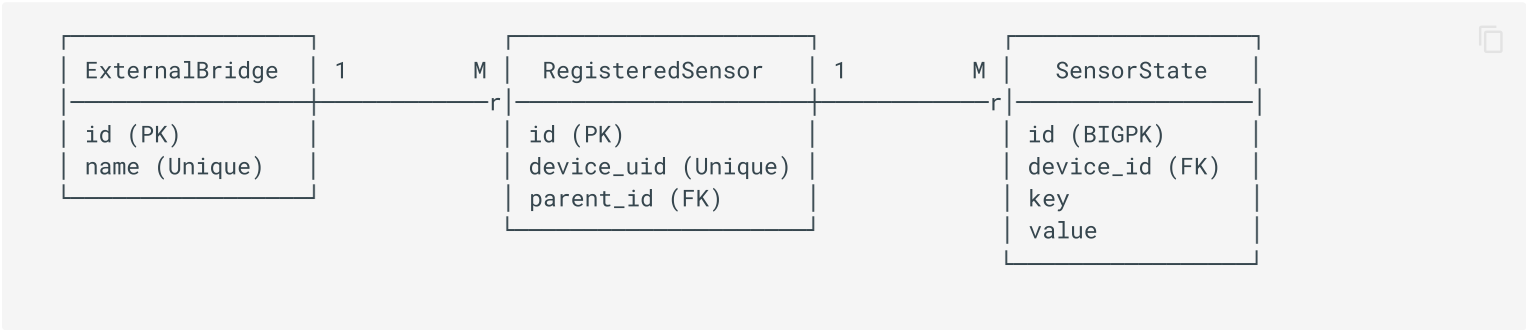
- **id** : Primary key, type `Integer` .
- **device_id** : Foreign key referencing `registred_sensors.id` , type `Integer` , configures cascade deletion.
- **key** : Textual metric name (e.g., `temperature` , `humidity`), type `String(20)` .
- **value** : Numeric measurement value, type `Float` .
- **created_at** : Timestamp indicating when the measurement was received and committed to the database.

4.3.5 Registration Subsystem Data Schema (Registration Service Schema)

1. Logical Architecture and Relationships (ER Model)

The registration subsystem provides accounting for physical equipment and metrics originating from external automation loops. The data is organized within a three-tier hierarchical structure:

- ExternalBridge (Integration Bridge):** The root element that aggregates a group of sensors operating via a specific network gateway.
- RegisteredSensor (Registered Sensor):** A device linked to a specific hardware platform and physical location. It is directly subordinate to one of the bridges (`parent_id`).
- SensorState (Historical State of Metrics):** Time series of indicators collected from a sensor. Each record is strictly tied to a specific sensor (`device_id`).



2. Physical Level Specification (Database)

The physical deployment of tables is implemented based on the PostgreSQL DBMS using Alembic migrations.

2.1. Bridges Table (`external_bridges`)

Stores a list of authorized integration platforms.

- Cascading:** When a bridge record is deleted in SQLAlchemy, all child sensors are automatically deleted via `cascade="all, delete-orphan"` mode.
- DB-Level SQL Constraint:** At the database level (`Alembic`), the foreign key of child tables is configured as `ON DELETE SET NULL` , which ensures the safety of sensor data during manual SQL operations without the involvement of the ORM.

| Column | DB Data Type | ORM Type | Constraints | Description |
|-------------------|--------------------------|-------------------------|---|-------------------------------------|
| <code>id</code> | <code>SERIAL</code> | <code>Integer</code> | <code>PRIMARY KEY</code> | Autoincrementing bridge identifier. |
| <code>name</code> | <code>VARCHAR(50)</code> | <code>String(50)</code> | <code>UNIQUE</code> , <code>NOT NULL</code> | Unique textual name of the bridge. |

2.2. Sensors Table (`registred_sensors`)

The central registry of all end telemetry devices.

| Column | DB Data Type | ORM Type | Constraints | Description |
|-----------------|--------------|-------------|----------------------------------|--|
| id | SERIAL | Integer | PRIMARY KEY | Sensor record identifier. |
| device_uid | VARCHAR(50) | String(50) | UNIQUE , NOT NULL | String UID of the physical device. |
| mac_address | VARCHAR(17) | String(17) | UNIQUE , NOT NULL | Physical MAC address of the device. |
| platform | VARCHAR(20) | String(20) | NOT NULL | Hardware platform (e.g., ESP32, RPi). |
| type | VARCHAR(20) | String(20) | NOT NULL | Sensor type (climate, lighting, etc.). |
| location | VARCHAR(20) | String(20) | NOT NULL | Placement area (physical location). |
| bridge_name | VARCHAR(20) | String(20) | NOT NULL | Parent bridge name (string duplicate). |
| bridge_key | TEXT | String(255) | UNIQUE , NOT NULL | Token/security key for bridge communication. |
| status | INTEGER | Integer | NOT NULL , DEFAULT 0 | Digital code of the current sensor status. |
| sensor_metadata | JSONB | JSON | NULLABLE | Arbitrary sensor metadata in JSON. |
| power_priority | INTEGER | Integer | NOT NULL , DEFAULT 0 | Power supply priority during failures. |
| last_seen | TIMESTAMP | DateTime | NULLABLE | Time when the device was last online. |
| parent_id | INTEGER | Integer | FOREIGN KEY , ON DELETE SET NULL | Reference to external_bridges.id . |
| created_at | TIMESTAMP | DateTime | NOT NULL , DEFAULT NOW() | Time of automatic record registration. |

 **Note on Change History:** In migration the column for storing JSON data was renamed from `metadata` to `sensor_metadata` to eliminate name conflicts with internal ORM attributes.

2.3. States Table (`sensors_states`)

A table for storing time-series of sensor indicators. It uses an increased capacity of the key to optimize for large volumes of records.

- Cascading:** The `ON DELETE CASCADE` rule is configured at the DB level. When a sensor is physically deleted, the entire history of its metrics is cleared automatically. In the ORM, this is duplicated with the declaration `cascade="all, delete-orphan"`.

| Column | DB Data Type | ORM Type | Constraints | Description |
|------------|--------------|--------------|-------------------------|--|
| id | BIGSERIAL | Integer (PK) | PRIMARY KEY | Global unique record index. |
| device_id | INTEGER | Integer | FOREIGN KEY, NOT NULL | Reference to <code>registred_sensors.id</code> . |
| key | VARCHAR(50) | String(20) | NOT NULL | Metric identifier (e.g., "temp", "hum"). |
| value | FLOAT | Float | NOT NULL | Floating-point numeric value of the metric. |
| created_at | TIMESTAMP | DateTime | NOT NULL, DEFAULT NOW() | Exact time when the indicator was recorded. |

3. Logical Level of Validation (Pydantic Schemas)

Pydantic schemas provide strict data type validation when passing through the API boundaries of the S.U.B.O.T.A. system controllers. All read models use the configuration parameter `model_config = ConfigDict(from_attributes=True)` for transparent parsing of SQLAlchemy ORM objects.

3.1. Integration Bridge Models (Bridges)

- `BridgeBase`: The base schema, containing the mandatory field `name: str`.
- `BridgeCreate`: Inherits from `BridgeBase`. Serves as an incoming DTO for creating a new gateway record.
- `BridgeRead`: The output model. Adds a mandatory `id: int`.
- `BridgesAll`: A service model for returning a flat associative array containing a dictionary of registered bridges in the `Dict[int, str]` (`id: name`) format.

3.2. Sensor Models (Sensors)

- `SensorBase`: Contains immutable attributes of the physical device: `device_uid`, `mac_address`, `platform`, `type`, `location`, `bridge_name`, and an optional `parent_id: Optional[int]`.
- `SensorCreate`: Extends the base schema with a secret token `bridge_key: str`, an initial status `status: int`, an optional dictionary `sensor_metadata: Optional[Dict[str, Any]]`, and a power supply priority `power_priority: int` with a default value of `0`.
- `SensorRead`: Used when reading data. Adds the mandatory fields `id: int`, `created_at: datetime`, and an optional last activity time `last_seen: Optional[datetime]` to `SensorCreate`.

- `SensorUpdate` : A dedicated structure for PATCH modification of a sensor. All fields are strictly optional: `status` , `sensor_metadata` , `power_priority` , `parent_id` , `last_seen` . This prevents accidental overwriting of system constants (such as `mac_address` or `device_uid`).

3.3. Telemetry Models (States)

- `StatesBase` : Includes three basic parameters of a metric transaction: device identifier `device_id: int` , string name `key: str` , and value `value: float` .
- `StatesCreate` : A complete analog of `StatesBase` , used in incoming stream write endpoints.
- `StatesRead` : History output model. Extended with `id: int` fields and the system time of saving `created_at: datetime` .

3.4. Composite / Aggregated Models (Combined models)

Designed for complex, optimized API queries that minimize the number of network round-trips:

- `BridgeWithSensors` : Extends `BridgeRead` by nesting a full list of child sensors in the `list[SensorRead]` format inside the response.
- `SensorWithStates` : Extends `SensorRead` by including an array of historical telemetry data `states: list[StatesRead]` .
- `BridgeWithSensorsAndStates` : The deepest hierarchical structure. It inherits from `BridgeWithSensors` and adds a flat or structured result list `all_bridges_sensors_states: list[SensorWithStates]` , allowing a single query to retrieve a slice of a bridge, all its sensors, and their time-series data.

4.3.6 Debug Intent Service (Debug Intent Service)

1. Purpose and Configuration of the Service

The `DebugIntentService` acts as an internal tier-2 core infrastructure component (`tier=2`). It functions as a specialized interceptor (Sniffer/Logger) for event messages on the data bus that are classified as intents (Intents).

The primary objective of the service is the centralized monitoring, deserialization, and secure, structured logging of debug information during the processing of user or system commands, mitigating any risk of leaking sensitive data into log files.

Module System Parameters:

- **Parent Class:** `SubotaInternalService` .
- **Service Name (`BASE_NAME`):** `debug_intent_service` .
- **Utilized Configuration:** `in_development_settings` (the module is tailored for development environments and comprehensive testing loops).
- **Worker Name in TaskWatcher:** `DebugIntentService_LOOP` .
- **Bus Group and Consumer Identifiers:**
 - `GROUP_NAME` : `group_debug_intent_service`
 - `CONSUMER_NAME` : `consumer_debug_intent_service_1`

2. Intercepted Intents (KNOWN_INTENTS)

The service filters the general event stream of the broker bus. The processing dispatcher `stream_event_dispatcher` reacts exclusively to event types (`event_type`) contained within a rigidly defined `KNOWN_INTENTS` array.

All intents recognized by the system are partitioned into the following functional categories:

| Category | List of Intercepted Event Tokens (<code>event_type</code>) |
|---------------------|---|
| Information and Web | <code>WEATHER</code> , <code>SEARCH_WEB</code> , <code>TIME_CHECK</code> , <code>INTERNET_CHECK</code> , <code>INTERNET_DIAGNOSTIC</code> |
| LLM Management | <code>LLM_START</code> , <code>LLM_RESTART</code> , <code>LLM_STOP</code> |
| Host Management | <code>OFF_BOT</code> , <code>OFF_PC</code> , <code>PROTOCOL_WATCH_DOG</code> |
| Organizer and Tasks | <code>ADD_NOTE</code> , <code>ADD_REMINDER</code> , <code>ADD_EVENT</code> , <code>ADD_TASK</code> , <code>SCHEDULE_ALARM</code> , <code>SCHEDULE_REMINDER</code> |
| Lists and Media | <code>PLAY_MUSIC</code> , <code>CREATE_SHOPPING_LIST</code> , <code>ADD_TO_SHOPPING_LIST</code> , <code>CREATE_TO_DO_LIST</code> , <code>REPLAY_TO_DO_LIST</code> |

| Category | List of Intercepted Event Tokens (<code>event_type</code>) |
|----------------------------|---|
| Power Supply and Utilities | <code>BATTERY_MONITORING</code> , <code>DTEK_SCHEDULE_UPDATE</code> , <code>BATTERY_CRITICAL</code> , <code>BATTERY_LOW</code> , <code>POWER_LOSS_DETECTED</code> , <code>POWER_RESTORED</code> , <code>SENSOR_LOW_BATTERY</code> |

3. Secure Logging Algorithm (Sanitization)

When an event type matches any token from the `KNOWN_INTENTS` list, the payload data is passed to the asynchronous `get_pretty_secure_log(data)` method. This method implements a deep recursive sanitization (masking) mechanism for information.

3.1. Masking Criteria (Stop Words)

Vulnerable data discovery is performed by checking dictionary key names (converted to lowercase). Any field whose name contains at least one substring from the `SENSITIVE_KEYWORDS` set is subject to masking:

- `password` , `pass` , `pwd`
- `token` , `access_token`
- `secret` , `key`
- `cookie` , `auth` , `credential`

3.2. Operational Logic of the Recursive `_sanitize(obj)` Parser

The internal converter analyzes the type of the passed object and performs the following transformations:

- Dictionaries (`dict`):** Iterates over key-value pairs. If a key name contains a stop word, its original value is completely overwritten with the constant string `"[REDACTED]"` . Otherwise, the value is passed to the next recursive level of sanitization.
- Iterable Collections (`list` , `tuple` , `set`):** Unpacked, and each internal element is sequentially processed by the `_sanitize` function.
- Time Objects (`datetime`):** Automatically converted into a strictly specified string format: `YYYY-MM-DD HH:MM:SS` .
- Primitives:** Numbers, strings, and `None` values are returned in their original form without modification.

3.3. Log Output

Upon completion of the sanitization process, the data structure is serialized into a JSON string format using `json.dumps` with the following parameters:

- An indentation level of 4 spaces (`indent=4`) is applied to ensure structural readability.
- Escaping of non-ASCII characters is disabled (`ensure_ascii=False`), allowing Cyrillic and special characters to be retained in the logs in their original form.
- The `default=str` parameter acts as a safeguard: if an untyped or custom class is encountered in the structure, the serializer invokes the `str(obj)` method instead of raising a `TypeError` exception.

The final sanitized JSON string is sent to the system logger with a debug level of `logger.debug`.

4.3.7 Remote Config Manager (Remote Config Manager)

1. Purpose and System Parameters

The `RemoteConfigManager` module is an externally oriented core component (`SubotaExternalService`) that ensures the secure distribution of settings and dynamic access parameters among peripheral bridge services (Bridges).

Module System Parameters:

- **Service Name (`BASE_NAME`):** `remote_config_manager` .
- **Utilized Configuration:** `remote_config_manager` .
- **Worker Name in TaskWatcher:** `RemoteConfigManager Loop` .
- **Bus Group and Consumer Identifiers:**
 - `GROUP_NAME` : `group_remote_config_manager`
 - `CONSUMER_NAME` : `consumer_remote_config_manager_1`

2. Event Dispatching Finite State Machine

The primary duty cycle of the service listens to the event group `group_remote_config_manager` . The processing logic for incoming packets is built on pattern matching (`match state`), which maps the event type against the status of the system readiness flags (`bridge_MQTT_Bridge_initialized` and `bridge_MQTT_Bridge_online`).

In all legitimate transitions, a time skew (Skew) is verified: the difference between the current Unix timestamp of the core and the packet timestamp `ts` must strictly be less than 10 seconds:

$$|\Delta t| = |t_{\text{core}} - t_{\text{event}}| < 10 \text{ s}$$

Handler State Matrix (`event_handler`)

| Matching Expression
(<code>event_type</code> ,
<code>initialized</code> , <code>online</code>) | Expected Context | Executed Action / Core Reaction |
|--|--|---|
| (<code>'spirit_response_True'</code> , <code>False</code> , <code>False</code>) | Successful primary authorization of the bridge; configuration has not been issued yet. | Validation of Δt \$. Passing control to the <code>send_config(payload)</code> method. |
| (<code>'spirit_response_True'</code> , <code>_</code> , <code>_</code>) | The bridge is already initialized or online; a duplicate request has been received. |  SECURITY ALERT: Interception/replay attempt. The request is rejected, and a log is written to the forensic analysis circuit <code>sec_forensics</code> . |
| (<code>'MQTT_Bridge_SHUTDOWN'</code> , <code>True</code> , <code>True</code>) | Normal or registered shutdown of an external gateway. | Validation of Δt \$. Resetting (clearing) system initialization and presence events of the bridge in <code>infra</code> . |

| Matching Expression | Expected Context | Executed Action / Core Reaction |
|--|--|--|
| <code>(event_type , initialized , online)</code> | | |
| <code>('MQTT_Bridge_SHUTDOWN' , _ , _)</code> | The shutdown signal was received in an incorrect phase of the lifecycle. |  SECURITY ALERT: The request is rejected, logging the anomaly into the <code>sec_forensics</code> circuit. |

3. Cryptographic Circuit and Data Protection

3.1. Stream XOR Cipher (`__xor_cipher`)

To obscure transmitted secrets while in transit through external or intermediary data streams, the module utilizes a symmetric keystream algorithm based on the XOR (`^`) operation.

- Based on the passed dynamic key string (`key_string`), a SHA-256 cryptographic fingerprint (32 bytes) is computed:
$$\text{KeyHash} = \text{SHA256}(\text{key_string})$$
 - The original byte array (`data`) is combined element-by-element modulo 2 with the bytes of the resulting hash. If the length of the original data exceeds 32 bytes, the hash byte sequence is cyclically repeated using `itertools.cycle` :

$$C_i = P_i \oplus \text{KeyHash}_{i \bmod 32}$$

Where C_i is the encrypted byte, and P_i is the original data byte.

The operation is strictly reversible (re-applying the function to the encrypted array with the same key returns the original text).

3.2. Configuration Routing (`send_config`)

The method is protected by the data structure validation decorator `@SubotaExternalService.check_payload` . Based on the `key_id` attribute of the incoming message, a decision is made regarding the target platform type:

- `mqtt_bridge` : The configuration object is extracted from `self.config.mqtt_bridge` . Access rights are requested from the general permissions subsystem via `permissions.get_permissions_for('mqtt_bridge')` . The data is directed to the private packet assembly method `__send_mqtt_config` .
- `bot_bridge` : Currently in the design phase (raises a `NotImplementedError` exception).
- Any other value**: An access attempt to a non-existent gateway is recorded. A security incident report (`report_incident`) with the violation type `NOT_EXISTING` is dispatched to the Security Center.

4. Configuration Assembly and Transmission Algorithm (`__send_mqtt_config`)

Provided that the `bridge_mqtt_initialized` flag has not yet been set, the manager executes the secure assembly procedure for the transport container:

- Extraction of Salts and Parameters**: The client session dynamic salt (`nonce`) and the system string name of the gateway (`service_name`) are extracted from the payload `payload.data['content']` .

2. **Transit Key Generation:** The baseline master key `kms_key_service` is requested by service name from the core key management subsystem (`self.kms`). The service adds the gateway name to the list of registered devices `registered_services` . The master key is encrypted using the `__xor_cipher` method with the client `nonce` as the encryption key and is converted into a hexadecimal string (`hex`).
3. **Dynamic Authorization in Redis:** Temporary/dynamic credentials (`user` and `password`) are requested through the `redis_auth_manager.get_user_for_bus` interface based on permissions (`perm`) to isolate bridge access rights within Redis Streams. The obtained credentials are injected on the fly into the resulting structure `config.redis` .
4. **Serialization:** The configuration object is exported into a dictionary (`model_dump`) and serialized into a JSON string with forced lexicographical sorting of keys (`sort_keys=True`) to guarantee structural determinism.
5. **Publishing to the External Bus:** The outgoing content is formed:

```
{
  "kms_key": "<hex_encrypted_kms_key>",
  "config": "<sorted_json_config_string>"
}
```

The container is published via the `publish_to_external_event_bus` method with a response type of `MASHIN_SPIRIT_FOR_{target}` into a dedicated dynamic registration stream: `subota:register:response:{nonce}` .

1. **State Retention:** Upon completion of the transmission, the event `self.__bridge_mqtt_initialized.set()` is raised, completely blocking re-execution of this algorithm until a reset occurs via the `SHUTDOWN` signal. Any subsequent attempt to enter the method without resetting the flag generates an information security alert in the logger `sec_forensics.warning` .

4.3.8 Metrics Archiving and Rotation Subsystem (Archive Service)

1. Orchestration Architecture (Parent Service)

The `ArchiveService` class acts as the central scheduler and coordinator of data rotation processes in the S.U.B.O.T.A system. It encapsulates the isolation logic of child archiving submodules using `ChildAPI` interfaces and dynamic environment assembly.

1.1. Dynamic Assembly Mechanism (`on_load`)

Upon initialization, the service automatically inspects its own interface container `self.interfaces` using the `dir()` function. All public callable methods (excluding the utility attributes `status` and `config`) are converted into an isolated `module_interfaces` map, which is passed to child submodules via `ChildAPI`. This eliminates high coupling between the executable workers and the core.

1.2. Staggered Time Cascade (Staggered Startup)

To prevent peak loads on the disk subsystem and CPU when processing large arrays of data, `ArchiveService` implements a staggered startup phase shift algorithm. For each module declared in the `archive_config.moduels` configuration, the base start time is shifted forward by 15 minutes relative to the previous worker:

$$T_{\text{start}}(n) = T_{\text{base}} + (n \times 900 \text{ seconds})$$

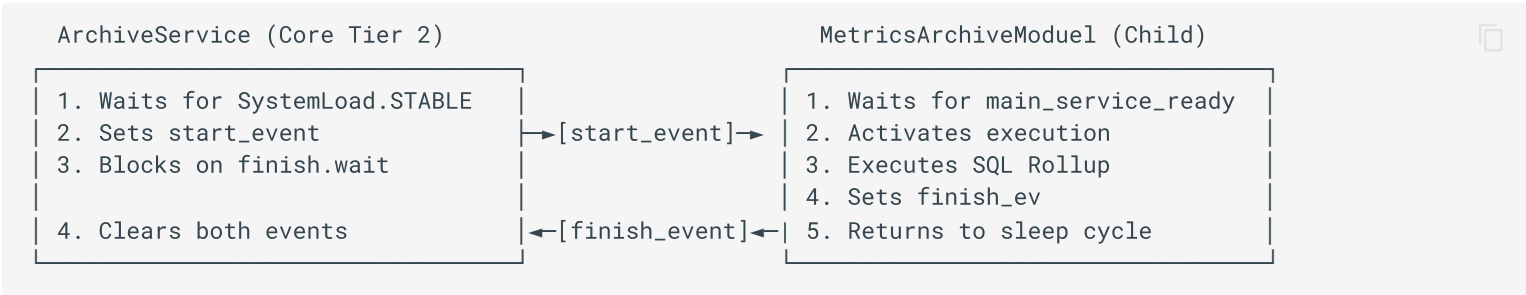
1.3. Stability Control Gate (Load Protection Gate)

Before transferring control to any child archiving module, the parent service forcefully activates the asynchronous hardware load check gate `__wait_for_low_load()`. The method polls the system state manager `system_state_manager` at an interval of 60 seconds. The launch of the archiving procedure is blocked until the current system state returns to the reference stability value:

$$\text{SystemLoad} \equiv \text{SystemLoadState.STABLE}$$

2. Worker Lifecycle Synchronization

Interaction between the parent orchestrator `ArchiveService` and executable modules (for example, `MetricsArchiveModuel`) is built on the basis of a distributed signal bus `asyncio.Event` and the `child_events` structure.



Sequential Execution Algorithm (`__run_archivation`):

1. The orchestrator takes the module's configuration event map, checks the host stability flag, and sets `start_event` to `True`.

2. The executable child module, running within its debug cycle `_nightly_archive`, exits the `_start_event.wait()` wait state and proceeds to data aggregation.

3. The orchestrator enters an asynchronous block on the completion event: `await events['finish_event'].wait()`.

4. The child module completes the execution of all SQL transactions and raises the flag `_finish_event.set()`.

5. The parent service records the execution time delta (`finish_time`), resets both events to their initial state (`clear()`), maintains a technical pause of 30 seconds for the disk subsystem to cool down, and moves to the next module in the queue.
- Upon completion of the full archiving cycle of all components, an aggregated transport container `SubotaMessage` is formed. The package is placed in the system emergency queue `emergency_queue` with the event type `ARCHIVE_DONE` to send notifications to administrators via the `telegram_sender` and `cli_sender` channels.

3. Time Series Archiving (Metrics Rollup Pipeline)

The child module `MetricsArchiveModule` performs tasks for the sequential compression of metrics and the purging of operational database tables.

3.1. Ensuring Atomicity and Idempotency (`_try_mark_done`)

To prevent repeated archiving of data from the same period during failures or core restarts, the module uses the transaction log `archive_metrics_log`. Verification and commit of the step are performed within a single atomic SQL query with a conflict isolation construct:

```
INSERT INTO archive_metrics_log (archive_type, period_date)
VALUES ($1, $2)
ON CONFLICT (archive_type, period_date) DO NOTHING
RETURNING id;
```

If the record already existed, the method returns `False`, and the module skips processing this step, guaranteeing the idempotency of the pipeline.

3.2. Multi-level Table Compression Cascade (Data Retention)

The module sequentially moves data through the table hierarchy with an increasing aggregation period followed by the removal of source records with a time offset (Offset):

| Level (Tier) | Source | Target | Frequency / Condition | Purge Rule |
|--------------|-----------------------------|------------------------------|---|--|
| Daily | <code>system_metrics</code> | <code>metrics_daily</code> | Daily (for dates older than 3 days) | <code>DELETE from system_metrics for date \$T-3\$ days.</code> |
| Weekly | <code>metrics_daily</code> | <code>metrics_weekly</code> | Every Monday (<code>weekday == 0</code>) | <code>DELETE from metrics_daily with a 7-day offset.</code> |
| Monthly | <code>metrics_weekly</code> | <code>metrics_monthly</code> | 1st day of each month (<code>day == 1</code>) | <code>DELETE from metrics_weekly with a 30-day offset.</code> |

| Level (Tier) | Source | Target | Frequency / Condition | Purge Rule |
|--------------|---------------------|--------------------|--|---|
| Yearly | metrics_mont
hly | metrics_year
ly | January 1st of each year
(month == 1 && day == 1) | DELETE from
metrics_monthly with a 100-
day offset. |

4. Aggregation Logic by Data Sources (process_daily)

During the execution of the process_daily procedure, raw records are grouped by the source field. An individual mathematical model of metric compression is applied to each source:

4.1. HealthChecker Circuit

Calculates the average latency (Latency) and aggregates the availability status of components. The uptime percentage calculation is performed based on the occurrence of the string literal 'online' :

$$\text{Uptime \%} = \frac{\text{up_ticks}}{\frac{\text{total_samples}}{2}} \times 100$$

The final value is written into a text field in the format online (XX.X%) .

4.2. TaskWatcher Circuit

Analyzes the operational statuses of the system modules' asynchronous threads.

- If the string status equals 'Running' , the up_ticks counter is incremented.
- If any states other than Running are detected, they are made unique via set and appended to the resulting string field in the section | Other states: <statuses> .
- The numerical evaluation field avg_value takes strictly discrete values: 1.0 given 100% module uptime for the day, or 0.0 if there is at least one failure/downtime.

4.3. MetricsCollector Circuit

Performs standard compression of numerical series (preserving MIN , MAX , and AVG for each metric type). It has a built-in special rule for processing power supply and battery parameters (searching for the substring battery in the metric name):

- All numerical battery performance indicators are combined into a single integral daily record battery_summary .
- If the metric name contains the substring plugged , the value is interpreted as the AC power connection status. Based on these data, the final external power stability indicator is calculated: $\text{Power Online \%} = \frac{\sum \text{true_values}}{\text{total_plugged_samples}} \times 100$ This percentage is saved to the database as a formatted string Power Online: XX.X% .

4.4 Inputs/Senders

4.4.1 Telegram I/O Subsystem Reference

This document describes the software and technical implementation of the input/output interface based on the Telegram platform. In the Subota system, interaction with Telegram is separated into three isolated components to ensure fault tolerance, independent lifecycle management, and scalability.

1. Subsystem Architectural Overview

The core system's Telegram interface is implemented through the interaction of three entities across different hierarchy levels (Tiers):

1. **TelegramBot (Tier 1 Core Component)**: A low-level session holder. It is exclusively responsible for securely retrieving tokens, initializing `Bot` and `Dispatcher` objects from the `aiogram` library, and closing network connections.
 2. **TelegramInput (Tier 3 Internal Service)**: An asynchronous incoming data stream processing service. It implements handler registration, user access control verification (RBAC), and translates raw messenger messages into internal system event bus events (Event Bus).
 3. **TelegramSender (Tier 1 Sender Component)**: A component for sending outgoing notifications. It is loaded dynamically under the control of the `NotificationManager` and converts system messages into formatted Telegram text.
-

2. Autogeneration Specification (mkdocstrings)

```
bots.telegram_bot.TelegramBot()
```

Bases: `BaseClass`

```
inputs.telegram_input.TelegramInput()
```

Bases: `SubotaInternalService`

```
senders.telegram_sender.TelegramSender()
```

Bases: `BaseClass`

3. Session Layer: TelegramBot

The component is initialized at an early stage of the core startup (`Tier = 1`) and requests authorization methods from the `secret_manager` service.

3.1. Secret Extraction and Authorization

The following steps are executed within the `on_load()` method:

1. String tokens `TELEGRAM_BOT_TOKEN` and `TELEGRAM_CHAT_ID` are requested via the `secret_manager.get_secret` interface.

2. The administrator's chat identifier `TELEGRAM_CHAT_ID` is validated and cast to the `int` type to form the allowed chats list `_allowed_chats`. If the value is not a number, a critical `ValueError` exception is raised, and initialization is aborted.
3. The `Bot` object is initialized by passing a `DefaultBotProperties(parse_mode='HTML')` instance to support default message formatting.

3.2. Shutdown

When the module is unloaded (`on_unload`), the dispatcher is reset (`None`), and the bot session is forcibly closed by an asynchronous call to `await self._bot.session.close()`, preventing connection descriptor leaks.

4. Inbound Gateway: `TelegramInput`

The `TelegramInput` component inherits from `SubotaInternalService` and deploys update polling inside an isolated task under the supervision of the `task_watcher` process manager.

4.1. Dispatcher Registration and Command Routing

During the initialization process (`_initialize()`), the service extracts `Bot` and `Dispatcher` instances from the `telegram_bot` interface and registers a local `Router`:

| Command / Message Type | Invoked Method (Handler) | Required Permissions (RBAC) | Outbound Bus Event (<code>answer_type</code>) |
|-------------------------------|--|------------------------------|--|
| <code>Command("start")</code> | <code>_handle_start_comm</code>
and | Not required | None (direct reply to chat). |
| <code>Command("exit")</code> | <code>_handle_exit_comma</code>
nd | <code>system:shutdown</code> | <code>EXIT_SEQUENCE</code> (Core shutdown sequence). |
| Any text input | <code>_handle_text_inpu</code>
t | Not required | <code>user_command_input</code> (Passing the command to the core). |

4.2. Access Control and RBAC Exception Handling

Protection of critical commands (e.g., `/exit`) is implemented via the `require_permission_non_api` decorator, which accepts an `rbac_manager` instance and a string permission name.

When an unauthorized user attempts to execute a command:

- The component intercepts the `PermissionDenied` exception using the registered dispatcher error handler `_dp.errors.register(..., F.exception_type == PermissionDenied)`.
- A standardized response is sent to the user:  У вас нет разрешения на выполнение этой команды.
- The unauthorized access attempt is logged with a `WARNING` level, recording the user ID and command text.

4.3. Event Payload Structure

When translating text messages into the event bus, a unified `payload` dictionary is constructed:


```
{
  "text": "User message text",
  "session_id": "tg_<chat_id>",
  "source": "telegram",
  "source_user_id": "<user_id>",
  "timestamp": 1719920000.0,
  "data": {}
}
```



5. Outbound Gateway: TelegramSender

This component is intended exclusively for sending messages to allowed chats within `_chat_id`. It inherits from the `BaseClass` base class.

5.1. Message Urgency Levels

The `send_message` method accepts a string parameter `urgency`, based on which a graphical prefix is automatically prepended to the final message text:

- `warning` `$\rightarrow$` ⚠
- `error` `$\rightarrow$` ✖
- `info` `$\rightarrow$` ⓘ
- Other/unknown values `$\rightarrow$` prefix is absent.

Transmission is executed iteratively in a loop across all identifiers in the `_chat_id` list, with exception handling for each independent `bot.send_message` call.

6. Integration with NotificationManager and Night Mode

The `TelegramSender` outbound gateway is not invoked directly by application services; instead, it is managed by the centralized `NotificationManager` service (Tier 4).

6.1. Dynamic Notification Routing

`NotificationManager` listens to the `notifications_group` event group in Redis. Upon receiving an event of type `publish_notification`, it parses the `destinations` array. If the `telegram` target channel is specified in the array, the manager extracts the `telegram_sender` object from its local registry of child components and initiates an asynchronous transmission task via `asyncio.create_task`.

6.2. Handling Critical Events (`CRITICAL_NOTYFI`)

When emergency signals arrive from the core's local `emergency_queue`, the `_send_critical_notifi` method enriches the message text with a timestamp in the 🕒 `HH:MM:SS` format, transmits it via Telegram, and forcibly sleeps for the `critical_timeout` interval specified in the `NotificationManagerConfig` configuration, thereby preventing flooding.

6.3. "Night Mode" Buffering for `ARCHIVE_DONE`

For system notifications regarding archiving completion (`ARCHIVE_DONE`), a silence compliance algorithm is implemented for the host's nighttime:

1. The current time is calculated taking into account the local timezone from the `location_timezone.timezone` configuration via the `pytz` library.
2. Hard boundaries for night mode are established: **from 23:00 to 07:00**.
3. If the event occurs during the night, `NotificationManager` computes the exact delay in seconds until 07:00 of the following day using the auxiliary `__get_seconds_wake_up_time()` method.
4. The notification is deferred using the system event loop:

```
loop = asyncio.get_running_loop()
loop.call_later(time_to_wait, lambda: asyncio.create_task(...))
```



The message will be guaranteed delivery to Telegram immediately after the nighttime period ends (at 07:00).

4.4.2 Reference: Web UI & API Gateway

1. Purpose and Area of Responsibility

The **Web UI & API Gateway** component (module `web_controller`) acts as a single point of entry (API Gateway) for external requests to the Subota system. It is responsible for:

- Deploying and softly stopping an asynchronous HTTP/WebSocket server based on FastAPI and Uvicorn.
- Ensuring the operation of the user web interface (Web UI) by rendering Jinja2 HTML templates and serving static files.
- Verifying user authenticity (Authentication) and enforcing role-based access control (RBAC).
- Routing incoming commands into the internal core event bus.
- Maintaining two-way asynchronous data exchange with clients via WebSockets (chat, notification, and streaming audio channels).

2. Architecture and Network Endpoints

The server is initialized in a non-blocking mode under the control of the system process manager `TaskWatcher`. The component distributes incoming requests across five specialized router groups:

2.1. Authentication (`auth_router` with prefix `/api/v1`)

- **POST `/api/v1/auth/login`** — User authentication via `OAuth2PasswordRequestForm`. It sets an encrypted `HttpOnly` Cookie session `access_token` for 7 days and redirects to `/dashboard`. It is protected by a request rate limiter (`5 requests per minute`). Failures and successful attempts are logged into a specialized logger `security_forensics`.
- **GET `/api/v1/auth/test-protected`** — A test endpoint to verify the validity of the current session.

2.2. Command Management (`command_router` with prefix `/api/v1`)

- **POST `/api/v1/commands/execute`** — Receiving user text commands and forwarding them to the internal system core service. It requires mandatory authorization. Incoming data is validated via the Pydantic model `CommandPayload`.

2.3. System Status and Metrics (`status_routes.router` with prefix `/api/v1/status`)

- **GET `/api/v1/status/metrics`** — Returns a JSON dictionary of the latest collected system metrics from PostgreSQL (with priority given to numeric values in `value_float`).
- **GET `/api/v1/status/metrics/history`** — Returns a JSON array of historical metric records with a depth limit (parameter `limit`, defaulting to 30).
- **GET `/api/v1/status/log_history`** — Renders an HTML page containing a table of system logs.
- **GET `/api/v1/status/api/v1/logs`** — A paginated JSON list of system logs with support for filtering by severity level (`level`) and text search (`search`).

2.4. User Interface UI (`web_ui_router` without prefix)

All routes in this group (except `/login` and `/logout`) are protected by authorization checks and are bound to Jinja2 templates:

- `GET /` — Automatic redirect (HTTP 302) to `/dashboard`.
- `GET /login` — The system authorization page.
- `GET /dashboard` — The main control panel (passes the context of `user_id` and `username`).
- `GET /status` — A monitoring page for active core tasks, retrieved directly from `TaskWatcher`, and logging intervals.
- `GET /network` — Network diagnostics (displays the internal IP, external IP, ping status to DNS `8.8.8.8`, and Google API availability).
- `GET /scheduler` — A list of active scheduled tasks from the scheduler.
- `GET /logout` — Complete clearance of the `access_token` Cookie and redirection of the user to the login page.

2.5. WebSockets Protocols (`websocket_router` without prefix)

- `WS /ws/chat/{session_id}` — The assistant text message exchange channel. It maintains the connection in an infinite wait loop.
- `WS /ws/notifications/{session_id}` — A dedicated channel for broadcasting system notifications.
- `WS /ws/voice/stream` — Streaming input of uncompressed audio data (bytes). It streams bytes into the `VoiceController` and returns the recognized text to the client.

3. Data Contracts (JSON Schemas)

When transmitting commands through the API Gateway, the `CommandPayload` model is used:

```
{
  "text": "string",
  "session_id": "string",
  "source": "web_api",
  "source_user_id": "string" || null,
  "timestamp": 1717171717.0 || null,
  "data": {}
}
```

When passing through the `WebUiInput` layer, the structure is repackaged into an internal Payload for the system event bus (`Event Bus`) with the following contract:

- `answer_type`: `"user_command_input"`
- `payload`: Includes the text content, `session_id`, a hardcoded `source: "webui_input"`, as well as a nested `data.content` object containing session identifiers and request metadata.

4. Security and Access Differentiation (Auth & RBAC)

The API Gateway implements a hybrid scheme for token extraction via the `get_current_user` dependency:

1. **API Layer:** The HTTP header `Authorization: Bearer <token>` is verified.

2. **UI Layer:** If the header is absent, the token is extracted from the `access_token` Cookie file.

In the event that the token is missing or invalid, the global FastAPI exception handler differentiates the request type:

- If the path begins with `/api/v1/`, a JSON response with a `401 Unauthorized` code is returned to the client.
- If the request originated from the web interface, a transparent redirect (HTTP 302) to the `/login` page occurs.

After decoding the JWT token, the subsystem queries the role-based access control manager (`RBACManager`). The `user_id` identifier is converted into a numeric format to determine the user's role in the database. Restricting specific router methods is implemented via the `require_permission` dependency-decorator, which checks for the presence of the required system permission (`permission_name`) in the request headers.

5. Internal Components and Data Flows

Interaction between the gateway, internal core structures, and external interfaces is facilitated by a group of interconnected software modules:

1. `web/api_gateway/web_socket_manager.py` — Contains the `WebSocketConnectionManager` class, which performs low-level tracking of open sockets. It isolates connections into two independent types (`chat` and `notifications`) within unique string keys of `session_id`. The module prevents `RuntimeError` locks during mass update broadcasts by pre-copying a slice of active session keys.
2. `senders/web_ui_sender_3.py` — A high-level module for sending responses. It interacts with the `WebSocketConnectionManager` interfaces. It translates targeted assistant messages of type `ASSISTANT_RESPONSE` simultaneously into both channels (`chat` and `notifications`) of a specific user, or broadcasts critical system warnings (prefixed with  [Система/Внимание],  [Система/Ошибка]) exclusively into the notifications channel (`notifications`).
3. `inputs/web_ui_input_3.py` — Implements the `WebUiInput` class, which is inherited from the trusted base class of internal core services `SubotaInternalService`. It receives data from the gateway controller, signs it with fixed system markers `SOURCE = 'web_controller'` and `USER_ID = 'web_ui'`, and then safely publishes the command to the system data bus.
4. `core/notificatio_manager.py` — The architectural companion of the gateway that handles the aggregation of external notifications. It documents the existing technical debt regarding the end-to-end routing of `EMERGENCY_STOP` packets from the external queue circuit into the internal trusted core of the system, bypassing the API gateway isolation, which is scheduled to be moved into an isolated supervisor component in future sprints.

4.4.3 Web API Reference and RBAC Structure (Web API Reference)

1. RBAC Database Schema (Role Model)

Access control to gateway endpoints and core resources is based on a relational schema of access control tables (Role-Based Access Control, RBAC).

1.1. Roles Table (`roles`)

Defines global privilege levels within the system.

- `id` (SERIAL, Primary Key) — unique identifier of the role.
- `name` (TEXT, UNIQUE, NOT NULL) — string name of the role.
- `description` (TEXT) — description of the role's area of responsibility.

The system is initialized with three baseline roles:

1. `admin` (ID=1) — Full system access.
2. `user` (ID=2) — Standard user, device control (default value).
3. `guest` (ID=3) — Status view only.

1.2. Permissions Table (`permissions`)

Contains atomic system rights required to execute operations.

- `id` (SERIAL, Primary Key).
- `name` (TEXT, UNIQUE, NOT NULL) — permission token.
- `description` (TEXT) — description of the action.

Predefined permission matrix:

- `script:edit` — Editing and creating user scripts.
- `device:control` — Turning on/off/managing devices.
- `system:shutdown` — Complete shutdown or reboot of the core.
- `user:admin_management` — Creating, modifying, and deleting users/roles.

1.3. Access Matrix Junction Table (`role_permissions`)

Provides a many-to-many relationship between roles and atomic rights.

- `role_id` (INTEGER, FK -> `roles(id)`).
- `permission_id` (INTEGER, FK -> `permissions(id)`).
- Composite Primary Key: (`role_id`, `permission_id`).

Relationship Configuration: The `admin` role is granted all four permissions (`script:edit` , `device:control` , `system:shutdown` , `user:admin_management`). The `user` role is granted exclusively the `device:control` permission.

1.4. User Accounts Table (`users`)

Stores data for authorization subjects.

- `id` (SERIAL, Primary Key).
- `username` (TEXT, UNIQUE, NOT NULL) — unique username (login).
- `hashed_password` (TEXT, NOT NULL) — cryptographic hash of the password.
- `role_id` (INTEGER, NOT NULL, FK -> `roles(id)` , DEFAULT 2).
- `created_at` (TIMESTAMP WITH TIME ZONE, DEFAULT CURRENT_TIMESTAMP).
- `telegram_id` (BigInteger, UNIQUE, NULL) — linked Telegram account identifier for end-to-end integration with the bot.

2. Monitoring and System State Endpoints (Status Routes)

GET `/metrics`

Returns a snapshot of the latest recorded system metrics from all active core collectors.

- **Input parameters:** None.
- **Dependencies:** `PostgresManagerProtocol` .
- **Transformation Logic:** A flat list of the latest records is retrieved from the database. If the `metric_value_float` field is populated, it is adopted as the primary value; if it equals `None` , the string representation `metric_value_string` is used.
- **Response Format (JSON, 200 OK):**

```
{
  "core_cpu_utilization": {
    "value": 14.5,
    "timestamp": "2026-07-04T15:30:00Z"
  },
  "mqtt_bridge_status": {
    "value": "online",
    "timestamp": "2026-07-04T15:30:02Z"
  }
}
```

- **Errors:** `500 Internal Server Error` (Storage access failure with the message `Failed to retrieve metrics: <description>`).

GET `/metrics/history`

Provides a retrospective cross-section of system parameter variations to build charts and analyze trends.

- **Input parameters (Query):**
- `limit` (int, default `30`) — the number of historical data points to return.
- **Response Format (JSON, 200 OK):**

```
[
  {
    "metric_name": "core_cpu_utilization",
    "value_float": 14.5,
    "value_string": null,
    "timestamp": "2026-07-04T15:30:00Z"
  }
]
```

- **Errors:** 500 Internal Server Error .

GET /log_history

Renders a graphical dashboard with a table containing the latest logging events.

- **Response Type:** HTML (TemplateResponse) based on the log_history.html template.
- **Dependencies and Security:** Requires mandatory session validation via get_current_user .
- **Passed Template Context:** request , user_id (the username field is extracted; if absent, "Гость" is displayed), logs (a fixed slice of the last 20 records via SystemLogProcessorProtocol).

GET /api/v1/logs

Provides programmatic access to system logs with filtering mechanisms and pagination.

- **Security:** Protected by the get_current_user dependency.
- **Input parameters (Query):**
 - limit (int, default 20 , strictly from 1 to 1000) — page size.
 - offset (int, default 0 , \$ \ge 0\$) — pagination offset.
 - level (string, Optional) — filtering by strict record severity level (INFO , WARNING , ERROR , CRITICAL).
 - search (string, Optional) — full-text substring search matching against the log message body or the name of the source component (source).
- **Response Format:** JSON array of log objects.
- **Errors:** 500 Internal Server Error .

3. Administrative Endpoints (User Management)

POST /

Registers a new user account within the system.

 **Important Security Note:** According to the source code documentation, executing this method conceptually requires the calling administrator to possess user:admin_management permissions. Developers must ensure that this restriction is correctly verified at the upper router or middleware layer, as the internal method signature accepts user_data directly without explicitly declaring permission verification dependency injection within the function body.

- **Input Container (JSON, Payload):**
 - username (string, mandatory) — login name for the new user.

- `password` (string, mandatory) – raw text password (will be automatically hashed via `auth_manager.get_password_hash` prior to database storage).
- `role_id` (int, optional, default `2` - the `user` role).
- **Business Logic Validation:**
- If `username` or `password` is missing, a `400 Bad Request` error is returned with the detail `Username and password are required.`
- Database unique key duplication conflicts are intercepted. If a user with that name already exists, a `409 Conflict` error is generated with the message `Пользователь '{username}' уже существует.`
- **Response Format (JSON, 201 Created):**

```
{
  "id": 14,
  "username": "karlson",
  "role_id": 2
}
```

- **Errors:** `400 Bad Request`, `409 Conflict`, `500 Internal Server Error` (in case of transaction failure).

4.4.4 Component Under Development

Omnissiah System Note

This section of the technical specification describes a component that is currently fixed in the architectural design plans, but does not yet have a physical implementation in the S.U.B.O.T.A. v3.0 core codebase.

1.2.1 Component Status

- **Current state:** Interface design / Writing stub modules.
- **Development branch:** `dev-v3-architecture`
- **Related release plan:** [WORKPLAN_3_RELEASE](#)

1.2.2 Architectural Boundaries

Once development of the module is complete, this page will expand into full technical documentation, including:

1. **API Contract:** A description of method input/output signatures, data types, and validation schemas (Pydantic models).
 2. **Component Lifecycle:** The initialization logic of the component within the core's overall boot cycle (Boot Manifest).
 3. **Security Context:** Access rights within the RBAC model and the `RestrictedProxy` restriction matrix.
 4. **Event Topology:** A list of events generated and handled on the `Redis Event Bus`.
-

1.2.3 What Should a Developer Do?

If you critically need the functionality of this component to write user scripts or related system modules:

1. **Use Mock objects:** During development, call stub interfaces that return deterministic test data.
2. **Check the task plan:** Current timelines and dependencies for the implementation of this module can be found in the document  Status -> Development Plans ([WORKPLAN_3_RELEASE](#)).
3. **Do not create hidden dependencies:** It is forbidden to build core logic that bypasses isolation by relying on this component's future structure. All interactions must strictly follow the Object Capabilities (OCaps) pattern.

4.4.5 Reference: CLI (Input/Output Interface)

1. Purpose and Area of Responsibility

The CLI (Command Line Interface) component of the input/output subsystem is designed to provide direct text interaction between the operator and the core of the Subota system via standard input/output streams (`sys.stdin` , `sys.stdout`) of the local terminal.

The area of responsibility of the subsystem includes:

- Asynchronous non-blocking polling and reading of user input in an interactive mode.
 - Primary parsing, filtering, and validation of incoming text strings.
 - Interception of debugging and service commands (injections into the event bus).
 - Initiating emergency and normal system shutdown procedures via a specialized `emergency_queue` .
 - Formatted output of system responses and alerts to stdout, taking into account event urgency levels.
-

2. Incoming Data Stream (`CommandLineInput`)

The input reading component is implemented in the `CommandLineInput` class, which inherits from the trusted base class of internal core services `SubotaInternalService` and is initialized at the third level of the hierarchy (tier=3).

2.1. Non-blocking Reading Mechanism

Since standard reading from `sys.stdin` is a blocking operation for the asynchronous event loop, thread isolation is applied in the module:

- In the `on_load` method, the component registers its working cycle in the core task manager via the `TaskWatcher` interface under the unique name `'CommandLineInput Loop'` .
- To perform input operations, an isolated thread pool `ThreadPoolExecutor(max_workers=1)` is deployed.
- The asynchronous `_input_loop` cycle calls the helper function `_read_input_safe` , which delegates the execution of the `sys.stdin.readline` function to the pool executor via the `run_in_executor` method. This prevents the main core thread from hanging while waiting for user input.

2.2. Fixed Context Identifiers

When generating data packets for internal routing, the module strictly defines context constants:

- `CLI_SESSION_ID = "cli_user_session"` — global identifier of the console session.
 - `CLI_SOURCE = "cli"` — request source marker.
 - `USER_ID = "system"` — system privilege level for local input.
-

3. Command Processing and Routing Protocol

Depending on the structure and prefixes of the incoming text string, `CommandLineInput` distributes packets across three different processing scenarios:

3.1. Shutdown Signal (`exit`)

If the user enters the `exit` command (case-insensitive), the emergency shutdown protocol is activated:

1. A `SubotaMessage` notification object is created, containing a unique generated `nonce` token (UUID4).
2. A packet of the form `{"event_type": "EXIT_SEQ_REQ", "payload": notification_payload}` is placed into the asynchronous infrastructure queue `emergency_queue`.
3. The internal console polling loop is interrupted.

3.2. Service Interception and Event Bus Injections (`//`)

For debugging and direct control of system components, syntactic interception of commands starting with the `//` prefix is implemented:

- The string is split by spaces. The first part (excluding `//`) is converted into the target event bus type `inner_answer_type`.
- The remaining arguments must be passed in a `key=value` format. They are parsed and stripped of accidental quotes, forming the parameter dictionary `parsed_payload`.
- **Special type handling:** For the `create_new_sensor` command, incoming parameters are automatically encapsulated into a nested structure `{'sensor_data': parsed_payload}`.
- The generated packet is sent directly to the internal data bus using the `publish_to_event_bus(answer_type=inner_answer_type, content=...)` method, bypassing the standard assistant command parsing interfaces.

3.3. Standard User Commands

Any text input that does not fall under the `exit` or `//` conditions is treated as a standard user command:

- The data is packed into a Payload containing basic metadata of the console session, the input text, and an exact Unix timestamp.
- The packet is sent to the core event bus with the type `user_command_input` for subsequent processing by the natural language understanding circuit.

Critical Error Handling

In the event of unexpected exceptions during the reading or parsing process, `CommandLineInput` does not terminate its operation; instead, it generates an emergency packet with the type `logging_error` containing the error description and transmits it to the data bus for the logging subsystem.

4. Outgoing Data Stream (`CLISender`)

To output system responses back to the console, the `CLISender` component is used, which inherits from the first-level base class (`BaseClass` , tier=1).

- **Interactive Verification:** The `send_message` method is called by the centralized `NotificationManager` . Before displaying the text on the screen, the component checks the `sys.stdin.isatty()` flag. If the script is running in a non-interactive environment (for example, as a background daemon or inside a CI/CD pipeline), the output is suppressed, and the message is sent to a debug logger to avoid cluttering system streams.
 - **Visual Formatting and Urgency:** Output to the terminal is visually separated from the rest of the log text by horizontal markers (`=====`). Depending on the passed urgency parameter (`urgency`), a graphical emoji prefix is automatically mixed into the system name `S.U.B.O.T.A.` :
 - `warning` → 
 - `error` → 
 - `info` → 
 - Default / empty type → 
-

5. Relationships and Architectural Constraints

1. **Dependence on Core Infrastructure:** `CommandLineInput` accesses system events and queues via infrastructure interfaces (`self.interfaces.infra`). In particular, `core_stop_event` is extracted from there to correctly stop the thread pool when the system shuts down.
2. **Architectural Routing Trade-off (Technical Debt):** When the `exit` directive is entered, the component interacts with the `emergency_queue` . As documented in [notificatio_manager.md](#), the current architecture routes emergency packets from this queue directly into the internal trusted core loop of the system, which is a temporary release solution and needs to be moved to an isolated `Watchdog/Supervisor` component in future sprints.

4.5 Models and NLP

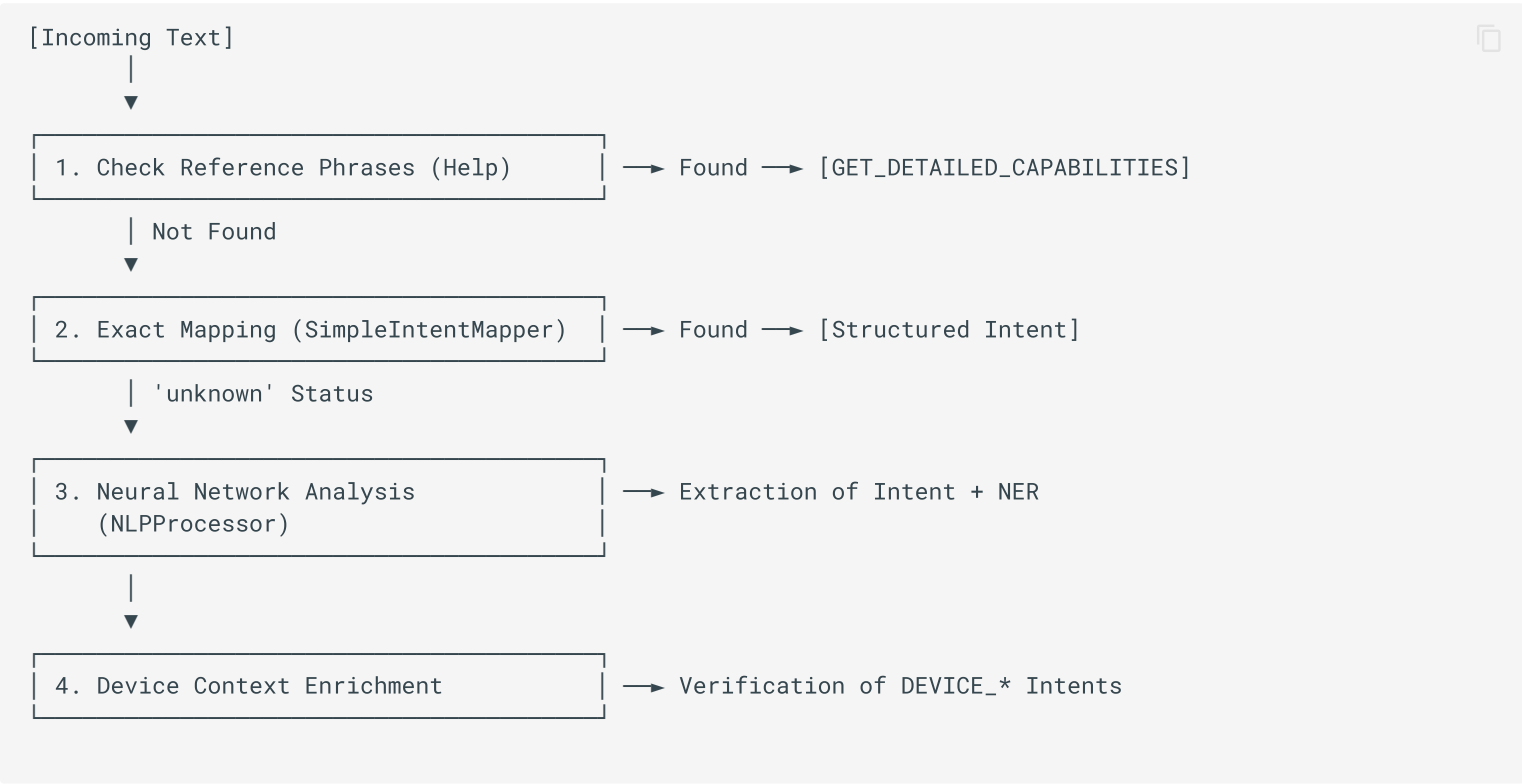
4.5.1 Component Specification: Intent Processor (Intent Processor)

1. Component Purpose

The module is designed to convert incoming user text commands into structured intents (Intents) while extracting named entities (Entities). The component orchestrates the operations of exact matching (Exact Matching), fuzzy search (Fuzzy Matching), and neural network classification (Multi-Task BERT) mechanisms.

2. Architecture and Processing Pipeline (Pipeline)

When the `process_intent` method is invoked, the processing of the incoming `raw_intent` string strictly passes through four sequential stages:



Stage 1: Filtering of Reference Requests (Pattern Matching)

The system converts the string to lowercase and checks it for the presence of rigidly defined prefixes:

- "расскажи подробнее про" - Tell me more about it
- "расскажи подробнее о" - Tell me more about
- "подробнее о" - more about
- "подробнее про" - more details about

If a prefix is detected, the remainder of the string (`target_name`) is stripped of whitespace and passed to the `_find_script_module_name` fuzzy search method. The method matches the target against the `script_documentation` script documentation dictionary.

- Utilized Library:** `rapidfuzz.fuzz.ratio`.

- **Cutoff Threshold (Threshold):** 75 . Both the module name (`module_name`) and the `doc[ "name" ]` field are compared.
- **Result:** Upon a successful match, the `GET_DETAILED_CAPABILITIES` intent is returned, specifying the target module `target_script` . If the threshold is not met, a system response regarding a help search error is returned.

Stage 2: Direct Matching (SimpleIntentMapper)

If the search prefixes are absent, the string is passed to the `SimpleIntentMapper` . The search is performed via an exact key matching method within the `INTENTS_TO_LEARN` dictionary.

Stage 3: Neural Network Classification (NLPProcessor)

If an exact match is not found (the `unknown` status is returned), the component lazily initializes the `NLPProcessor` (if it was not loaded previously) and passes the text to the model for analysis.

- If the model returns an empty intent or an intent with the value `UNKNOWN` , a response with the type `unrecognized_command` is generated.

Stage 4: Contextual Enrichment of Devices

If the resulting intent from the `NLPProcessor` starts with the `DEVICE_` prefix, the presence of the `target_id` key is verified:

- If `target_id` is missing but the string parameter `context` is provided, the `target_id` value is populated with data from `context` .
- If `target_id` is missing and `context` is empty, execution is aborted, and the `unrecognized_device` response type is returned.

3. Specification of Subcomponents

3.1 SimpleIntentMapper

Performs synchronous command matching against a static table. If a command is missing from the table, a structure with the text `'unknown'` and the original command in the `input` field is returned.

Static Command Registry (`INTENTS_TO_LEARN`):

| Incoming Command (Key) | Target Intent (<code>intent</code>) | Additional Payload Parameters |
|--|---------------------------------------|--|
| <code>статус системы</code> | <code>SYSTEM_STATUS</code> | — |
| <code>ВКЛЮЧИТЬ СВЕТ В ГОСТИНОЙ</code> | <code>device.set_power</code> | <code>device_id: 'light1', power: 'on'</code> |
| <code>ВЫКЛЮЧИТЬ СВЕТ В ГОСТИНОЙ</code> | <code>device.set_power</code> | <code>device_id: 'light1', power: 'off'</code> |
| <code>до свидания</code> | <code>EXIT_SEQUENCE</code> | <code>text: 'до свидания'</code> |

| Incoming Command (Key) | Target Intent (intent) | Additional Payload Parameters |
|------------------------|---------------------------|-------------------------------|
| пока пока | EXIT_SEQUENCE | text: 'пока пока' |
| что ты умеешь | GET_CAPABILITIES | text: 'что ты умеешь' |
| расскажи подробнее | GET_DETAILED_CAPABILITIES | target_id: None |
| что по свету | DTEK_STATUS | text: 'что по свету' |

3.2 NLPProcessor & MultiTaskBERT

A classifier based on the BERT architecture (BertPreTrainedModel) that simultaneously performs two tasks (Multi-Task Learning): named entity recognition (NER) and intent classification.

- **Model Location:** models/nlp/model (the absolute path is calculated relative to the project root via BASE_DIR).
- **Configuration Files:** Upon initialization, 4 identifier mapping files are strictly loaded from the model folder:
- label2id.json — mapping of text NER labels to numeric IDs.
- id2label.json — reverse mapping of numeric IDs to text NER labels.
- intent2id.json — mapping of string intents to numeric IDs.
- id2intent.json — reverse mapping of numeric IDs to string intents.
- **Tokenization Parameters:** Maximum text length max_length=512 , truncation is enabled truncation=True , returning token_type_ids is disabled.

Entity Assembly Algorithm (combine_continuous_entities)

Service tokens [CLS] , [SEP] , [PAD] are completely ignored. The following rules apply for gluing together words split by the tokenizer:

1. If a word starts with the ## prefix, its characters (without ##) are concatenated directly onto the text of the currently active entity.
2. If the label prefix equals B- or the entity type has changed, the current entity is closed, added to the final list, and a new record is opened.
3. If the label prefix equals I- and the type matches the current one, the text is appended with a space separator.
4. Labels with the value 0 are discarded at the primary filtering stage and are not included in the assembly.

4. Interface Contracts (Output Format)

The process_intent method always returns a dictionary or an object containing two root keys: answer_type (a string identifier of the action or error type) and content (the informational payload).

Examples of Actual Response Structures:

- **Upon Successful Triggering of the Help Workaround:**

```
{
  "answer_type": "GET_DETAILED_CAPABILITIES",
  "content": {
    "intent": "GET_DETAILED_CAPABILITIES",
    "target_script": "name_of_matched_module"
  }
}
```



- **In the Absence of Context for DEVICE Commands:**

```
{
  "answer_type": "unrecognized_device",
  "content": {
    "intent": "DEVICE_SOME_ACTION",
    "entities": []
  }
}
```



- **Upon Successful Parsing via the Model (NLPProcessor):**

```
{
  "answer_type": "INTENT_NAME_FROM_MODEL",
  "content": {
    "intent": "INTENT_NAME_FROM_MODEL",
    "entities": [
      {
        "type": "ENTITY_TYPE",
        "text": "extracted text"
      }
    ]
  }
}
```



4.6 Emotional Engine (Identity)

4.6.1 Technical Component & Interface Manual: Identity Subsystem

1. Document Purpose

This technical manual describes the physical structure of the `Identity` (Machine Spirit) subsystem codebase, data storage contracts in PostgreSQL, topology of Redis Streams queues, and requirements for the structure of configuration files (templates). The document is intended for engineers involved in the support, modification, and deployment of the subsystem.

2. File and Module Architecture

The subsystem is located in the `identity/` directory and consists of the following components:

- `identity/emotional_engine.py` : The main service of the subsystem, inherited from `SubotaInternalService` . It coordinates the lifecycle, manages subscriptions to the event bus, caches state in Redis, and initiates a clean shutdown with data preservation.
- `identity/vector_math.py` (`SpiritState`): The mathematical core and finite state machine (FSM). It computes changes in emotional vectors based on incoming events and determines the active system behavior protocol (`SUBOTA` , `OMNISSIAH` , `ALPHARIUS`).
- `identity/db_handler.py` (`EmotionalEngineDBHandler`): The data access layer (DAL) for working with PostgreSQL. It is responsible for retrieving history, updating the current state, and executing the routine nightly experience aggregation (archival).
- `identity/integrity_checker.py` (`IntegrityChecker`): The integrity control module for the project file system. It scans areas of responsibility based on `.gitignore` rules and generates core modification, skill modification, or data loss events.
- `identity/persona_decorator.py` (`PersonaDecorator`): The presentation layer (Presentation Layer). It loads text templates and performs contextual formatting/decoration of system responses in accordance with the active protocol.

3. Data Storage Specification (PostgreSQL)

The subsystem uses two tables for state persistence.

3.1 Table `machine_spirit_state`

Stores the operational state of vectors and metrics for the current (or last completed) working session. In normal operation, it contains a single up-to-date record.

| Field | Data Type | Description |
|--------------------------|--------------------------|---|
| <code>timestamp</code> | TIMESTAMP WITH TIME ZONE | Time of record capture (generated automatically by the DB). |
| <code>v_temper</code> | NUMERIC / FLOAT | Reactivity / aggression vector (from -1.0 to 1.0). |
| <code>v_stability</code> | NUMERIC / FLOAT | Stability / paranoia vector (from -1.0 to 1.0). |

| Field | Data Type | Description |
|--------------------------|----------------------|---|
| v_energy | NUMERIC / FLOAT | Resource state vector (from -1.0 to 1.0). |
| v_affinity | NUMERIC / FLOAT | User loyalty vector (from -1.0 to 1.0). |
| last_commands | JSONB / TEXT (Array) | History of the last 10 recognized user intents. |
| integrity_manifest | JSONB / TEXT | Complete snapshot of project file hashes at the time of scanning. |
| total_commands_count | INTEGER | Cumulative counter of processed commands for the session. |
| error_count | INTEGER | Cumulative counter of system errors for the session. |
| security_incidents_count | INTEGER | Cumulative counter of security incidents for the session. |
| was_clean_exit | BOOLEAN | Flag indicating whether the previous session terminated cleanly. |
| session_duration_seconds | INTEGER | System operation uptime in seconds. |
| version_tag | VARCHAR | Core version tag (defaults to 3.0.0-alpha). |
| last_wake_up | TIMESTAMP | Time of the last system startup. |
| event_description | TEXT | Description of the latest system event. |

3.2 Table spirit_experience_archive

A historical archive for long-term trend analysis ("memory") of the Machine Spirit. It is populated automatically during the nightly archival cycle.

- **Unique Key / Constraint:** date_day (ON CONFLICT DO NOTHING).

| Field | Data Type | Description |
|---------------|-----------------|---|
| date_day | DATE | Date of the archived day (Primary Key). |
| avg_temper | NUMERIC / FLOAT | Daily average value of the reactivity vector. |
| avg_stability | NUMERIC / FLOAT | Daily average value of the stability vector. |

| Field | Data Type | Description |
|--------------------|--------------------|--|
| avg_energy | NUMERIC /
FLOAT | Daily average value of the energy vector. |
| avg_affinity | NUMERIC /
FLOAT | Daily average value of the loyalty vector. |
| dominant_protocol | VARCHAR | Protocol that prevailed during the 24-hour period (SUBOTA , OMNISSIAH , or ALPHARIUS). |
| event_log | TEXT | Combined log of key textual descriptions of events for the day. |
| total_interactions | INTEGER | Total number of state transactions over the 24-hour period. |
| top_intents | JSONB | Top 5 most frequently used user intents (format: {"intent_name": count}). |

4. Redis Event Bus Integration Map

The `EmotionalEngine` tooling interacts with the rest of the system via a distributed bus based on Redis Streams. To ensure fault tolerance, dedicated Consumer Groups are created.

- **Service Base Name:** `emotional_engine`.
- **Consumer Name:** `consumer_emotional_engine_1_[suffix]`.

Streams and Subscriptions Map

| Stream Name
(Event Type) | Consumer Group | Handler Purpose in Module |
|-----------------------------|--|---|
| assistant_response | group_emotional_engine_resp | Interception of outgoing assistant responses for their subsequent emotional decoration. |
| log_to_notify_event | group_emotional_engine_logs | Analysis of incoming core system logs. Changes stability during critical failures and errors. |
| last_intent_stream | group_emotional_engine_intents | Capture of successfully recognized user command intents. Accumulation of history (up to 10 entries) in the DB. |
| unrecognized_commands | group_emotional_engine_unknown_command | Fallback stream for handling unrecognized or malformed requests. |
| user_command_input | group_emotional_engine_input | Tracking input source metadata (<code>source</code> and <code>session_id</code>) to maintain session context. |

Outgoing Events

After modifying text or changing vector states, the subsystem publishes an event to the `emotional_assistant_response` stream. The payload (`payload`) contains:

- Adjusted/decorated text;
- Session markers (`source` , `session_id`);
- Metadata `emotional_meta` , including current numerical values of all 4 vectors and the string identifier of the active protocol.

When vectors change, the current state is also duplicated into the operational Redis hash cache under the key `subota:system:status:cache` in the `spirit_state` field.

5. JSON Template Contract and Specification

The `PersonaDecorator` module expects a strict structure of JSON files in the `identity/templates/` directory. Each file must contain sections for the three base protocols.

5.1 Format `mood_templates.json`

Defines prefixes and suffixes for decorating regular system responses based on impact keys (`impact_key`).

```
{
  "SUBOTA": {
    "COMMAND_ACK": {
      "prefixes": ["Принято. ", "Выполняю: "],
      "suffixes": ["", " Сделано."]
    },
    "COMMAND_REPETITIVE": { ... }
  },
  "OMNISSIAH": {
    "COMMAND_ACK": {
      "prefixes": ["[++ Направляю импульс к механизму ++]\n"],
      "suffixes": ["\n[++ Процесс завершен во славу Омниссии ++]" ]
    }
  },
  "ALPHARIUS": { ... }
}
```

5.2 Mandatory Lifecycle Template Files

The support engineer must guarantee the presence of the following files, whose structure repeats the division by protocols (`SUBOTA` / `OMNISSIAH` / `ALPHARIUS`), but contains context-specific lists of strings:

- `identity/templates/start_frazes_template.json` — startup greeting phrases.
- `identity/templates/exit_frazes_template.json` — phrases used during shutdown or termination (`EXIT_SEQUENCE`).
- `identity/templates/lifecycle_template.json` — phrases assessing the nature of the launch (`START_CLEAN` , `START_AFTER_CRASH` , `START_LOW_RESOURCES`).
- `identity/templates/session_duration_template.json` — phrases reacting to the duration of the previous session (`RESTART_PREMATURE` , `RESTART_AFTER_LONG_RUN` , `RESTART_CLEAN`).

5. `identity/templates/timestamp_template.json` — phrases for assessing user absence time (`LONG_ABSENCE_CRITICAL` , `LONG_ABSENCE_WARNING` , `QUICK_RESTART`).
 6. `identity/templates/manifest_template.json` — phrases for the file integrity control subsystem (`INTEGRITY_FIRST_SCAN` , etc.).
 7. `identity/templates/impact_template.json` — general trigger lists of text reactions to state changes.
-

6. Routine Processes and Decay

6.1 State Decay (`_decay_loop`)

A running asynchronous loop with a periodicity of **once every 60 seconds** smoothly returns emotional background parameters to baseline (zero) values, simulating the dissipation of impulses:

- The reactivity vector (`v_temper`) is multiplied by `0.95` .
- The energy vector (`v_energy`) is multiplied by `0.98` .
- The stability vector (`v_stability`) asymptotically approaches the ideal value of `1.0` with a recovery rate of `1%` per minute.

6.2 Nightly Archival (`_nightly_archive_loop`)

Every day at **03:00 local time** (determined by the `location_timezone` configuration), a procedure for clearing the operational table and aggregating logs for the past 24 hours is launched. Data is averaged, top intents are calculated using a built-in `Counter` accumulator, and then the operational history from `machine_spirit_state` for the past day is deleted to prevent disk space overload.

4.7 Embedded scripts

4.7.1 Built-in Scripts Reference

All built-in scripts of the S.U.B.O.T.A. system inherit from the base class `SubotaInternalService`. They are executed within the context of the `script_runner` component, isolating their logic inside internal event loops (Loops), and perform asynchronous messaging via the `event_bus` event bus using the `SubotaMessage` data structure.

1. General Rules for Handling Debug Mode (Dry Run)

Upon initialization, every built-in script checks the debug flag `interfaces.infra.dry_run`. If `dry_run == True`, the execution of actual business operations (shutting down a server, sending network requests, publishing responses to the user) is blocked, and information is rerouted exclusively to the system log with a special `DRY RUN` marker.

2. Directory of System Scripts

2.1. Alarm Handler (`alarm_handler/v3.py`)

- **System Name (service_name):** `alarm_handler_script`
- **Subscription Point (Event Bus):** Group `group_alarm_handler_script`, consumer `consumer_alarm_handler_script_1`.
- **Triggers (Intents):** `SCHEDULE_ALARM`, `SCHEDULE_REMINDER`.
- **Operating Logic:** Intercepts events from the internal task scheduler. Extracts the task name (`task_name`) and the original text command (`original_command`) from the message body (`payload.data.get('content')`). Generates a notification for the user.
- **Generated Response:** Publishes an event of type `assistant_response` with formatted text: `🔔 НАПОМИНАНИЕ: Пришло время выполнить {task_name}! ...`

2.2. Shutdown Script (`exit_sequence/v3.py`)

- **System Name (service_name):** Dynamic (`__name__`)
- **Subscription Point (Event Bus):** Group `group_scripts.exit_sequence.v3`.
- **Triggers (Intents):** `EXIT_SEQUENCE`, `OFF_BOT`.
- **Dependencies:** Farewell templates file `scripts/exit_sequence/farewells.json`.
- **Operating Logic:**
 - Validates the event transmission time: if the difference between the current server time and the event timestamp (`ts`) exceeds 10 seconds, the event is ignored to avoid phantom shutdowns when reading old logs.
 - Loads an array of strings from the JSON file and selects a random farewell message (`random.choice`).
 - Publishes it to `assistant_response`, pauses for 0.5 seconds, and sends the `EXIT_SEQ_REQ` command to the critical core queue `emergency_queue` for a complete process shutdown.

2.3. Internet Check (`internet_check/v3.py`)

- **System Name (service_name):** `internet_check`
- **Triggers (Intents):** `INTERNET_CHECK`.

- **Utilized Interfaces:** `network_controller`.
- **Operating Logic:** Performs a comprehensive asynchronous verification of the network stack:
- Sends an ICMP request (ping) to the public Google DNS `8.8.8.8` with a 2-second timeout.
- Executes a test HTTP request to `[https://www.google.com](https://www.google.com)` (3-second timeout).
- Requests the external gateway IP address via `get_external_ip()`.
- **Output Data:** If both checks are successful, returns an `available` status and the current external IP address. In case of failure, returns an `unavailable` or `error` status.

2.4. Reminder Creation (`reminder_script/v3.py`)

- **System Name (service_name):** `reminder_handler_script`
- **Triggers (Intents):** `ADD_REMINDER`.
- **Metadata Particularity:** In the source code, metadata mistakenly duplicates the description of the alarm handler.
- **Natural Language Parsing (NLP/Regex):** Independently parses the incoming string using regular expressions based on three patterns:
- *Relative:* `"через X минут/часов/дней"` (calculates `timedelta`).
- *Cyclic:* `"каждый день в HH:MM"` or `"каждый день в HH.MM"` (rolls over to the next day if the time for today has already passed, setting a periodicity flag).
- *Absolute:* `"в HH:MM"` (for the current or next 24-hour period).
- **Action:** Cleanses the text of stop words (`"напомни"` , `"создай задачу"` , etc.), and then registers the task in the scheduler `scheduler.add_persistent_task` with a trigger type of `once` and a target intent of `SCHEDULE_REMINDER`.

2.5. System Status (`system_status/v3.py`)

- **System Name (service_name):** `script_runner` (module base name: `system_status`).
- **Triggers (Intents):** `SYSTEM_STATUS`.
- **Interfaces:** Storage `redis_client`.
- **Operating Logic:** Invokes an atomic `hgetall` operation for the Redis key `subota:system:status:cache`. Iterates over the retrieved data, automatically deserializing JSON strings of any nesting level. Passes the aggregated dictionary to the report generator `StatusReportBuilder`.
- **Response Routing:** Adapts the report delivery destinations (`destinations`) based on the request source. If the request originated from `voice_input`, the response is simultaneously duplicated to the console (`cli`) and to `telegram`. In all other cases, it is routed strictly back to the source (`source`).

2.6. Task Scheduler (`task_scheduler/v3.py`)

- **System Name (service_name):** `script_runner` (module `task_scheduler`).
- **Triggers (Intents):** `ADD_TASK`.
- **Operating Logic:** Accepts structured NLP entities (`entities`) of types `TIME` , `DATE` , `TEXT`.
- Relative time interval markers (`"через"` , `"спустя"` , `"позже"`) are processed by a rigid deterministic finite state machine (FSM) based on built-in dictionaries of numerals and time units.
- Absolute values are delegated to the `dateparser` library with support for Russian, Ukrainian, and English languages, explicitly bound to the timezone of the local infra-environment (`self.timezone`).

- **Protection:** Blocks attempts to schedule tasks in the past (with a validation threshold of 10 seconds from current UTC).
- **Result:** Adds a persistent task to the `scheduler` with an ultimate execution intent of `SCHEDULE_ALARM`.

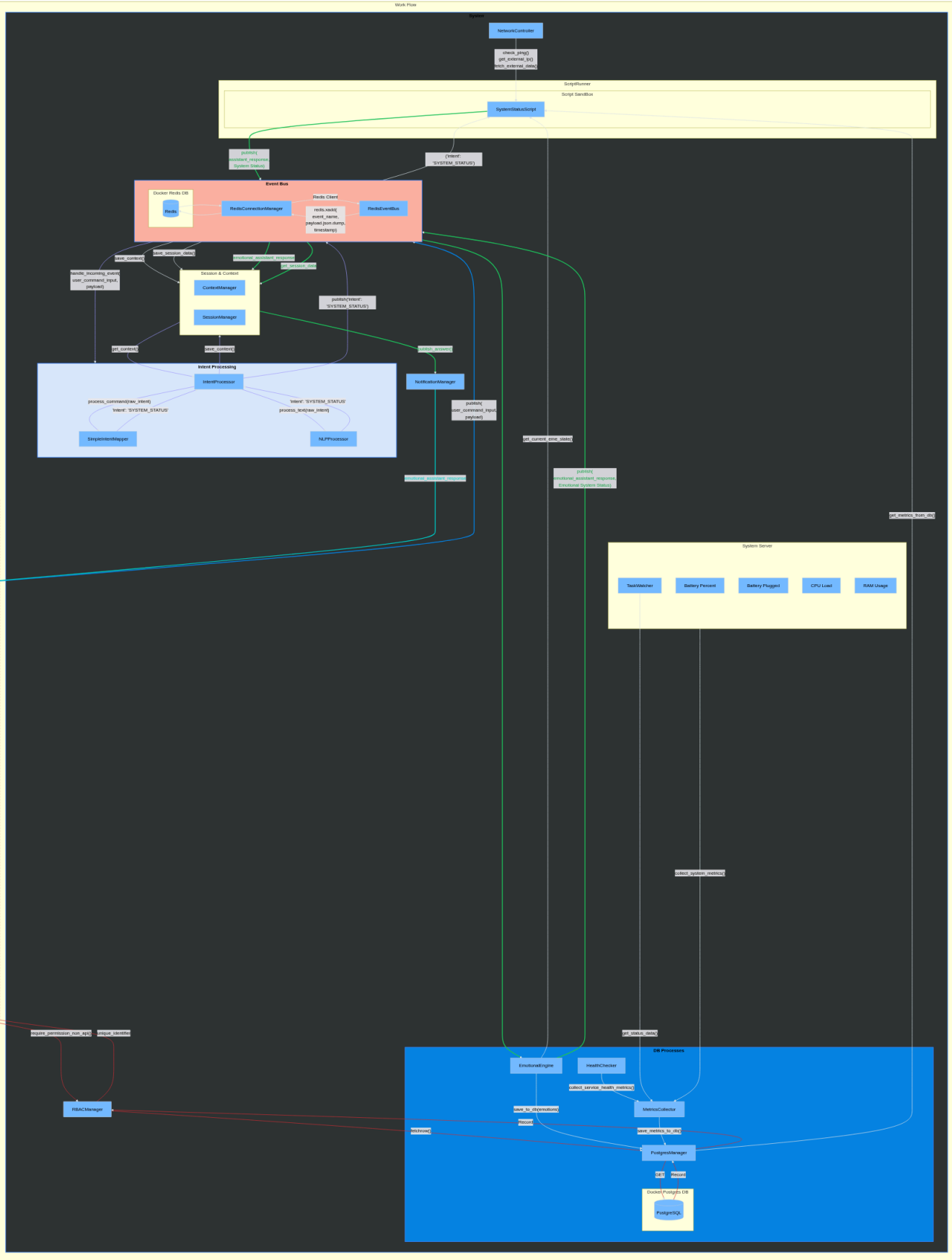
2.7. Time Check (`time_check/v3.py`)

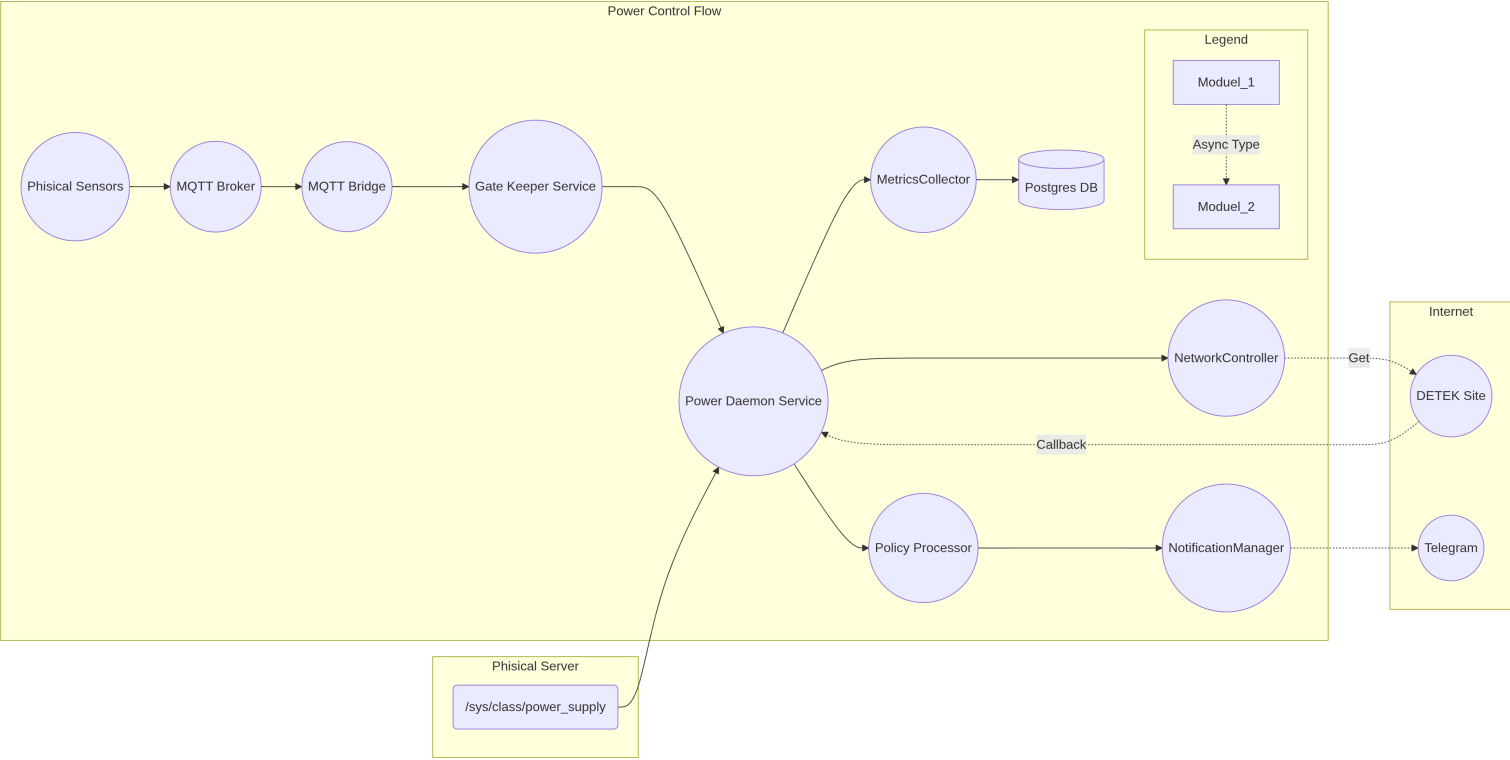
- **System Name (service_name):** `script_runner` (module `time_check_script`).
- **Triggers (Intents):** `TIME_CHECK`.
- **Operating Logic:** A very simple atomic script. Reads the current system time of the server and sends a text message in the `HH:MM` format via `assistant_response`.

2.8. Weather Informer (`weather_script/v3.py`)

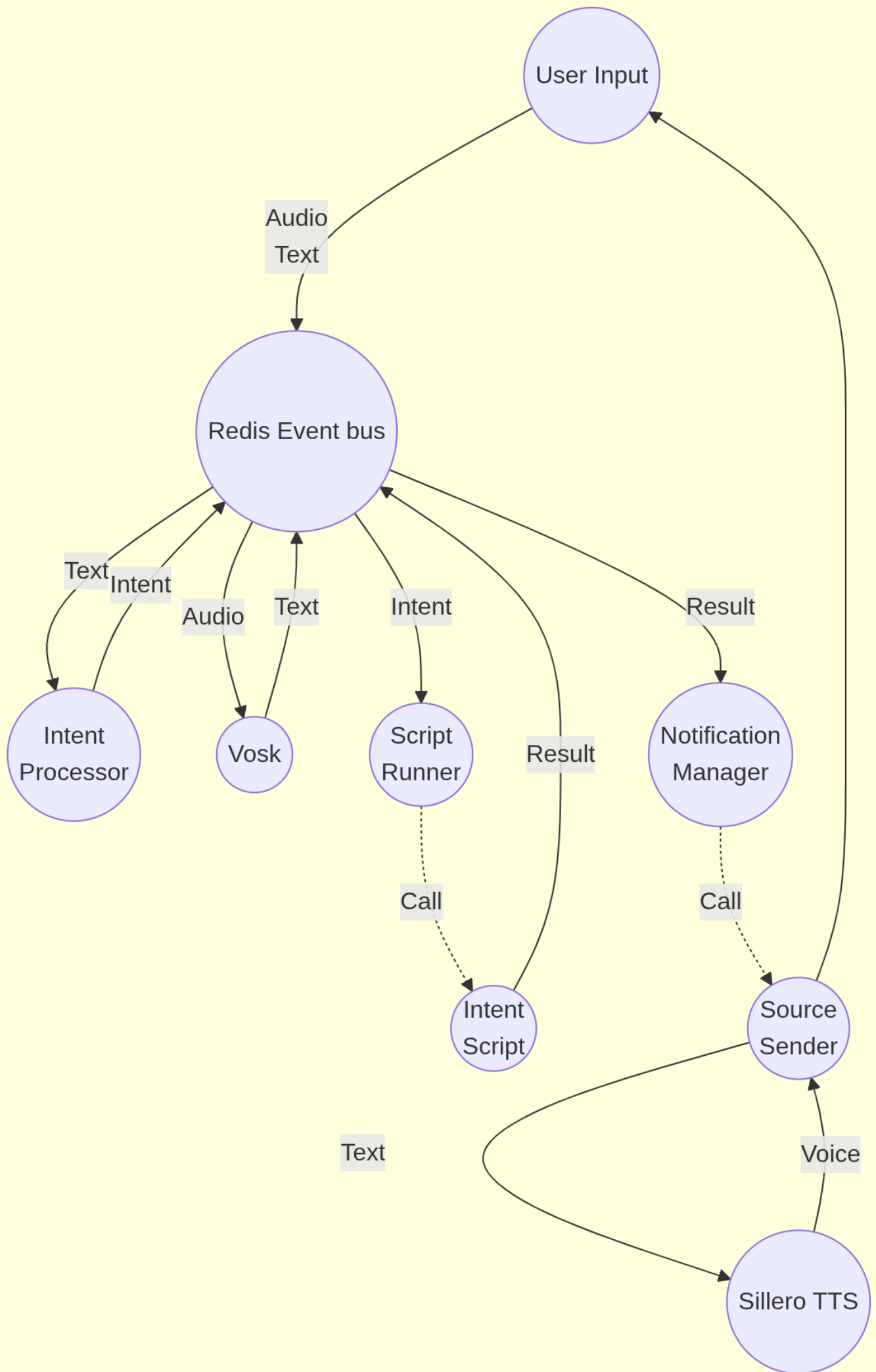
- **System Name (service_name):** `weather_script`
- **Triggers (Intents):** `WEATHER`.
- **Dependencies and Integrations:** `SecretManagerProtocol` (key `WEATHER_KEY`, home coordinates `home_lat`, `home_lon`), external OpenWeatherMap API, and the `resolve_city` module (geocoding city names).
- **Caching:** Implements method-level LRU caching (`@alru_cache`) with a maximum capacity of 10 unique cities and a record Time-To-Live (TTL) of 1800 seconds (30 minutes).
- **Output Formatting by Perception Channels:**
 - *Channel `voice` (Voice Interface):* The text is fully converted into words for numerals using the `num_to_text` module (e.g., "Температура двадцать два градуса"), so that the speech synthesizer correctly reads the units of measurement. Wind gusts are spoken out as text.
 - *Other Channels (Text Messengers/CLI):* Generates structured text utilizing visual Emojis, line breaks, showing the temperature in numeric format (`°C`) and the wind direction.

5 Diagrams





User Input Flow



6 Status

6.1 S.U.B.O.T.A. — Architectural Design

System Unit of Binary Operations & Technical Assistance

Document Version: 3.0-draft | Status: Pre-production

This document describes the system's behavioral model, user experience, and interaction principles. It does not contain implementation requirements — refer to the Technical Specification for those.

6.1.1 Table of Contents

- 1. System Identity and Voice
- 2. Interaction Interface
- 3. Voice Loop
- 4. Behavioral Engine
- 5. Modular UX Map
- 6. Notification Principles

6.1.2 1. System Identity and Voice

S.U.B.O.T.A. is an autonomous home system. In communication with the operator, it adheres to the following principles:

Communication Style:

- Concise and to the point. No filler phrases.
- Informs, does not lecture. If the operator has made a decision, the system executes or logs the risk, but does not repeat warnings twice.
- In ambiguous situations, it clarifies with a single short question.
- In critical situations — only facts and specific actions, no "atmosphere."

Interface Language:

- Primary interface language: Russian.
- Technical identifiers (sensor IDs, event codes): Latin script.
- In Telegram notifications: obfuscated identifiers without geodata (see Technical Specification, Section 11).

6.1.3 2. Interaction Interface

Channels

| Channel | Type | Application |
|-------------------------|------|---|
| Server Physical Console | Text | Administration, cold start, LUKS passphrase entry |

| Channel | Type | Application |
|-----------------------|--------------|--|
| Voice Interface | Voice | Household commands, queries, scenario management |
| Telegram Bot | Text/Buttons | Notifications, metrics, home automation |
| Discord Bot | Text | Alternative channel, lower priority |
| Companion App | GUI | MFA, system status viewing, RBAC management |
| Web Interface (debug) | Text | Event stream viewing. Read-only. V3.0 only. |

Channel Prioritization Logic

When a command is received simultaneously from multiple channels:

Physical Console > Companion App > Voice > Telegram > Discord

Command conflict (e.g., voice says "turn on lights," Telegram says "turn off" within < 2 sec) → system pauses both and clarifies through the priority channel.

6.1.4 3. Voice Loop

Minimal Dependency Principle

Voice input is a convenience, not a critical function. All security, monitoring, and degradation logic works without voice.

Stack (V3.0)

| Component | Technology | Works Offline? |
|--------------------------------|---------------------------------|----------------|
| Wake Word | Porcupine (wake word: «Субота») | Yes |
| VAD (Voice Activity Detection) | WebRTC VAD | Yes |
| Speech Recognition (STT) | Whisper (local, INT8-quantized) | Yes |
| Speech Synthesis (TTS) | Coqui TTS (local) | Yes |
| NLP Processing (Intents) | FastText + Rules | Yes |

Voice Verification (Biometrics)

- Used only as a second factor for critical commands (see Technical Specification, Section 8).

- Not an identification system for "who is speaking" for routine commands — routine commands are accepted from any verified voice source within the home.
- During degradation ("Mid" or "Min" modes), voice input is limited to security commands. LLM dialogue is unavailable.

Voice Loop Degradation

When RAM is low (< 500 MB free), Whisper is the first component unloaded from the voice stack. Porcupine (wake word) remains active — the operator receives a text response via Telegram that voice input is temporarily unavailable.

6.1.5 4. Behavioral Engine

Principle

The Behavioral Engine is not a "personality" system nor a "character" switcher. It is an adaptive adjustment of the communication register and processing priorities based on situational context.

The state vector is described by four metrics:

| Metric | What it Measures | Range |
|-----------|--|-----------|
| Stability | System stability (Uptime, errors, degradations over last 24 h) | 0.0 – 1.0 |
| Energy | Battery charge level / power availability | 0.0 – 1.0 |
| Threat | Threat activity on the perimeter (sensor anomalies, ban-lists) | 0.0 – 1.0 |
| Load | Normalized CPU load average / max_expected_load | 0.0 – 1.0 |

The vector updates in real-time. Based on it, the following are adjusted:

- Volume and tone of text responses (concise vs. detailed).
- Notification trigger thresholds (what becomes an alert vs. silent logging).
- Priority of the operator request processing queue.

Communication Modes

| Mode | Activation Condition | Behavior |
|-----------|----------------------------------|---|
| Normal | Threat < 0.3, Energy > 0.5 | Full responses, proactive notifications, LLM dialogue available |
| Vigilant | Threat 0.3–0.7 OR Energy 0.2–0.5 | Notifications only for essentials, LLM dialogue limited |
| Emergency | Threat > 0.7 OR Energy < 0.2 | Only critical notifications. Responses — one phrase. All resources to security. |

Thresholds are not rigid: they are starting values for configuration via the config file. The operator can adjust them via the console.

Behavioral Learning (Learning Gap)

Caveat: any learning in V3.0 is offline only, via manual approval.

The system registers recurring commands not mapped in the current configuration and generates a proposal:

```
"Recurring command detected: [X]. Current configuration does not handle it automatically.
Recommendation: [description of proposed rule]. Deploy? [Yes / No / Defer]"
```

Autonomous learning (changing weights, rules, or logic without operator confirmation) — excluded in V3.0 and V4.0. Horizon for introduction: V5+, only after security audit.

6.1.6 5. Modular UX Map

Module Map by Domain



Playbooks

Playbooks are named chains of actions triggered by an event. Not algorithms: each step is atomic and can be blocked by the `policy_engine`.

Example V3.0 Playbooks:

| Playbook | Trigger | Chain |
|------------|------------------------------------|--|
| Night Mode | Voice "Good night" / time 23:00 | Dim lights → Turn off non-essential outlets → Activate perimeter → Quiet notification mode |
| Alarm | Motion sensor (Night mode) | Photo capture → Alert via Telegram → Increase sensitivity of other sensors |
| Guest | Voice "Guest mode" | Deactivate internal motion sensors in guest areas → Temporary RBAC role "Guest" |
| Away | Voice "Leaving" / Geofence (V4.0+) | Turn off everything except perimeter and fridge → Activate "Security" mode → Heartbeat to phone every hour |

6.1.7 6. Notification Principles

Severity Levels

| Level | Channel | Example |
|----------|---|---|
| INFO | Silent log to Postgres | Temperature sensor reported value out of range by 0.5°C |
| WARNING | Telegram (no sound) | Sensor unresponsive for 5 minutes |
| ALERT | Telegram + sound | Motion sensor triggered in Night mode |
| CRITICAL | Telegram + sound + repeats until acknowledged | Perimeter breach / Fire sensor |

Quieting Rule

| Level | Collapse Window | Note |
|----------|---------------------|--|
| INFO | 30 minutes | Counter in one message: "Sensor [Z3] - 12 triggers in 30 min." |
| WARNING | 5 minutes | Counter in one message |
| ALERT | 2 minutes | Counter in one message |
| CRITICAL | Collapsing disabled | Every trigger logged to Postgres without limits. Operator notifications — rate limit 1 repeat / 5 sec. |

CRITICAL Incident Acknowledgment

Each CRITICAL notification in Telegram includes a button:

[S.U.B.O.T.A.] ● CRITICAL: Zone_B / Sensor_E32_A4
Status: ACTIVE | 03:47:12

[✓ Acknowledged]

Before acknowledgment: repeat 1 time / 5 sec. **After acknowledgment:** repeat reduced to 1 time / 5 min ("aware, monitoring").
Incident closure: automatically when the sensor returns to normal. Postgres logs: who acknowledged, time, channel.

Escalation on Lack of Acknowledgment

If no one acknowledges — the system sequentially expands the notification circle by roles. Roles and contacts are configured by the operator via the console.

| Level | Role | Delay from Incident Start |
|-------|---------------------|---------------------------|
| 1 | Operator | Immediately |
| 2 | Residents | +2 min |
| 3 | Friends / Neighbors | +5 min |
| 4 | Children / Parents | +10 min |
| 5 | Close Acquaintances | +15 min |

Any recipient in the chain can press "Acknowledged" — this stops further escalation and reduces repeat frequency to 1/5 min for the entire chain.

Channel Escalation for Hacking / Intrusion

For incidents of class `INTRUSION` , the standard text channel is insufficient. The system sequentially activates channels:

1. Telegram / Discord (text)
↓ (no acknowledgment, dynamic delay*)
2. Voice TTS in the home (via speakers)
↓ (no acknowledgment, dynamic delay*)
3. Next in role chain (Telegram)
↓ ...

The system continues until acknowledgment is received from any recipient, regardless of the time of day.

*Dynamic delay (presence in the home + time of day) — implemented in V4.0 together with the `PresenceEngine` . In V3.0, delays are fixed: text → voice after 2 min without acknowledgment.

Obfuscation in Messengers

Notifications sent to external channels (Telegram, Discord) do not contain:

- Real first or last names.
- Geodata.

- Room names or addresses.
- Descriptions of physical house layouts.

All entities are hash-identifiers. Human-readable decryption is available only in the local interface.

6.2 S.U.B.O.T.A. — Technical Specification

System Unit of Binary Operations & Technical Assistance

Document Version: 3.0-draft | **Status:** Pre-production

6.2.1 Table of Contents

- Hardware Base and Limitations
- Network Topology
- Operating Modes
- Core Software Architecture
- Interaction Protocols
- Resource Management
- Data Lifecycle
- Security and Authentication
- State Machine and Degradation
- Startup and Recovery Procedure
- Monitoring and Emergency Notification
- V3.0 Readiness Criteria (Definition of Done)

6.2.2 1. Hardware Base and Limitations

Server

- Model:** Lenovo IdeaPad 330-15IKB

| Component | Characteristics | Hard Limit |
|-----------------|----------------------------------|--|
| RAM | 8 GB | Swap is forbidden (<code>swapoff -a</code>). All containers have strict cgroup limits. |
| Storage | HDD (mechanical) | High-intensity write operations are forbidden. WAL logs are optimized. Cache flushing is asynchronous. |
| GPU | Nvidia GeForce MX110 (2 GB VRAM) | Not used for LLM inference. MX110 does not support CUDA for modern neural networks. All AI inference runs strictly on the CPU via quantized models (FastText, SentenceTransformers). |
| Network | Ethernet + Wi-Fi | Ethernet → Mikrotik_1 (external network). Wi-Fi → Mikrotik_2 (isolated sensor network). |
| Thermal Profile | Laptop, 24/7 | Upon reaching 80°C, the system reduces the sensor polling frequency. At 85°C, it disables background analytics and vectorization. |

GPU Note: Section 2 of the VRAM budget (allocation of 1500 MB for LLM) refers to design goals for versions V5+, when a transition to a hardware-accelerated model is planned. In V3.0, the GPU is not utilized.

OS

Linux. The specific distribution is locked during the first on-hardware testing phase. Mandatory requirements prior to distribution selection:

- cgroups v2 support
- iptables/nftables
- systemd
- LUKS partition encryption

Peripherals (Sensor Mesh)

- **Node types:** Android (telemetry), ESP32/Arduino (raw sensors).
- **Estimated capacity V3.0:** Up to 50 active nodes.
- **Estimated capacity V4.0+:** Up to 200 active nodes (subject to validation via load testing).
- **Backpressure:** If the Redis queue overflows, non-critical packets are dropped (drop policy) to protect against OOM.

| Sensor Type | Mode | Maximum Frequency |
|--|-----------------|--|
| Event-driven (motion, door, emergency button, fire) | Push on trigger | No frequency limits; rate limit: maximum 10 events/sec per node (flood protection) |
| Periodic (temperature, humidity, charge) | Polling | 1 packet / 30 sec in nominal mode; 1 packet / 5 min in "Minimum" mode |
| System heartbeats (Dead Man's Switch) | Polling | 1 packet / 30 sec (fixed) |

6.2.3 2. Network Topology

Routers

| Identifier | Model | Purpose | Manageability |
|-------------------|-----------------|--|----------------------------------|
| Mikrotik_1 | RB2011UiAS-2HnD | External Internet. Ethernet only. | Unmodifiable by the system |
| Mikrotik_2 | RBSXTsq5HPnD | Isolated sensor network. 5 GHz Wi-Fi. No Internet. | System-managed (Mikrotik_2 only) |

Data Diode

Transitions between networks are impossible in both directions:

- Accessing Mikrotik_2 from Mikrotik_1 is **impossible**.
- Accessing Mikrotik_1 from Mikrotik_2 is **impossible**.

Implementation on the server:

- **Ethernet Interface (Mikrotik_1):** `iptables` in `ESTABLISHED,RELATED` mode. All incoming connections (`INBOUND`) are dropped (`DROP`). The server initiates outbound connections but does not accept incoming ones.
- **Wi-Fi Interface (Mikrotik_2):** Routing to the external network is disabled at the router level.

Mikrotik_2 Network — Sensor DMZ

- Closed whitelist of devices.
- A new sensor receives `QUARANTINE` status: it can only send metrics and cannot request core data.
- Full admission — only via manual Operator authorization at the physical server console.
- Any unregistered node is blocked at the hardware level without the ability to send data to the network.

Network Power Priority

Both routers are powered by an independent source (battery pack, estimated autonomy: 24 hours). The network remains active even if the server shuts down.

6.2.4 3. Operating Modes

The system operates in three deterministic modes. A mode change is always accompanied by a user notification.

"Maximum" Mode (Nominal)

- **Activation conditions:** Stable power supply + Internet available + perimeter normal.
- **Active functions:**
 - All modules and services are running.
 - Local LLM is available in memory.
 - Proactive behavior pattern analysis.
 - Full perimeter control.
 - Detailed logging.
 - Voice interface + messengers.
- **Priority:** User convenience.

"Medium" Mode (Degradation)

- **Activation conditions:** Power outages **OR** internet loss **OR** perimeter anomalies. The transition is initiated when the battery charge drops below 25% (stable for 60 seconds).
- **Active functions:**
 - LLM is unloaded from memory. Classic NLP is used instead (SentenceTransformers / FastText, ~100–200 MB RAM).
 - *On Internet loss:* Responses to queries from the local knowledge base (trimmed dump).

- *On power outages (Internet available):* Heavy computations shift to external services — unclassified queries only, stripped of context.
- *On Internet loss:* Heavy computations are not executed. Responses come from the local cache.
- *Under both conditions simultaneously:* Local cache only.
- *On perimeter anomalies:* Increased sensor polling frequency, resource priority shifted to security.
- Incoming traffic from sensors is accepted in batches (Batching).
- The system does not trust the external Internet, even if available — responses come from the local cache.
- **Priority:** Home and life safety.

"Minimum" Mode (Survival / Emergency)

- **Activation conditions:** Battery charge below 20% **OR** a major perimeter security breach.
- **Active functions:**
 - Security perimeter loop only.
 - Non-security household queries are ignored or deferred.
 - All modules with Tier ≥ 4 are unloaded.
 - Perimeter control transitions to autonomous ESP32/Android nodes (powered by power banks, heartbeat every 5 minutes instead of a constant stream).
 - The Lenovo server can be turned off via software (ACPI script) to maximize autonomous runtime.
- **Priority:** Human life and health.

Mode Switching Hysteresis

To eliminate bouncing during charge fluctuations, a double hysteresis is introduced: a 5% margin + a 60-second time buffer.

- **DEGRADATION (Drop):** [100% $\rightarrow$ 25%] —(stable 60 sec)—> [preparing for degradation] —(down to 20%)—> [MINIMUM]
- **RECOVERY (Rise):** [20% $\rightarrow$ 25%] —(stable 60 sec)—> [module-by-module startup by Tier] —> [MEDIUM/MAXIMUM]

The upward transition from "Minimum" mode is unlocked only at a charge of $\geq 25\%$, not at 21–24%.

6.2.5 4. Core Software Architecture

Principle of Operation

Hybrid model: a single lightweight asynchronous core (`asyncio`) + isolated OS processes for heavy computations (`multiprocessing`).

V3.0 Technology Stack

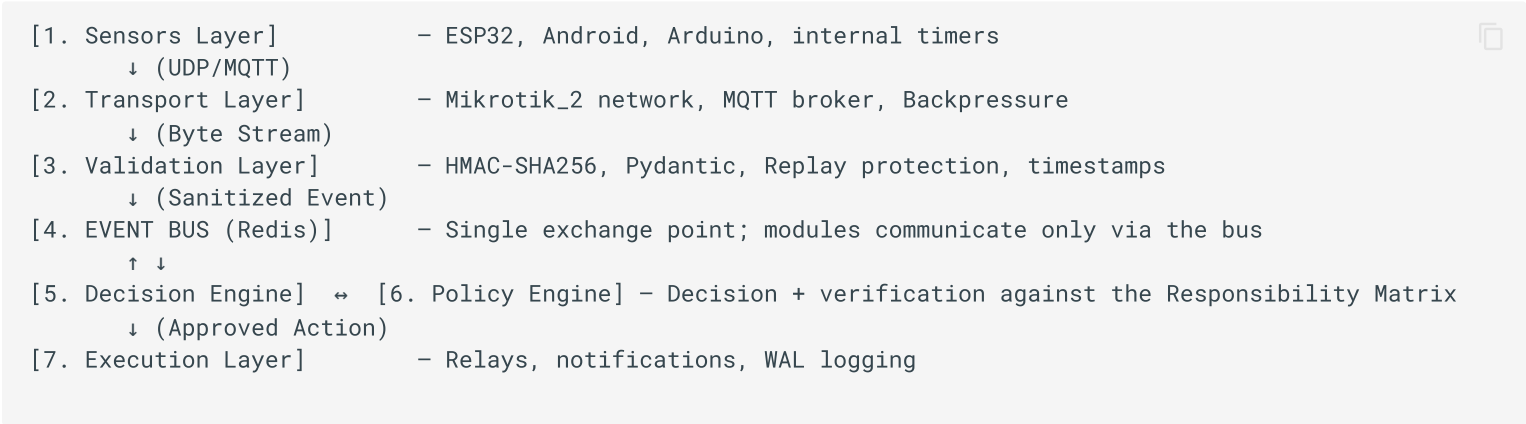
| Layer | Technology | Rationale |
|---------------|------------------------|--|
| Language | Python 3.12+ (asyncio) | Optimal for I/O-bound tasks. |
| Orchestration | Docker + Compose | Isolation, limit management via cgroups. |

| Layer | Technology | Rationale |
|-------------------|---------------------------|---|
| IPC | Redis Pub/Sub + Streams | Single bus. Direct calls between modules are forbidden. |
| Storage | PostgreSQL | Structured data (RBAC, configs, metrics). Logs are not in the DB. |
| Emergency Storage | SQLite (emergency_buffer) | Emergency buffer if Postgres/Redis fails. |
| Messaging | MQTT (Mosquitto) | Lightweight IoT protocol. |
| Validation | Pydantic V2 | Strict data contract. |
| API | FastAPI | Minimal overhead, async. |
| NLP (V3.0) | FastText / Porcupine | Classic algorithms only. LLM — only in "Maximum" mode, V4+. |

Freeze Policy: Any new dependency in V3.0 passes through two filters: (1) is it necessary for the basic Sensor → Bus → Actuator cycle? (2) does it increase RSS by more than 50 MB? If the answer is "no" to the first or "yes" to the second, it goes to the V4.0 backlog.

Data Processing Pipeline

Data moves strictly unidirectionally:



Criticality Hierarchy (Tiers)

| Tier | Type | Failure Behavior |
|------|--|--|
| 0–1 | Core System (State Manager, Security, DB, Network) | Hard Reset of the entire system |
| 2–3 | Core Services (coordination, validation) | Degradation, but the core continues to operate |
| 4–5 | Business Logic (pipelines, scripts) | Restarted or terminated without harming the core |
| ≥4 | Isolated Modules (IsolatedRunner) | Killed via <code>SIGKILL</code> when limits are exceeded |

Isolated Modules (IsolatedRunner)

- Heavy tasks (NLP, LLM, parsing) run in separate OS processes (`multiprocessing.Process`).
- Interaction with the core occurs via `multiprocessing.Pipe` and `ThreadPoolExecutor` .
- Each isolator has its own PID in the OS; metrics are sent to the bus on a timer.
- **Requirements for placement in IsolatedRunner:** The module must have "dead-end" dependencies (the core does not depend on it) + the module resolves only one task.

Module Software Contract

| Class | Scope / Domain | Features / Characteristics |
|-----------------------|------------------------|---|
| BaseClass | Tier 0–2, base | Do not interact with the bus |
| SubotaInternalService | Internal core modules | Internal Redis stream only, Pydantic without HMAC |
| SubotaExternalService | External world modules | HMAC-SHA256, SecureEnvelope, Replay protection |

Каждый модуль обязан реализовать:

- `async def on_load(self)` — Initialization, reading config, registering dependencies.
- `async def on_unload(self)` — Graceful termination: clearing streams, saving pending transactions.
- `get_passport()` — Status (ACTIVE/STARTING/ERROR/OFFLINE), Tier, capabilities, routing topics, dependencies, proxy methods.

Domain Isolation (Bounded Contexts)

| Domain | Responsibility | Publishes | Reads |
|----------|---------------------------|---|----------------------|
| Sensors | Telemetry, Backpressure | Telemetry_Event | Nothing |
| Security | Threats, perimeter, bans | Threat_Level_Changed, Host_Ban_Command, Emergency_Event | Everything |
| Energy | Battery, UPS, modes | Power_State_Changed, Degradation_Trigger | Telemetry_Event |
| Identity | Biometrics, authorization | User_Authenticated, Auth_Failed, Voice_Command_Ingested | Audio, console |
| Memory | Vector DB, WAL, incidents | Data_Restored, Query_Result | All confirmed events |

Isolation Rule: Direct function calls and module imports between domains are forbidden. Communication occurs exclusively through the Event Bus.

Internal Loop (IPC)

Interaction between modules occurs exclusively via Redis (Pub/Sub for events, Streams for data). Direct calls are forbidden.

External Loop

Each incoming packet from peripherals (ESP32, Android) must contain:

1. HMAC-SHA256 signature.
2. `device_timestamp` (epoch in milliseconds according to the sensor clock).

Upon receipt, the core forcibly appends a `server_timestamp`. All business logic operates exclusively with `server_timestamp`. `device_timestamp` is used solely for network latency diagnostics.

If the delta (`server_timestamp - device_timestamp`) exceeds 5000 ms, the sensor is flagged with a "Time Anomaly", and a WARNING is sent to Postgres. Node ban occurs only if the delta > 5000 ms AND recurs more than 3 times consecutively. Packets without a valid HMAC signature or with an expired timestamp are destroyed before reaching the bus.

Protocol Versioning

- Each packet in the Event Bus contains a version field: `"v": 1`.
- The version is hardcoded into the MQTT topic: `subota/v1/sensors/perimeter/motion`.
- When sensor firmware is updated to v2, traffic flows to `subota/v2/...`, leaving old coroutines unaffected.
- A packet with a version higher than declared in `get_passport()` is ignored, and the incident is logged by `task_watcher`.

Single Source of Time (Time SSoT)

The Lenovo server is the sole NTP source for the Mikrotik_2 network. This is not Stratum 1, but a local NTP server (`chrony`) operating as an intermediate layer:

```
[External Internet NTP Servers]
  ↓ (Only in "Maximum" mode via Mikrotik_1)
[Lenovo Server / chrony]
  ↓ (Wi-Fi via Mikrotik_2)
[Peripherals: ESP32, Android]
```



In "Medium" and "Minimum" modes (no Internet), the server relies on its hardware RTC clock. Manual correction is done via the console. Direct peripheral access to the Internet for time synchronization is blocked on Mikrotik_2.

Replay Attack Protection: A packet with a timestamp drifting from the server's internal time by more than 2 seconds is destroyed, and the node is banned on Mikrotik_2.

Internet Availability Check (NetworkController)

The Captive Portal Check method is used (the same one used by Android and ChromeOS):

- HTTP GET to `[http://connectivitycheck.gstatic.com/generate_204]`
(`http://connectivitycheck.gstatic.com/generate_204`) (status 204 expected).
- HTTP GET to `[http://www.msftconnecttest.com/connecttest.txt]`
(`http://www.msftconnecttest.com/connecttest.txt`) (reference text).
- HTTP GET to `[http://captive.apple.com](http://captive.apple.com)`.

Requests are rotated. Total Internet loss is logged only when the entire pool fails. Intervals: nominal — 60 sec, during failure — 10 sec, during prolonged offline — 5 min.

6.2.7 6. Resource Management

RAM Budget

[Physical RAM: 8192 MB]

└─ Linux OS Reserve

└─ S.U.B.O.T.A. Budget

└─ Tier 0-5 (Core Loop)

└─ Docker Containers

└─ IsolatedRunner

└─ Episodic external services

└─ Reserve/buffer

→ 1200 MB

→ 4600 MB

→ max 1100 MB

→ max 1000 MB

→ max 2500 MB

→ max 1500 MB

→ 892 MB

Docker Limits:

| Container | Docker mem_limit | Internal Limit |
|------------------------|------------------|--|
| Redis | 150 MB | maxmemory 100mb , volatile-lru policy |
| PostgreSQL | 300 MB | shared_buffers=64MB , work_mem=8MB |
| Vector DB | 350 MB | Temporarily unallocated until deployment in V5.0 |
| MQTT + bots (combined) | 200 MB | ≤50 MB per container |

Zero-Swap Policy

Swap is strictly forbidden (`swaponoff -a`). If an active swap partition is detected during Tier 0 initialization, system startup is blocked, throwing a critical error to the console.

Rationale: Swapping `IsolatedRunner` pages to the HDD causes I/O Thrashing (latencies ranging from milliseconds to tens of seconds), which breaks the security loop. Continuous overwriting reduces disk lifespan.

CPU Priorities (Nice Levels)

| Component | Nice Level | Rationale |
|------------------------------|-------------------------|--|
| S.U.B.O.T.A. Core (Tier 0-5) | -20 (or chrt real-time) | Maximum priority, events processed first |
| Docker Containers | 0 | Standard |
| IsolatedRunner (Tier ≥4) | +15 | LLM does not block the core at 100% CPU load |

TaskWatcher (Tier 0) — Forced Resource Enforcement

Metric polling occurs every 1000 ms via `/proc/[pid]/stat` or `psutil`.

| Phase | Trigger | Action |
|-------------|---|--|
| Warning | RSS > 85% of quota | Sending <code>handle_degraded_mode()</code> to the violator — clearing intermediate caches |
| Liquidation | RSS > 100% of quota OR no healthcheck response >5000 ms | <code>SIGKILL</code> (<code>kill -9</code>), core continues operating |
| Recovery | After liquidation | Incident registration, restart allowed no earlier than after 10 sec |

If free RAM drops below 500 MB, Tier 4 processes, including the LLM, are forcibly terminated.

Latency Contracts

| Event | Maximum | Medium | Minimum |
|--|----------|-------------------------|----------|
| Critical security events (perimeter breach, fire, emergency button) | < 150 ms | < 200 ms | < 150 ms |
| Household telemetry (temperature, charge, logs) | < 500 ms | < 5000 ms (Batching) | Ignored |
| Intelligent loop (NLP/LLM, intent analysis) | < 500 ms | < 1500 ms (classic NLP) | Blocked |

Note: In "Minimum" mode, the latency for critical events is not lower than in "Maximum" — the Tier 0 core does not speed up when other modules are turned off, but simply gains more free resources. The < 150 ms guarantee is maintained.

During peak loads (dozens of sensors coming offline simultaneously): `task_watcher` suspends household metrics processing, allocating 100% of resources to critical security events.

6.2.8 7. Data Lifecycle

Storage Architecture (4 Layers)

- In-Memory (Redis)** — Event bus and hot state. Routing only, no long-term storage.
- PostgreSQL** — Long-term memory. Configs, RBAC, metrics, scheduler tasks. Logs are not in the DB.
- Vector DB** — Knowledge base embeddings, Cause-and-Effect vectors, incidents.
- SQLite (emergency_buffer)** — Tier 0 emergency buffer. Activated if Postgres/Redis fails.

Emergency Gate

Any critical write method is wrapped with the `@emergency_gate` decorator:

```
[ Write to DB ]
↓
```



The encrypted transaction is inaccessible during physical disk seizure without the `EmergencyService` private key.

WAL Policy

A custom software WAL at the Python code level does not exist. Functions are distributed:

- **PostgreSQL:** Built-in WAL (replica / minimal).
- **Redis:** RDB snapshots or AOF within memory limits.
- **S.U.B.O.T.A. Core:** `@emergency_gate` + `emergency_buffer` as a high-level WAL.

TTL and Data Rotation

| Data Type | Retention / Storage | Action Upon Expiration |
|---|---------------------------|--|
| Raw component logs (files) | 3 files × 10 MB
(FIFO) | Overwrite the oldest file |
| System events WARNING+
(Postgres) | 3 days (raw) | Aggregation into archival rows, raw entries deleted |
| Household sensor telemetry | 3 days (detailed) | Cascading aggregation: daily → weekly → monthly → yearly |
| Incident log | Indefinitely | — |

90% HDD Trigger: The system halts the collection of non-critical metrics and aggregates: 1,000 rows from the previous month into a single statistical row.

Aggregation triggers priority:

1. 90% HDD → Immediate aggregation, regardless of TTL.
2. TTL 3 days → Scheduled aggregation (at night, if power is available). In case of a conflict, 90% HDD takes priority.

Minor garbage (`INFO` , `DEBUG`) is not written to the Postgres database.

6.2.9 8. Security and Authentication

Disk Encryption

LUKS encryption of all HDD partitions (except `/boot`), AES-256 algorithm. The key is entered manually during Cold Boot and is not stored in plaintext in memory.

Principle of Least Privilege

S.U.B.O.T.A. runs under an unprivileged user `subota_runtime` without `sudo`. Code defects or hallucinations cannot damage the OS configuration.

Zero Trust

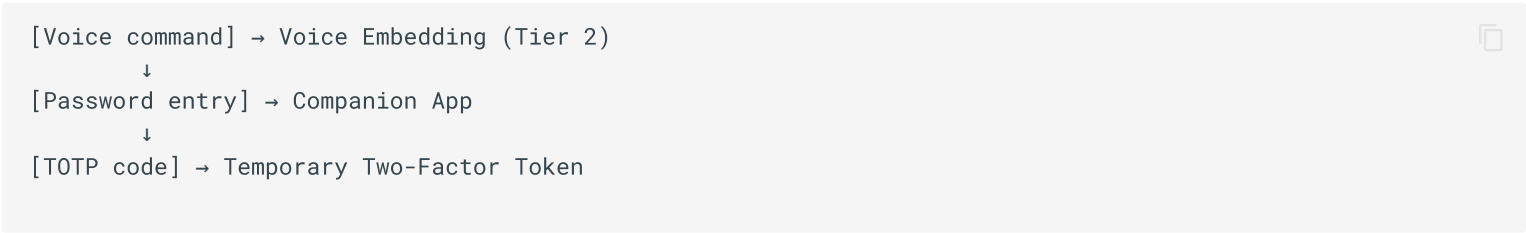
Every external request (bridge, sensor, API) is assumed hostile by default until it passes the Validation Layer. The MAC address is not an authentication factor – it is used only as a passive supplemental tag.

Trust Hierarchy

| Tier | Channel | Verification | Privileges / Rights |
|-----------------|-------------------------|--|---|
| 0
(Absolute) | Physical server console | Superuser password + TOTP | Full access: core, modules, reset |
| 1 (High) | Companion app / Web | Login + Password + TOTP | Full operational management |
| 2 (Medium) | Voice interface | VAD + FFT voice spectrum + voice embedding | Standard scenarios; critical via MFA |
| 3 (Basic) | Telegram bot | <code>unique_identifier</code> + RBAC | Metrics, home automation, notifications |

MFA for Critical Commands

CRITICAL_SECURITY class commands (disabling sensors, unlocking doors, resetting DB) require three-step verification:



Entry timeout is 60 seconds. Failure of any step leads to query cancellation, session termination, and an elevated alertness mode on the target node.

Biometric Identification Loop

- **Authorization:** Fingerprint (first line of defense) + voice embedding (second line of defense for critical commands).
- Facial recognition is used strictly for photo capturing of incidents, not for admission decisions.
- Compromise of a single peripheral sensor does not grant access to the core.

Paranoia Mode

Activated upon successful password verification if a biometric anomaly occurs (change in timbre, possible coercion, playback recording attempt).

Actions:

- Ban on configuration modifications, adding/removing modules, updating peripheral firmware, changing RBAC policies.
- Maximum covert logging: all commands are duplicated to a secured buffer; voice stream is recorded into `.wav` ; cameras capture a series of snapshots.
- At the end of the session — an encrypted anomaly report containing timeline, audio, and photos.
- Deception Protocol:** Upon receiving a critical command (e.g., "disable security"), the core mimics successful execution, returning a "Done" status to the user. No physical action happens at the Tier 0 level. The interface displays a virtualized "clean" picture. The real security loop continues running covertly.

6.2.10 9. State Machine and Degradation

SystemStateManager (Tier 0)

- Lightweight synchronous in-memory transmitter. It has no database of its own and does not write to disk.
- Upon state change, it broadcasts the event to all subscribers via the `on_state_change` hook without delay.

Responsibility Matrix

| Category | Automatically | Requires Confirmation | Strictly Forbidden |
|------------------------------|---|--|---|
| Data and Software Management | Log compression, LLM unloading, sensor key rotation | Deleting incident logs, resetting configurations | <code>eval()</code> , script rewriting, self-updating over the Internet |
| Network Security | Banning sensors on Mikrotik_2 upon anomalies, isolating Android boxes | Modifying routing rules on Mikrotik_2, banning nodes on Mikrotik_1 | Opening inbound ports on Mikrotik_1, disabling traffic logging |
| Physical Perimeter / Loop | Disconnecting non-critical Tier-4 loads (smart plugs) on battery drop | Room power relays, siren activation | Autonomous unlocking of electronic locks/doors, false alarm signal to external services |

Prohibition of Dynamic Code Generation

`eval()` , `exec()` , and background overwriting of `.py` files by the LLM are completely excluded. System evolution occurs via console suggestions:

"Sir, I have detected a regular need for task [X], but the current software base lacks the required adapter. Please consider writing a Python module..."

Implementation of a new automatic action passes through: "Suggestion" → "Manual approval in the console" → "Test period (10 triggered instances under supervision)".

6.2.11 10. Startup and Recovery Procedure

Initialization Sequence (5 Phases)

```
[Linux OS START]
↓
Phase 1: Database Containers —> [Postgres / Redis / Vector DB]
↓ (10 sec pause)
Phase 2: S.U.B.O.T.A. Core —> [State Manager / Security Center / Task Watcher]
↓ (5 sec pause)
Phase 3: Network Bridges —> [MQTT / Telegram / Discord / Web]
↓ (5 sec pause)
Phase 4: Peripheral Polling —> [Collecting states from all sensors - passive reading]
↓ (10 sec pause)
Phase 5: Heavy Processes —> [Analytics / LLM Inference (only in "Maximum" mode)]
```

- **Phase 1:** Postgres checks table integrity. Redis initializes In-Memory.
- **Phase 2:** `system_state_manager` sets the initial "Minimum" mode. `security_center` reads master keys. `task_watcher` launches thread monitoring.
- **Phase 3:** The system restores its ability to hear commands and send notifications.
- **Phase 4:** The core opens ports for traffic from Mikrotik_2. Sensor states are read, not overwritten.
- **Phase 5:** `system_state_manager` switches the system to nominal mode ("Medium" or "Maximum") depending on Internet availability. Launching heavy modules happens sequentially, with a step of several seconds, to avoid generating an overload avalanche on CPU and HDD.

Emergency Parking (5% Battery)

```
[Battery 20%]
↓
1. Notification via Telegram —> SOS message to the Operator
↓
[Battery 5%]
↓
2. Last Will to sensors —> Broadcast signal SYSTEM_SHUTDOWN_WARNING
                          Sensors switch to autonomous standby mode
↓
3. Transaction freeze —> @emergency_gate into hard lock mode
                          SQLite takes final snapshot
↓
4. Stop Docker —> docker stop -t 10 (Postgres flushes buffers, closes WAL)
↓
5. Shutdown —> shutdown -h now
```

Important: The Telegram bridge (Echelon 2) disconnects when the charge is < 20%. The notification at step 1 is guaranteed only if the bridge was active before parking initiated. If dropping from 20% to 5% without external power, the notification may not go through. This is a limitation of the current architecture.

6.2.12 11. Monitoring and Emergency Notification

Dead Man's Switch

Two levels of implementation (only the second is included in V3.0):

- **V3.0 — Local Buzzer:** An energy-efficient ESP32/Arduino with a physical buzzer in the Mikrotik_2 network. Every 30 seconds, the server sends a UDP Heartbeat: `{"status": "ALIVE", "timestamp": ...}`. If the buzzer does not receive a

pulse for 2 minutes, it alerts physically inside the house.

- **V4.0 — Out-of-band Watchdog:** A standalone device (Android tablet or ESP32) with an independent LTE/GSM modem and SIM card. It accepts a Heartbeat every 5 seconds. If silent for more than 30 seconds, it sends an SMS/push directly to the Operator's phone. The device is strictly a follower: it cannot reboot the server and has no access to the core. Configured exclusively from the server in nominal mode.

Lenovo Policy for a "Disconnected" Sensor

If a sensor does not respond to a ping for more than 10 minutes, its status becomes `LOST`. The system does not attempt to revive it with heavy packets (DDoS risk in the Mikrotik2 network). *An alert is issued to the Operator, and dependent scenarios switch to a pessimistic mode:* "hallway motion sensor is dead → consider the hallway a potential threat."_

External Notification Regulations (Telegram/Discord)

- **Ban on metadata:** No real names, geotags, or room titles.
- **Obfuscation:** All entities are replaced with hashed identifiers.
- *Example:* `[S.U.B.O.T.A. ID:01] ALERT: Triggered Zone_B (Sensor_E32_A4) → State: CRITICAL.`
- **Volume limit:** No more than 512 bytes per external packet (`GateKeeper` discards packets over the limit, and the incident is logged in `security_audit_log`).

6.2.13 12. V3.0 Readiness Criteria (Definition of Done)

The system is considered ready for V3.0 if all of the following conditions are met:

- **Autonomous Bootstrapping:** Full initialization (`ConfigManager` → `HealthCheck` → `Infra Setup` → `Module Loading`) without manual intervention.
- **Infrastructure Stability:** Watchdog and HealthCheck ensure Uptime > 99% during continuous operation for 7 days.
- **Secure Bridge:** `MQTT_Bridge` successfully passes HMAC authentication, operates in an isolated container, and transmits data without packet loss.
- **Policy Enforcement:** Correct switching of "Maximum"/"Minimum" modes during software simulation of Internet loss or exceeding CPU/RAM thresholds.
- **Minimal Functional Loop:** The chain `Sensor Input` → `Event Bus` → `Policy Engine` → `Actuator Execution` processes with an average latency of < 200 ms (on one real device: light or thermostat).

Out of Scope for V3.0 (not implemented, does not block the release)

- Any forms of Self-learning (on-the-fly learning).
- LLM/NLP models (except basic intent parsers).
- Dynamic device discovery (SSDP/mDNS — in V4.0).
- Integration of third-party cloud APIs.
- Graphical web interface (only a debug event monitor is allowed).

6.3 WORKPLAN — S.U.B.O.T.A.

The main rule for reading this file: **everything above the line** — **V3.0 RELEASE** — **is the path to the release. Everything below it must not be touched until the release.**

6.3.1 Constant Processes (Always Active)

- Writing and testing code
 - Updating `system_map.yaml` with every architectural change
-

6.3.2 Phase 1 — Base Stability **DONE**

- Unified ConfigManager 
 - Health Check — monitoring service availability 
 - Watchdog monitoring of internal processes 
 - Centralized Exception Handler 
 - Graceful shutdown and restart (SIGTERM, Ctrl+C) 
 - Configuration validation at startup 
 - Event logging system (JSON, rotation) 
 - PostgreSQL — universal long-term storage 
 - Secret Manager 
 - Redis Streams — centralized event queue 
-

6.3.3 Phase 2 — Base Functionality & Communications **DONE**

- Voice input (wake word, streaming processing) 
 - Network controller (ping, API availability, external IP) 
 - Telegram bot — two-way channel 
 - JWT authorization for API and web interface + rate limiting 
 - Notification system (Telegram) 
 - Base web interface (logs, device status, running scripts) 
 - Task scheduler with PostgreSQL integration 
 - API Gateway 
-

6.3.4 Phase 3 — Expansion & Developer Tools **DONE**

- pytest: unit and integration tests ✓
- Notification Center (real-time, filtering, history) ✓
- Access control system (roles: admin / user / guest) ✓
- Graceful degradation — operation during module unavailability ✓
- DB migrations (Alembic) ✓
- Temporal data system (metrics, time series) ✓
- System Health Dashboard ✓
- Backup and recovery ✓
- Versioning and dry-run for scripts ✓
- Advanced help system (CLI + UI) ✓

6.3.5 ✓ Phase 3.5 — Architectural Transformation **PARTIALLY DONE**

Completed as part of 3.5:

- `SubotaService` → `SubotaInternalService` + `SubotaExternalService` ✓
- `SecureEnvelope` + `SubotaMessage` (Pydantic schemas for two message types) ✓
- HMAC-SHA256 for external services ✓
- `EmergencyService` / `emergency_gate` / `EmergencyBuffer` — adapted for two streams ✓
- `IsolatedRunner` — moving heavy modules into separate processes ✓
- `ScriptRegistry` — asynchronous management of script documentation ✓
- `system_audit` logger — the system's black box ✓
- `DiscoveryManager` — discovering devices on the network ✓
- Shutdown exclusively via `asyncio.Queue` + logging in `audit_logger` ✓
- `ResourceGovernor` — canceled, replaced by `IsolatedRunner` ✓

Disabled / Canceled in 3.5:

- `scripts/alarm_handler`, `help_script`, `logger_script`, `task_scheduler` — excluded as unnecessary at this stage
- Phase 3.6 (mini-OS) — `#CANCELED`, replaced by process isolation

6.3.6 ■ Phase 4 — THE HOME STRETCH TO V3.0

This is the only thing standing between you and the release. Nothing extra.

4.0 — Fixing Known Bugs and Technical Debt ● **FIRST**

- [x] **BUG:** `weather_script/v2.py` line 148 — `IndexError: list index out of range` when `entities[]` is empty. Reason: `city = payload['data']['content']['entities'][0]` — no check for an empty list. Fix: guard-clause before accessing the index.

- [x] **HACK:** `core/simple_intent_mapper.py` — temporary intent mapping logic. Action: document explicitly (comment `# TODO: replace after NLP training`) and ensure that the hack does not break `intent_processor` in edge cases (empty input, unknown intent).
- [x] **SECURITY:** Complicate login via API Gateway (FastAPI). Minimum: rate limiting on `/login`, IP banning after N failed attempts, record in `security_audit_log`.
- Notes: IP Banning: Dropping this feature is a deliberate architectural decision, not a missed task. For a local network with dynamic IPs (DHCP), automatic hard banning would bring more self-blocking problems than real benefit. It has been replaced with a soft time limit (via `slowapi`), which completely solves the security issue without complicating life.
- [x] **SCHEDULER:** Add handling for recurring tasks in `task_scheduler_script.py`. Without this, `task_scheduler` cannot be considered functional for release.

4.1 — Secure Bridge (MQTT + sensor network isolation) ●

Without this, there is no real periphery, which means there is no security system — only virtual sensors.

- [x] Deploy Mosquitto in Docker (port 8883, TLS).
- [x] Write MQTT Bridge: A Python service that pipes packets from MQTT to Redis.
- Rate Limiter on incoming packets from the sensor.
- Appending an HMAC signature before sending to `subota:external:events`.
- [x] Configure `Mikrotik_2`: ACL, isolating IoT devices from `Mikrotik_1`.
- [x] Configure Legacy Android (Termux): data transmission only, no incoming commands without a signature.

4.2 — PowerDaemon (survival during a blackout) ●

Without this, the system doesn't know it is dying. This blocks DoD item 4 (Policy Enforcement).

- [x] `PowerDaemon` — reads `/sys/class/power_supply/`, monitors battery charge.
- [x] Thresholds and hysteresis:
- 25% (stable for 60 sec) → transition to "Middle" mode, unloading LLM from memory.
- 20% → transition to "Minimum" mode, disabling Tier ≥ 4.
- 5% → emergency parking procedure (notification → Last Will to sensors → SQLite snapshot → docker stop → shutdown).
- [x] DTEK parser integration: upon power loss — request fresh schedule, compare with current time, notify operator.
- [x] `SQLite Checkpoint Store`: background recording of State snapshots to SQLite on every mode change.

4.3 — GateKeeper (Internal ↔ External gateway) ●

Does not block the launch, but without it, the boundary between streams relies solely on trust.

- [x] Write `GateKeeper` — a shim module between `subota:internal:events` and `subota:external:events`.
- Incoming packet validation: schema + HMAC + timestamp drift ≤ 2 sec.
- Packet without a valid signature — drop + record incident in `security_audit_log`.
- Replay protection: cache of the last N timestamp/HMAC pairs.

4.4 — IsolatedRunner: transition to Redis IPC ●

Currently, isolated modules (TTS, VOSK, NLP) communicate directly via Pipe. For V3.0 it is sufficient to lock down the working state, but the issue of transferring audio files needs to be closed.

- [] Resolve the issue of transferring the voice file between `IsolatedRunner` and the main process. Options: temporary file on disk with the path passed via Redis / Unix Domain Socket. Choose one option, document, implement.
- [] Ensure that `MetricsCollector` does not just log metrics of isolated processes, but sends them to Redis (for `task_watcher`).

4.5 — Core Protection from OOM and swap

- [] Add a check during Tier 0 initialization: if swap is active — halt startup with an explicit error to the console.
- [] Configure `cgroups` limits for Docker containers according to the RAM budget from the Tech Spec.

4.6 — Final DoD Verification (Definition of Done V3.0) LAST STEP

All 5 criteria must pass on real hardware (Lenovo + at least one physical sensor).

- [] **DoD 1 — Autonomous Bootstrapping:** startup without manual intervention from ConfigManager to Module Loading.
- [] **DoD 2 — Infrastructure Stability:** Uptime > 99% during continuous operation for 7 days.
- [] **DoD 3 — Secure Bridge:** MQTT Bridge passes HMAC authentication, is isolated in a container, and does not lose packets.
- [] **DoD 4 — Policy Enforcement:** correct switching of "Maximum"/"Minimum" modes during software simulation of internet loss and CPU/RAM threshold breach.
- [] **DoD 5 — Minimal Functional Loop:** Sensor → Event Bus → Policy Engine → Actuator with latency < 200 ms on one real device (light or thermostat).

6.3.7 ————— V3.0 RELEASE —————

Everything below — do not touch until passing the DoD. Items may change.

V4.0 — Infrastructure Expansion and UX

From Postponed 3.5:

- [] LTR579 value range validation.
- [] Data buffering upon Wi-Fi loss.
- [] Cleaning and compressing packet size.
- [] **Intent Engine / IntentBroker:** Intent → StatusObserver → Execution router. Support for priorities (CRITICAL / SYSTEM / USER / BACKGROUND).
- [] **StatusObserver:** registry of alive modules. If a module is asleep — wake up or place into the `PendingTasks` queue.
- [] **File Broker:** Unix Domain Sockets or mmap for binary data transmission (voice, images). Redis should not see files. (Currently covered by a temporary solution in 4.4.)
- [] **KMS Watchdog:** transition to `pyudev`, monitoring physical key extraction. Instant SIGTERM upon extraction.

- [] **Redis ACL:** generating `redis.conf` with users (`core_user` , `bridge_user` , `observer_user`) at KMS startup, instead of internal HMAC.

Tools and UX:

- [] Script debugging tools (tracing, advanced logs).
- [] A/B testing of scripts — safe testing of new versions.
- [] Admin Panel:
- Script editor (Monaco Editor) with syntax highlighting.
- Device Manager UI.
- User permissions management (UI).
- Interactive console (Chat interface).
- [] Physical USB remote (Arduino over Serial, security status indication).

Logic and Scenarios:

- [] Virtual sensors (`VirtualContactSensor`) for hardware-free testing.
- [] Aggregation of old data: Retention Policy, AUTOVACUUM for `system_metrics` .
- [] Plugin system: `manifest.json` for modules.
- [] Dialogue context: relative commands ("make it warmer").
- [] Contextual command chains.
- [] Voice messages in Telegram: sending/receiving audio.
- [] Device discovery (UPnP / mDNS) with quarantine for new nodes.

Security:

- [] TOTP (Google Authenticator) for critical commands via Telegram/voice.
- [] `PresenceEngine` : Fusion sensors (Wi-Fi + BT + Motion), protection against false positives.
- [] `MikrotikScanner` : `known_devices` table, `unrecognized_device_detected` alert.
- [] `UpdateManager` : atomic updates, hash checking, anti-rollback.
- [] Docker Hardening: network separation, non-root user, key rotation.
- [] **Out-of-band Watchdog**: a separate device with LTE/GSM, independent of the server. Receives Heartbeats, if silent for > 30 sec — sends SMS to the Operator. Strictly subordinate: cannot reboot the server.

Integrations:

- [] UPS Integration (NUT). Signal from UPS → notification → graceful Docker shutdown when charge < 20%.
- [] Geofencing: automatic mode switching based on phone geolocation (push from a companion app, data does not leave the local network).
- [] Zigbee (USB stick + Zigbee2MQTT) — consider as a replacement for Wi-Fi sensors.

- [] **Local LLM Inference:** first measure CPU inference on real hardware (Phi-3 Mini Q4 / Gemma 2B Q4). Track latency. Only after measurements — make a decision on deployment or replacing the GPU.
 - [] **Vector Long-Term Memory:** pgvector or a separate vector DB. TTL for vectors. Semantic search over command history.
 - [] **NLP Expansion:** new intents, fine-tuning on user data.
 - [] **Local API Cache.**
 - [] **RAG:** LLM + vector memory to answer questions about past events.
 - [] **Voice Sentiment Analysis.**
 - [] **Resource Forecasting:** predicting CPU/RAM load.
 - [] **Nightly Reflection (learning cycle):**
 - `Reflector Agent` launches at 03:00 if power is available.
 - Analysis of `IntentLog` + `ContextStore`.
 - Generation of `script_candidate`.
 - Human-in-the-loop: proposal in Telegram/WebUI → script moves to `scripts/` only after `/approve`.
 - [] **Machine Spirit / The Trinity (FSM of contexts):**
 - Formalization of triggers for switching `SubotaContext` → `AlphariusContext`.
 - Device status table: Loyal / Shrouded / Apostate / In Meditation.
 - Bind `S_factor` (noise coefficient) to operational modes from Tech Spec.
 - `HydraWhisper` — logger decorator.
 - Voice profiles for three contexts.
-

V6.0 — Distant Future

- [] Physical independence of the Watchdog: its own backup power source, satellite/LoRa communication channel, local storage for the last 7 days of critical events.
 - [] Instance federation: multiple physically independent nodes, synchronization without a central server.
 - [] Auto-restart of Docker containers (healthcheck-based).
 - [] First-run script (Wizard).
 - [] Native Android Applications:
 - "Station" mode (Vosk, WebSocket stream).
 - "Remote" mode (biometrics, mDNS discovery).
 - Global notification synchronization (history in DB, "read" status across devices).
 - [] Full Duplex (barge-in mode): VAD runs constantly, Cancel Token for speech synthesis when user begins speaking.
 - [] Multimodality (Vision Language Models): camera frame → "a courier is at the door."
 - [] "Self-writing" of scripts: behavior pattern analysis → automation proposal.
 - [] Self-Healing Loop: Watchdog + LLM analyzes traceback → suggests optimization → user confirms.
-

6.3.8 Margin Notes (No Version, Keep in Mind)

- **Web Interface:** connect history and system performance statistics; ensure all errors and messages are visible on the web interface.
- **Associative Memory:** pgvector + command history = semantic search. "Turn on the lights where we have dinner" → "kitchen".
- **Zigbee vs Wi-Fi:** Wi-Fi sensors clutter the airwaves and depend on clouds. Zigbee is a mesh, lasts years on a battery, entirely local. Fits right into the already laid out MQTT infrastructure.
- **DTEK Parser:** there is a ready-made database in `old files`. Logic: power goes out → request schedule → compare with current time → calculate Time-to-Die → notification. Bring to a functional state in V4.0.